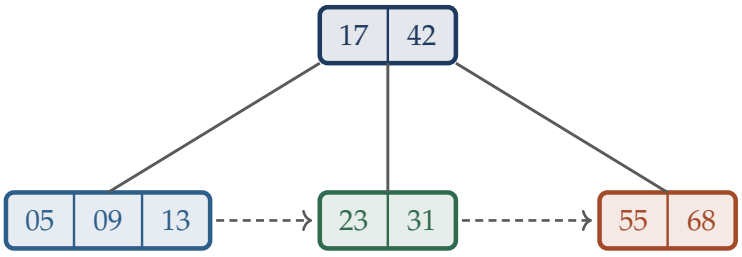
INFORMATIK VON GRUND AUF — BAND 10

# Das Gedächtnis der Systeme

## Eine vollständige Einführung in Datenbanken und Information Retrieval




*verkettete Blätter*

Vom Datensatz bis zur Rangfolge

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

**Informatik von Grund auf** — Band 10
*Das Gedächtnis der Systeme* · Datenbanken und Information Retrieval

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

**Lizenz**
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

**Quellen und Fassungen**
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

**Zitiervorschlag**
Maier, J. (2026). *Informatik von Grund auf, Band 10: Das Gedächtnis der Systeme*. Fassung 1.0.1. `https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 162 Seiten.

Stand der Forschung: August 2026

# Inhaltsverzeichnis


Vorwort 1

I Modell und Sprache 9

1 Wozu ein Datenbanksystem 11

2 Das relationale Modell 21

3 SQL 29

4 Entwurf und Normalformen 37

II Wie es innen arbeitet 45

5 Speicherung 47

6 Indexstrukturen 55

7 Anfrageausführung 65

8 Anfrageoptimierung 73

III Gleichzeitigkeit und Dauerhaftigkeit 83

9 Transaktionen und Isolation 85

10 Nebenläufigkeitskontrolle 93

11 Wiederanlauf 103

IV Finden statt Abfragen 111

12 Der invertierte Index 113

13 Rangfolge 121

14 Verbindungen und Vektoren 129

**15 Der Stand im Jahr 2026** **139**

**V Anhang** **145**

**A Zahlen und Formeln** **147**

**B Landkarte des Bandes** **149**

**C Glossar** **151**

**D Zeittafel** **153**

**E Kommentierte Quellen** **155**

# Vorwort: Was sagen und nicht wie

**Dieser Band handelt von der Frage, wie man Daten so ablegt, dass man sie wiederfindet.** *Das klingt bescheiden und enthält fast alles, was in den vorigen Bänden entwickelt wurde: Datenstrukturen, Nebenläufigkeit, Dauerhaftigkeit, Übersetzerbau, Verteilung.*

### Zwei Wege zu derselben Auskunft

Gesucht sind alle Kunden aus Köln, die im letzten Quartal mehr als tausend Euro umgesetzt haben.

```
SELECT k.name, SUM(p.betrag) AS umsatz
FROM kunde k JOIN posten p ON p.kunde
= k.nr
WHERE k.ort = 'Köln' AND p.datum >=
'2026-04-01'
GROUP BY k.nr, k.name
HAVING SUM(p.betrag) > 1000;
```

> *In diesen fünf Zeilen steht,* was *gesucht ist. Es steht nirgends, in welcher Reihenfolge gelesen wird, welcher Index benutzt wird, wie die Verknüpfung ausgeführt wird oder ob dabei sortiert oder gestreut wird.*

**Die andere Fassung.**

```
OPEN kunde INDEX ort
SEEK 'Köln'
WHILE kunde.ort = 'Köln'
  summe = 0
  OPEN posten INDEX kundennr
  SEEK kunde.nr
  WHILE posten.kunde = kunde.nr
    IF posten.datum >= ...  THEN summe
+= posten.betrag
    NEXT posten
  WEND
  IF summe > 1000 THEN PRINT
kunde.name, summe
  NEXT kunde
WEND
```

*Diese Fassung sagt genau, wie es zu geschehen hat. Sie ist länger, sie legt die Indexwahl fest, und sie ist an eine bestimmte Datenorganisation gebunden. Wer diese Art zu programmieren kennt, erkennt hier die satzorientierte Verarbeitung aus Band 5 wieder.*

| | **beschreibend** | **satzorientiert** |
|---|---|---|
| Sagt | **was** | **wie** |
| Indexwahl trifft | das System | **der Programmierer** |
| Bei neuem Index | schneller, ohne Änderung | Code umschreiben |
| Bei 10-facher Datenmenge | anderer Plan | derselbe Plan |
| Vorhersagbarkeit | gering | hoch |

*Die letzte Zeile ist der ehrliche Nachteil der beschreibenden Fassung, und er wird gern unterschlagen. Wenn der Optimierer seine Meinung ändert, wird eine Abfrage über Nacht hundertmal langsamer, ohne dass jemand etwas geändert hätte.*

### Warum nicht einfach Dateien

Band 5 hat gezeigt, wie ein Dateisystem arbeitet. Warum genügt das nicht?

| | |
|---|---|
| **Struktur** | Eine Datei ist eine Bytefolge; die Bedeutung kennt nur das Programm |
| **Nebenläufigkeit** | Mehrere Schreiber brauchen Sperren; wer setzt sie richtig? |
| **Dauerhaftigkeit** | Was gilt nach einem Stromausfall mitten in einer Änderung? |
| **Wiederfinden** | Ohne Index bleibt nur das vollständige Durchsuchen |
| **Regeln** | Eine Bestellung ohne Kunden darf es nicht geben |
| **Unabhängigkeit** | Ändert sich die Ablage, müssen alle Programme geändert werden |

*Jede dieser Aufgaben lässt sich von Hand lösen. Ein Datenbanksystem ist die Beobachtung, dass jede Anwendung sie sonst erneut lösen müsste, und zwar meist schlechter.*

**Die letzte Zeile ist die wichtigste.**

| | |
|---|---|
| **Physische Unabhängigkeit** | Der Index ändert sich, die Abfragen bleiben |
| **Logische Unabhängigkeit** | Die Tabellen ändern sich, die Sichten darüber bleiben |

*Das war 1970 der eigentliche Streitpunkt, und Codds Aufsatz beginnt genau damit. Nicht die Tabellenform war das Neue, sondern die Zusage, dass sich die Ablage ändern darf, ohne dass Programme brechen.*

## Was dieses Buch will

**Erstens: Die Ebene wechseln, statt bei einer zu bleiben.**
Man kann Datenbanken benutzen, ohne zu wissen, was drinnen geschieht. Man kann auch die Innereien studieren, ohne je eine Abfrage geschrieben zu haben. Beides führt zu schlechten Ergebnissen.

> *Ich gehe deshalb bei jedem Thema beide Wege: erst die Sicht des Benutzers, dann die Umsetzung darunter, und dann die Frage, was daraus für die Benutzung folgt.*

**Zweitens: Die zweite Hälfte ernst nehmen.**
Datenbanken beantworten Fragen, deren Antwort feststeht. Suchmaschinen beantworten Fragen, deren Antwort eine Rangfolge ist. Teil IV behandelt das, und der Unterschied ist grundlegender als er aussieht.

| | **Datenbank** | **Suche** |
|---|---|---|
| Antwort | **richtig oder falsch** | **mehr oder weniger passend** |
| Ergebnis | eine Menge | eine **geordnete** Liste |
| Fehlt etwas | Fehler | normal |
| Gütemaß | — | Precision, Recall |

**Drittens: Die Zahlen mitliefern.**
Wie viele Seitenzugriffe kostet eine Verknüpfung? Wie hoch ist ein B-Baum über einer Milliarde Sätzen? Wie viel bringt ein Index wirklich? Ich rechne das durch, weil sich die meisten Entwurfsfragen damit von selbst beantworten.

### Drei Warnungen

**Erstens: Die Standardeinstellung ist selten die richtige.**
Fast jedes Datenbanksystem läuft ab Werk mit einer Isolationsstufe, die Anomalien zulässt, von denen die meisten Anwender nichts wissen. Kapitel 9 zeigt, welche.
**Zweitens: Ein Index ist nicht umsonst.**

> *Jeder Index beschleunigt das Lesen und verlangsamt jedes Schreiben. Eine Tabelle mit zwölf Indizes ist kein sorgfältig optimiertes System, sondern meist ein Zeichen dafür, dass niemand aufgeräumt hat.*

**Drittens: Messen, nicht raten.**
Jedes ernsthafte System zeigt seinen Ausführungsplan an. Wer eine langsame Abfrage vor sich hat und diesen Plan nicht angesehen hat, rät. Kapitel 8 zeigt, wie man ihn liest.

## Zum Aufbau

- **Teil I** (Kap. 1–4): Modell und Sprache. Wozu ein Datenbanksystem, das relationale Modell, SQL, Entwurf und Normalformen.
- **Teil II** (Kap. 5–8): Wie es innen arbeitet. Speicherung, Indexstrukturen, Ausführung, Optimierung.

- **Teil III** (Kap. 9–11): Gleichzeitigkeit und Dauerhaftigkeit. Transaktionen, Nebenläufigkeitskontrolle, Wiederanlauf.
- **Teil IV** (Kap. 12–15): Finden statt Abfragen. Invertierte Indizes, Rangfolge, Verbindungen und Vektoren, und der Stand im Jahr 2026.

### Ein Aufsatz, den die eigene Firma ignorierte

EDGAR F. CODD arbeitete 1970 im IBM-Forschungslabor in San Jose. Er veröffentlichte einen Aufsatz von elf Seiten mit dem Titel „A Relational Model of Data for Large Shared Data Banks“.

> *Sein Ausgangspunkt war ein Ärgernis: Die damaligen Systeme verlangten, dass Programme die Zugriffspfade kannten. Wer eine Datei anders organisierte, musste jedes Programm ändern, das sie benutzte.*

| | **damals üblich** | **Codds Vorschlag** |
|---|---|---|
| Daten sind | Sätze mit Zeigern | **Tabellen** |
| Zugriff über | den Weg dorthin | **den Inhalt** |
| Programm kennt | die Struktur | nur das Schema |
| Grundlage | Datenverarbeitung | **Mengenlehre** |

**Der Widerstand.**
IBM hatte Millionen in IMS investiert, ein hierarchisches System, das für das Apollo-Programm entstanden war und gut lief. Ein Forscher, der behauptete, das alles sei der falsche Weg, war nicht willkommen.

> *Es kam hinzu, dass Codds Vorschlag als unpraktikabel galt. Eine Sprache, die nur sagt,* was *man will, verlangt ein System, das den Weg selbst findet. Die verbreitete Ansicht lautete, ein solches System könne niemals mit handoptimiertem Zugriff mithalten.*

Das ist derselbe Einwand, den Backus 1957 gegen FORTRAN zu hören bekam (Band 6). Und er wurde auf dieselbe Weise entkräftet: Man baute es, und es war gut genug.

| | |
|---|---|
| 1970 | Codd: das relationale Modell |
| 1973 | System R beginnt bei IBM San Jose |
| 1974 | Chamberlin, Boyce: SEQUEL |
| 1974 | Öffentliche Debatte Codd gegen Bachman |
| 1976 | Ingres in Berkeley, unabhängig |
| 1979 | Oracle liefert vor IBM aus |
| **1979** | **Selinger u. a.: der kostenbasierte Optimierer** |
| 1981 | Codd erhält den Turing-Award |
| 1986 | SQL wird genormt |

*Die hervorgehobene Zeile ist die eigentliche Antwort auf den Einwand. Solange der Optimierer schlecht war, hatten die Kritiker recht. Selingers Arbeit zeigte, wie man aus Statistiken über die Daten einen guten Plan errechnet, und damit war die Sache entschieden.*

**Eine Pointe zum Schluss.** Codd hatte eine Sprache namens ALPHA vorgeschlagen, die enger an der Logik war. Durchgesetzt hat sich SEQUEL, später SQL, das er selbst für einen unsauberen Kompromiss hielt.

*Er hat sich mehrfach kritisch dazu geäußert, unter anderem wegen der Behandlung von Duplikaten und von fehlenden Werten. In beiden Punkten hatte er recht, und Kapitel 3 kommt darauf zurück.*

## Sechzig Jahre Datenbanken und Suche

| | | |
|---|---|---|
| 1961 | Bachman | IDS, erstes Datenbanksystem |
| 1966 | IMS | hierarchisch, für Apollo |
| 1968 | Salton | SMART: Vektorraummodell |
| **1970** | **Codd** | **Das relationale Modell** |
| **1972** | **Bayer, McCreight** | **B-Bäume** |
| 1972 | Codd | Normalformen |
| 1974 | Chamberlin, Boyce | SEQUEL |
| 1975 | Salton u. a. | Vektorraum, formal |
| 1976 | Chen | Entity-Relationship-Modell |
| **1979** | **Selinger u. a.** | **kostenbasierte Optimierung** |
| 1981 | Codd | Turing-Award |
| **1983** | **Härder, Reuter** | **ACID** |
| 1986 | SQL-86 | erste Norm |
| 1987 | Bernstein u. a. | Nebenläufigkeit und Wiederanlauf |
| 1992 | Mohan u. a. | ARIES |
| 1993 | Graefe | Ausführungsverfahren im Überblick |
| 1994 | Robertson u. a. | BM25 |
| 1995 | Berenson u. a. | Kritik der Isolationsstufen |
| 1996 | O'Neil u. a. | Protokollstrukturierter Mischbaum |

| | | |
|---|---|---|
| **1998** | **Brin, Page** | **PageRank** |
| 1999 | Kleinberg | HITS |
| 2003 | Google | MapReduce, GFS |
| 2006 | Bigtable | |
| 2007 | Stonebraker u. a. | „Das Ende einer Architekturepoche“ |
| 2007 | Dynamo | Verfügbarkeit vor Konsistenz (Band 9) |
| 2009 | „NoSQL“ | Sammelbegriff setzt sich durch |
| 2010 | Spaltenspeicher | im Massengebrauch |
| **2012** | **Spanner** | **verteilt und streng konsistent** |
| 2013 | Abadi u. a. | Spaltenspeicherung im Überblick |
| 2016 | Einbettungen | Vektorsuche beginnt |
| 2018 | Malkov, Yashunin | HNSW |
| 2020er | Trennung | Speicher und Rechenwerk getrennt |
| 2023 | Vektordatenbanken | eigene Produktklasse |
| 2026 | — | SQL und Vektoren wachsen zusammen |

Zwei Zeilen verdienen eine Anmerkung.

**1972, B-Bäume.** Rudolf Bayer und Edward McCreight beschrieben eine Baumform, die auf Blockspeicher zugeschnitten ist. Sie ist über fünfzig Jahre alt, praktisch unverändert im Einsatz und steckt in jedem Datenbanksystem und in den meisten Dateisystemen aus Band 5.

**2007, Stonebraker.** Er argumentierte, die klassische Architektur stamme aus den 1970er Jahren und passe nicht mehr zu Rechnern mit großem Hauptspeicher. Die These war überzogen und hat die Entwicklung der folgenden fünfzehn Jahre erheblich beeinflusst.

*Daten überleben Programme.*

— ein Satz, der in verschiedenen Fassungen umläuft
*und erklärt, warum ein sauberes Datenmodell wichtiger ist als eine saubere Anwendung: Die Anwendung wird in zehn Jahren neu geschrieben, die Daten nicht.*

# Part I

# Modell und Sprache

# 1 Wozu ein Datenbanksystem

### Die vier Zusagen

Ein Datenbanksystem gibt Zusagen, die ein Dateisystem nicht gibt. Sie sind unter dem Kürzel ACID bekannt und stammen in dieser Form von THEO HÄRDER und ANDREAS REUTER, 1983.

| | |
|---|---|
| **Atomarität** | Eine Transaktion geschieht ganz oder gar nicht |
| **Konsistenz** | Sie führt von einem gültigen Zustand in einen gültigen |
| **Isolation** | Sie läuft, als wäre sie allein |
| **Dauerhaftigkeit** | Was festgeschrieben ist, überlebt einen Absturz |

*Die vier Buchstaben sind ungleich gewichtet. Atomarität und Dauerhaftigkeit sind Eigenschaften des Systems. Isolation ist einstellbar und wird in der Praxis fast immer abgeschwächt. Und Konsistenz ist eigentlich eine Zusage der Anwendung, nicht des Systems.*

**Der Klassiker.**

```
BEGIN;
UPDATE konto SET stand = stand - 100
WHERE nr = 4711;
UPDATE konto SET stand = stand + 100
WHERE nr = 815;
COMMIT;
```

| Absturz nach | ohne Atomarität | mit |
|---|---|---|
| Zeile 2 | 100 Euro weg | nichts geschehen |
| Zeile 3 | richtig | richtig |
| `COMMIT` | richtig | richtig |

*Ohne Atomarität kann Geld verschwinden. Das ist kein seltener Grenzfall, sondern tritt bei jedem Stromausfall auf, der zufällig in dieses Zeitfenster fällt. Bei Millionen von Buchungen am Tag ist das eine Frage der Zeit.*

**Isolation ist selten vollständig**

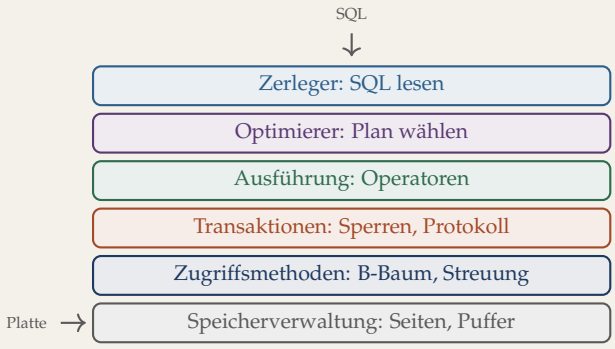
Die Norm kennt vier Stufen, und die meisten Systeme laufen ab Werk nicht auf der höchsten.

| System | Voreinstellung |
|---|---|
| PostgreSQL | `READ COMMITTED` |
| Oracle | `READ COMMITTED` |
| MySQL (InnoDB) | `REPEATABLE READ` |
| SQL Server | `READ COMMITTED` |

*Auf keiner dieser Stufen gilt die Zusage „als wäre sie allein“ vollständig. Es sind Anomalien möglich, die die meisten Anwendungsentwickler nicht kennen, weil sie selten auftreten und schwer zu reproduzieren sind. Kapitel 9 führt sie einzeln vor.*

*Das ist keine Schlamperei der Hersteller. Vollständige Isolation kostet erheblich, und für die meisten Anwendungen ist die Abschwächung tragbar. Unangenehm ist nur, dass sie stillschweigend geschieht.*

### Der Aufbau eines Datenbanksystems




| Schicht | Aufgabe | Kapitel |
|---|---|---|
| Zerleger | Text zu Baum | wie Band 6 |
| Optimierer | Plan wählen | 8 |
| Ausführung | Plan abarbeiten | 7 |
| Transaktionen | Sperren, Protokoll | 10, 11 |
| Zugriffsmethoden | Sätze finden | 6 |
| Speicher-verwaltung | Seiten und Puffer | 5 |

*Die oberen drei Schichten sind ein Übersetzer, wie er in Band 6 beschrieben wurde: lesen, prüfen, optimieren, ausführen. Die unteren drei sind ein Betriebssystem im Kleinen, wie in Band 5: Puffer, Sperren, Protokolle, Wiederanlauf.*

**Warum ein Datenbanksystem das Betriebssystem oft umgeht.**

| | |
|---|---|
| **Eigener Puffer** | Das System weiß besser als der Kern, welche Seite es gleich wieder braucht |
| **Direkte Ein-/Ausgabe** | Doppelte Pufferung wird vermieden |
| **Eigene Ablaufplanung** | Ein wartender Faden je Verbindung skaliert schlecht |
| **Eigene Dateiverwaltung** | Zusammenhängende Bereiche statt vieler kleiner Dateien |

*Die erste Zeile ist der wichtigste Punkt. Der Kern verdrängt nach dem Uhrzeigerverfahren (Band 5, Kapitel 5) und kennt die Zugriffsmuster nicht. Ein Datenbanksystem weiß, dass es gerade einen Durchlauf über eine große Tabelle macht, und darf diese Seiten sofort wieder verwerfen.*

### Wann sich ein Datenbanksystem nicht lohnt

| Fall | Besser | Grund |
|---|---|---|
| Einstellungsdatei | Datei | kein gemeinsamer Zugriff |
| Protokolldateien | Anhängen an Datei | nur schreiben, sequentiell |
| Sehr große Einzelobjekte | Objektspeicher | Bilder, Video |
| Zwischenspeicher | Schlüssel-Wert im Speicher | Dauerhaftigkeit unnötig |
| Ein Programm, eine Datei | Datei | **Zusagen nicht gebraucht** |

*Die letzte Zeile fasst es zusammen. Ein Datenbanksystem löst Probleme, die aus gemeinsamem, gleichzeitigem Zugriff und aus dem Bedarf an Zusagen entstehen. Ohne beides bezahlt man Aufwand für nichts.*

**Die Gegenrechnung.**

| Vorgang | Datei | Datenbank |
|---|---|---|
| Einen Satz anhängen | $\approx 1\,\mu s$ | $\approx 50\,\mu s$ |
| Einen Satz nach Schlüssel | $\mathcal{O}(n)$ | $\approx 3$ Zugriffe |
| Bei Absturz | unklar | definiert |
| Zwei Schreiber | Wettlauf | serialisiert |

*Fünfzigmal langsamer beim einfachen Anhängen, und beliebig viel schneller beim Suchen. Welche Zeile zählt, entscheidet die Anwendung und nicht die Vorliebe.*

### Aufgabe 1.1.

(a) Zeigen Sie, was ohne Atomarität geschieht.
(b) Rechnen Sie aus, was ein fehlender Index kostet.
(c) Untersuchen Sie, warum Datenbanksysteme eigene Puffer führen.

### Lösung Schritt für Schritt.

**(a) Ohne Atomarität.**
**Der Fall.** Eine Umbuchung besteht aus zwei Änderungen. Der Strom fällt dazwischen aus.

| Zeit | Vorgang | Konto 4711 | Konto 815 |
|---|---|---|---|
| $t_0$ | Ausgangslage | 1000 | 500 |
| $t_1$ | erste Änderung | 900 | 500 |
| $t_2$ | Stromausfall | 900 | 500 |
| | **Summe vorher** | 1500 | |
| | **Summe nachher** | 1400 | |

> *Hundert Euro sind verschwunden, und niemand bemerkt es sofort. Erst der nächste Abgleich der Summen fällt auf, und dann ist unklar, wann und warum.*

**Wie oft das vorkäme.**
Nehmen wir eine Umbuchung, deren kritisches Fenster zwei Millisekunden umfasst, bei einer Million Umbuchungen am Tag und einem unerwarteten Neustart im Monat.

$$P = \frac{10^{6} \cdot 0{,}002\ \text{s}}{86\,400\ \text{s}} = 0{,}0231.$$

| Größe | Wert | |
|---|---|---|
| Wahrscheinlichkeit je Absturz | 2,3 % | |
| Abstürze je Jahr | 12 | |
| Erwartete Vorfälle | ≈ 0,28/Jahr | |
| In zehn Jahren | ≈ 3 | drei verlorene Buchungen |

*Selten genug, um beim Testen nie aufzufallen, und häufig genug, um im Betrieb wirklich zu passieren. Genau diese Kombination macht solche Fehler so unangenehm; sie ist dieselbe wie beim Wettlauf aus Band 5.*

**Was das System stattdessen tut.**

1. Vor jeder Änderung wird ins Protokoll geschrieben, was sich ändert.
2. Beim `COMMIT` wird ein Festschreibungssatz protokolliert und auf die Platte gezwungen.
3. Beim Wiederanlauf werden festgeschriebene Transaktionen wiederholt.
4. Nicht festgeschriebene werden **zurückgerollt**.

*Es ist dasselbe Verfahren wie das Journal eines Dateisystems aus Band 5, Kapitel 12, nur feiner und mit der Möglichkeit, gezielt zurückzurollen. Kapitel 11 führt es vollständig durch.*

**(b) Was ein fehlender Index kostet.**
**Die Tabelle.** Zehn Millionen Sätze zu je 200 Byte, Seitengröße 8 KB.

| Größe | Wert | |
|---|---|---|
| Sätze je Seite | 40 | 8192/200 |
| Seiten insgesamt | 250 000 | |
| Tabellengröße | 2 GB | |

**Vollständiger Durchlauf.**

| Medium | Rate | Zeit |
|---|---|---|
| Festplatte, sequentiell | 150 MB/s | 13,3 s |
| SSD | 2 GB/s | 1,0 s |
| Hauptspeicher | 20 GB/s | 0,1 s |

**Mit B-Baum-Index.**
Die Höhe eines B-Baums über $n$ Schlüsseln mit Verzweigungsgrad $f$ ist $\log_f n$.

$$h = \log_{400} 10^7 = \frac{\log 10^7}{\log 400} = \frac{7}{2{,}602} = 2{,}69 \quad \Rightarrow \quad h = 3.$$

| Zugriffe | Zahl | Bemerkung |
|---|---|---|
| Wurzel | 1 | fast immer im Puffer |
| Zwischenebene | 1 | meist im Puffer |
| Blatt | 1 | |
| Datensatz | 1 | |
| **Von Platte** | $\approx$ 1–2 | $\approx$ 16 ms |

**Der Vergleich.**

| Medium | ohne Index | mit Index | Faktor |
|---|---|---|---|
| Festplatte | 13,3 s | 16 ms | **830** |
| SSD | 1,0 s | 0,4 ms | 2500 |
| Hauptspeicher | 0,1 s | 0,5 µs | 200 000 |

*Bemerkenswert ist, dass der Gewinn im Hauptspeicher am größten ist, obwohl dort der Durchlauf am schnellsten geht. Der Grund: Ein Index spart Arbeit proportional zur Datenmenge, und dieser Vorteil bleibt, gleichgültig wie schnell das Medium ist.*

**Wann der Durchlauf trotzdem gewinnt.**
Liefert die Bedingung viele Treffer, so kostet der Umweg über den Index mehr als er spart.

| Trefferanteil | über Index | Durchlauf | besser |
|---|---|---|---|
| 0,001 % | 2 Seiten | 250 000 | Index |
| 1 % | ≈ 60 000 | 250 000 | Index |
| 5 % | ≈ 250 000 | 250 000 | **gleich** |
| 20 % | ≈ 900 000 | 250 000 | **Durchlauf** |

*Ab etwa fünf Prozent Trefferanteil kippt es. Der Grund ist, dass die Treffer über die ganze Tabelle verstreut liegen: Jeder einzelne kostet einen zufälligen Seitenzugriff, während der Durchlauf sequentiell liest.*

*Genau diese Abwägung trifft der Optimierer aus Kapitel 8, und dafür braucht er Statistiken über die Verteilung der Werte. Sind diese veraltet, entscheidet er falsch, und das ist eine der häufigsten Ursachen für plötzlich langsame Abfragen.*

**(c) Warum eigene Puffer.**
**Der Fall.** Ein vollständiger Durchlauf über eine Tabelle, die größer ist als der Puffer.

| | |
|---|---|
| **Betriebssystem** | Verdrängt die am längsten nicht benutzte Seite. Der Durchlauf verdrängt **alles Übrige** |
| **Datenbanksystem** | Weiß, dass es einen Durchlauf macht, und benutzt einen kleinen abgetrennten Bereich dafür |

| Nach dem Durchlauf | Trefferquote | |
|---|---|---|
| Vorher | 95 % | |
| Mit Kernverdrängung | 12 % | Puffer geleert |
| Mit eigener Strategie | 93 % | |

*Ein einziger Bericht über die ganze Tabelle würde sonst den Puffer leerräumen, und danach wäre das System für Minuten langsam, bis sich der Puffer wieder gefüllt hat. Genau das ist das Flattern aus Band 5, Kapitel 5, auf einer anderen Ebene.*

**Der zweite Grund: Reihenfolge des Zurückschreibens.**
Das Protokoll muss vor den Datenseiten auf der Platte sein. Der Kern weiß das nicht.

| | |
|---|---|
| **Regel** | Keine geänderte Datenseite darf geschrieben werden, bevor der zugehörige Protokolleintrag sicher liegt |
| **Ohne eigene Kontrolle** | Der Kern schreibt zurück, wann es ihm passt, und die Regel wird verletzt |

*Deshalb öffnen Datenbanksysteme ihre Dateien meist unter Umgehung des Seitenpuffers und verwalten alles selbst. Sie bauen ein Betriebssystem im Betriebssystem, weil die Schnittstelle des Kerns die nötigen Zusagen nicht anbietet.*

Härder und Reuter[15] prägten das Kürzel ACID; Hellerstein, Stonebraker und Hamilton[16] geben einen sehr guten Überblick über den Aufbau; Ramakrishnan und Gehrke[23] führen das Gebiet gründlich ein.

# 2 Das relationale Modell

### Eine Tabelle ist eine Menge

Codds Einfall war, Daten als mathematische Relationen aufzufassen und nicht als Sätze mit Zeigern.

**Definition 2.1.** Eine **Relation** über den Wertebereichen $D_1, \ldots, D_n$ ist eine Teilmenge des kartesischen Produkts $D_1 \times \cdots \times D_n$. Die Elemente heißen **Tupel**.

| nr | name | ort | seit |
|---|---|---|---|
| 4711 | Meier | Köln | 2019 |
| 4712 | Schulz | Bonn | 2021 |
| 4713 | Weber | Köln | 2023 |

**Was daraus folgt.**

| | |
|---|---|
| **Keine Reihenfolge** | Eine Menge hat keine; die Anzeige in einer bestimmten Folge ist Sache der Abfrage |
| **Keine Duplikate** | Eine Menge enthält jedes Element einmal |
| **Keine Zeiger** | Beziehungen entstehen über **Werte**, nicht über Adressen |
| **Spalten sind atomar** | In einer Zelle steht ein Wert, keine Liste |

*Die dritte Zeile ist der eigentliche Bruch mit den Vorgängersystemen. Dort war eine Beziehung ein physischer Verweis; hier ist sie die Übereinstimmung zweier Werte. Deshalb darf sich die Ablage ändern, ohne dass Programme brechen.*

### SQL hält sich an keine dieser Regeln

| | Modell | SQL |
|---|---|---|
| Duplikate | keine | erlaubt |
| Reihenfolge | keine | Spalten sind geordnet |
| Fehlende Werte | nicht vorgesehen | `NULL` |

*Codd hat das zeitlebens kritisiert. Die Zulassung von Duplikaten geschah aus Gründen der Umsetzbarkeit: Sie zu entfernen kostet eine Sortierung, und die will man nicht bei jeder Zwischenstufe bezahlen.*

Die dritte Zeile ist die folgenreichste und wird in Kapitel 3 eigens behandelt.

### Schlüssel

| | |
|---|---|
| **Superschlüssel** | Eine Attributmenge, die ein Tupel eindeutig bestimmt |
| **Schlüsselkandidat** | Ein Superschlüssel, aus dem sich nichts weglassen lässt |
| **Primärschlüssel** | Ein ausgewählter Kandidat |
| **Fremdschlüssel** | Verweist auf einen Primärschlüssel einer anderen Relation |

*Die Unterscheidung zwischen Kandidat und Primärschlüssel ist keine mathematische, sondern eine Entwurfsentscheidung. Es kann mehrere Kandidaten geben, und welcher zum Primärschlüssel wird, hat praktische Gründe.*

**Natürlich oder künstlich.**

| | natürlich | künstlich |
|---|---|---|
| Beispiel | ISBN, Steuernummer | fortlaufende Nummer |
| Aussagekraft | vorhanden | keine |
| Stabilität | kann sich ändern | nie |
| Länge | oft lang | kurz |
| Bei Fusionen | Kollisionen möglich | Kollisionen sicher |

*Die vorletzte Zeile entscheidet meist. Eine Steuernummer sieht eindeutig und unveränderlich aus, und dann heiratet jemand, zieht um, oder eine Behörde ändert das Format. Ein Primärschlüssel, der sich ändert, zieht Änderungen in allen verweisenden Tabellen nach sich.*

**Die referentielle Zusage.**

| | |
|---|---|
| **Die Regel** | Zu jedem Fremdschlüsselwert muss es das zugehörige Tupel geben |
| **Beim Löschen** | Entweder verbieten, oder mitlöschen, oder auf `NULL` setzen |

*Das Mitlöschen ist bequem und gefährlich. Wer einen Kunden löscht und damit unbemerkt seine gesamte Bestellhistorie mitlöscht, hat mehr getan als beabsichtigt. In den meisten Fällen ist das Verbieten die richtige Wahl.*

### Die relationale Algebra

Sie ist die formale Grundlage jeder Abfragesprache und besteht aus wenigen Operatoren, die Relationen auf Relationen abbilden.

| Zeichen | Name | Wirkung |
|---|---|---|
| $\sigma$ | Selektion | wählt Zeilen aus |
| $\pi$ | Projektion | wählt Spalten aus |
| $\times$ | Produkt | alle Paare |
| $\bowtie$ | Verbund | Paare mit gleichem Wert |
| $\cup, \cap, -$ | Mengen-operationen | wie üblich |
| $\rho$ | Umbenennung | |

*Alle Operatoren nehmen Relationen und liefern eine Relation. Daraus folgt, dass sie sich beliebig verschachteln lassen, und genau diese Abgeschlossenheit macht die Optimierung aus Kapitel 8 überhaupt erst möglich.*

**Der Verbund als abgeleiteter Operator.**

$$R \bowtie_\theta S = \sigma_\theta(R \times S).$$

*Formal ist der Verbund also nur ein gefiltertes Produkt. Praktisch wäre diese Umsetzung katastrophal: Bei zwei Tabellen mit je einer Million Zeilen entstünden $10^{12}$ Paare. Kapitel 7 zeigt, wie man ihn stattdessen ausführt.*

**Die Umformungsregeln.**

| | |
|---|---|
| Selektion nach unten | $\sigma_\theta(R \bowtie S) = \sigma_\theta(R) \bowtie S$, wenn $\theta$ nur $R$ betrifft |
| Verbund ist vertauschbar | $R \bowtie S = S \bowtie R$ |
| Verbund ist assoziativ | $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$ |
| Projektion zusammenfassen | $\pi_A(\pi_B(R)) = \pi_A(R)$ bei $A \subseteq B$ |

*Die erste Regel ist die wirkungsvollste Einzeloptimierung überhaupt. Sie verkleinert die Eingabe des Verbunds, bevor er gerechnet wird, und spart damit oft Größenordnungen.*

### Aufgabe 2.1.

(a) Werten Sie einen Ausdruck der relationalen Algebra vollständig aus.
(b) Zeigen Sie, wie viel das Herunterziehen der Selektion spart.
(c) Untersuchen Sie, was der Verbund über die Ergebnisgröße verrät.

### Lösung Schritt für Schritt.

**(a) Ein Ausdruck, vollständig ausgewertet.**
**Die Relationen.**

**kunde**

| nr | name | ort |
|---|---|---|
| 1 | Meier | Köln |
| 2 | Schulz | Bonn |
| 3 | Weber | Köln |

**posten**

| id | kunde | betrag |
|---|---|---|
| 10 | 1 | 500 |
| 11 | 1 | 700 |
| 12 | 2 | 300 |
| 13 | 3 | 900 |

**Der Ausdruck.**

$$\pi_{\text{name, betrag}}\left(\sigma_{\text{ort}='\text{Köln}' \wedge \text{betrag}>400}(\text{kunde} \bowtie_{\text{nr}=\text{kunde}} \text{posten})\right).$$

**Schritt 1 – Der Verbund.**

| nr | name | ort | id | betrag |
|---|---|---|---|---|
| 1 | Meier | Köln | 10 | 500 |
| 1 | Meier | Köln | 11 | 700 |
| 2 | Schulz | Bonn | 12 | 300 |
| 3 | Weber | Köln | 13 | 900 |

**Schritt 2 – Die Selektion.**

| | | | | |
|---|---|---|---|---|
| 1 | Meier | Köln | 10 | 500 |
| 1 | Meier | Köln | 11 | 700 |
| 3 | Weber | Köln | 13 | 900 |

*Schulz fällt wegen des Ortes weg, Posten 12 zusätzlich wegen des Betrags.*
**Schritt 3 – Die Projektion.**

| name | betrag |
|---|---|
| Meier | 500 |
| Meier | 700 |
| Weber | 900 |

*Streng nach dem Modell müsste die Projektion Duplikate entfernen. Hier entstehen keine, weil die Beträge verschieden sind. In SQL bliebe ein Duplikat stehen, wenn es eines gäbe, sofern man nicht* `DISTINCT` *schreibt.*

**(b) Die Selektion herunterziehen.**
**Die Größen.**

| Relation | Zeilen | |
|---|---|---|
| kunde | $10^5$ | |
| davon in Köln | $10^3$ | 1 % |
| posten | $10^7$ | |
| je Kunde | 100 | im Mittel |

**Fassung 1 – Erst verbinden, dann filtern.**

$$|\text{kunde} \bowtie \text{posten}| = 10^7.$$

$$\text{danach filtern} \;\to\; 10^5 \text{ Zeilen.}$$

**Fassung 2 – Erst filtern, dann verbinden.**

$$|\sigma_{\text{ort}}(\text{kunde})| = 10^3.$$

$$10^3 \bowtie \text{posten} \;\to\; 10^5 \text{ Zeilen.}$$

**Der Vergleich.**

| | Zwischenergebnis | Endergebnis |
|---|---|---|
| Fassung 1 | $\mathbf{10^7}$ | $10^5$ |
| Fassung 2 | $\mathbf{10^3}$ | $10^5$ |
| **Verhältnis** | **10 000** | gleich |

*Dasselbe Ergebnis, ein zehntausendfach kleineres Zwischenergebnis. Bei Fassung 1 müsste das System zehn Millionen Zeilen aufbauen und wieder wegwerfen, und bei Speicherknappheit auf die Platte auslagern.*

*Der Optimierer nimmt diese Umformung immer vor, wenn sie zulässig ist. Sie ist zulässig, sobald die Bedingung nur Attribute einer Seite betrifft. Bei einer Bedingung über beide Seiten geht es nicht, und dann bleibt sie am Verbund hängen.*

**(c) Was der Verbund über die Ergebnisgröße verrät.**
**Die Abschätzung.** Bei einem Verbund über einen Fremdschlüssel gilt

$$|R \bowtie_{A=B} S| \approx \frac{|R| \cdot |S|}{\max\{|A|_{\text{versch}}, |B|_{\text{versch}}\}}.$$

**Der Sonderfall Fremdschlüssel.**
Verweist $S.B$ als Fremdschlüssel auf den Primärschlüssel $R.A$, so hat jede Zeile von $S$ genau einen Partner:

$$|R \bowtie S| = |S|.$$

| **Art** | **Ergebnisgröße** | **Beispiel** |
|---|---|---|
| 1 zu 1 | $\leq \min(\|R\|, \|S\|)$ | Person, Ausweis |
| 1 zu $n$ | $= \|S\|$ | Kunde, Posten |
| $n$ zu $m$ | kann explodieren | **Vorsicht** |
| Kartesisch | $\|R\| \cdot \|S\|$ | vergessene Bedingung |

**Die letzte Zeile in Zahlen.**

| $\|R\|$ und $\|S\|$ | **Produkt** | **Zeit bei $10^7$/s** |
|---|---|---|
| $10^3$ | $10^6$ | 0,1 s |
| $10^5$ | $10^{10}$ | 17 min |
| $10^6$ | $10^{12}$ | 28 Stunden |

### Die vergessene Verbundbedingung

```
SELECT * FROM kunde, posten, artikel
WHERE posten.kunde = kunde.nr;
    – die Bedingung zu artikel fehlt
```

*Die Abfrage ist syntaktisch einwandfrei und liefert das Produkt mit der gesamten Artikeltabelle. Bei drei Tabellen braucht man zwei Bedingungen; bei $n$ Tabellen $n - 1$. Wer eine vergisst, bekommt keine Fehlermeldung, sondern ein sehr großes Ergebnis.*

*Die `JOIN`-Schreibweise mit ausdrücklichem `ON` macht diesen Fehler schwerer, weil die Bedingung neben der Tabelle steht, zu der sie gehört. Das ist der Hauptgrund, sie der Kommaschreibweise vorzuziehen.*

Codds Aufsatz[7] begründete das Modell; Date[9] ist sein gründlichster Erklärer und ebenso streitbar; Garcia-Molina, Ullman und Widom[11] entwickeln die Algebra sauber.

# 3 SQL

### Die Auswertungsreihenfolge

SQL wird nicht in der Reihenfolge ausgewertet, in der es geschrieben wird. Das erklärt die meisten Anfängerfehler.

| Geschrieben | Ausgewertet | Wirkung |
|---|---|---|
| 1. `SELECT` | 5. | Spalten wählen |
| 2. `FROM` | **1.** | Quellen bilden |
| 3. `WHERE` | 2. | Zeilen filtern |
| 4. `GROUP BY` | 3. | Gruppen bilden |
| 5. `HAVING` | 4. | Gruppen filtern |
| 6. `ORDER BY` | 6. | sortieren |
| 7. `LIMIT` | 7. | abschneiden |

*Daraus folgt unmittelbar, warum ein in `SELECT` vergebener Aliasname in `WHERE` nicht benutzt werden darf: Zum Zeitpunkt der Filterung gibt es ihn noch nicht. In `ORDER BY` darf man ihn, denn dort gibt es ihn schon.*

**Der Unterschied zwischen `WHERE` und `HAVING`.**

| | `WHERE` | `HAVING` |
|---|---|---|
| Filtert | **Zeilen** | **Gruppen** |
| Vor oder nach | vor der Gruppierung | danach |
| Aggregate erlaubt | nein | ja |
| Wirkung auf Aufwand | groß | gering |

*Die letzte Zeile ist praktisch bedeutsam. Was sich in `WHERE` unterbringen lässt, gehört dorthin: Es verkleinert die Datenmenge, bevor gruppiert wird. Eine Bedingung, die in `HAVING` steht, obwohl sie in `WHERE` stehen könnte, kostet unnötig.*

**Verbundarten**

| | |
|---|---|
| `INNER JOIN` | Nur Zeilen mit Partner auf beiden Seiten |
| `LEFT JOIN` | Alle Zeilen links; rechts `NULL`, wo Partner fehlt |
| `RIGHT JOIN` | Umgekehrt |
| `FULL JOIN` | Beide Seiten vollständig |
| `CROSS JOIN` | Alle Paare, das kartesische Produkt |

**A**

| id | wert |
|---|---|
| 1 | x |
| 2 | y |
| 3 | z |

**B**

| id | zahl |
|---|---|
| 2 | 20 |
| 3 | 30 |
| 4 | 40 |

**Ergebniszeilen**

| Art | Zeilen |
|---|---|
| `INNER` | 2 |
| `LEFT` | 3 |
| `RIGHT` | 3 |
| `FULL` | 4 |
| `CROSS` | 9 |

### Der äußere Verbund mit Bedingung

Eine Bedingung in `WHERE` hebt einen `LEFT JOIN` auf.

```
SELECT * FROM A LEFT JOIN B ON A.id =
B.id
WHERE B.zahl > 25;      – verwirft die
NULL-Zeilen
```

*Die Zeile mit `id = 1` bekommt beim äußeren Verbund `NULL` in `B.zahl`. Der Vergleich `NULL > 25` ergibt weder wahr noch falsch, sondern unbekannt, und die Zeile fällt heraus. Aus dem `LEFT JOIN` ist ein `INNER JOIN` geworden.*

```
SELECT * FROM A LEFT JOIN B
ON A.id = B.id AND B.zahl > 25;     –
richtig
```

*Die Bedingung gehört in die `ON`-Klausel, wenn sie vor dem Auffüllen wirken soll, und in `WHERE`, wenn sie danach wirken soll. Beides ist gelegentlich richtig; der Unterschied wird selten bewusst getroffen.*

### Der fehlende Wert

`NULL` bedeutet nicht null und nicht leer, sondern „unbekannt“. SQL rechnet deshalb mit drei Wahrheitswerten.

| `AND` | **wahr** | **falsch** | **unbek.** |
|---|---|---|---|
| **wahr** | wahr | falsch | unbek. |
| **falsch** | falsch | falsch | **falsch** |
| **unbek.** | unbek. | **falsch** | unbek. |

| Ausdruck | Ergebnis | |
|---|---|---|
| `NULL = NULL` | unbekannt | **nicht wahr** |
| `NULL IS NULL` | wahr | |
| `NULL <> 5` | unbekannt | |
| `5 + NULL` | `NULL` | |
| `COUNT(*)` | zählt alle | |
| `COUNT(spalte)` | ohne NULL | |
| `SUM(spalte)` | ignoriert NULL | |

*Die erste Zeile ist die häufigste Fehlerquelle. Ein Vergleich mit `NULL` ist niemals wahr, auch nicht bei Ungleichheit. Deshalb gibt es die eigenen Operatoren `IS NULL` und `IS NOT NULL`.*

**Wo es besonders unangenehm wird.**

```
SELECT * FROM kunde WHERE nr NOT IN
(SELECT kunde FROM posten);
```

*Enthält die Unterabfrage ein einziges `NULL`, so liefert diese Abfrage* keine *Zeile. Denn `nr NOT IN (1,2,NULL)` bedeutet `nr<>1 AND nr<>2 AND nr<>NULL`, und der letzte Teil ist nie wahr.*

*Mit `NOT EXISTS` tritt das Problem nicht auf. Das ist einer der wenigen Fälle, in denen zwei scheinbar gleichwertige Schreibweisen wirklich Verschiedenes bedeuten, und zwar stillschweigend.*

**Aufgabe 3.1.**

(a) Schreiben Sie eine Abfrage mit Gruppierung und rechnen Sie sie durch.
(b) Untersuchen Sie Fensterfunktionen an einem Beispiel.
(c) Zeigen Sie, wie sich Unterabfragen und Verbunde unterscheiden.

**Lösung Schritt für Schritt.**

**(a) Gruppierung.**
**Die Daten.**

| id | kunde | ort | betrag |
|---|---|---|---|
| 10 | Meier | Köln | 500 |
| 11 | Meier | Köln | 700 |
| 12 | Schulz | Bonn | 300 |
| 13 | Weber | Köln | 900 |
| 14 | Weber | Köln | 200 |
| 15 | Braun | Bonn | 1500 |

**Die Abfrage.**

```
SELECT ort, COUNT(*) AS anzahl,
SUM(betrag) AS summe,
       AVG(betrag) AS mittel
FROM posten
WHERE betrag > 250
GROUP BY ort
HAVING SUM(betrag) > 1000
ORDER BY summe DESC;
```

**Schritt 1 – `WHERE`.** Zeile 14 (Betrag 200) fällt weg. Fünf Zeilen bleiben.
**Schritt 2 – `GROUP BY`.**

| ort | Zeilen |
|---|---|
| Köln | 10, 11, 13 |
| Bonn | 12, 15 |

**Schritt 3 – Aggregate.**

| ort | anzahl | summe | mittel |
|---|---|---|---|
| Köln | 3 | 2100 | 700,00 |
| Bonn | 2 | 1800 | 900,00 |

**Schritt 4 – `HAVING`.** Beide Gruppen liegen über 1000; beide bleiben.
**Schritt 5 – `ORDER BY`.**

| | | | |
|---|---|---|---|
| Köln | 3 | 2100 | 700,00 |
| Bonn | 2 | 1800 | 900,00 |

*Beachten Sie die Wirkung von Schritt 1 auf das Mittel: Ohne die Bedingung läge Kölns Mittelwert bei* $(500+700+900+200)/4 = 575$*. Ein Filter vor der Gruppierung ändert die Aggregate, einer danach nicht.*

**(b) Fensterfunktionen.**
**Das Problem.** Man will je Zeile den Anteil am Gesamtumsatz des Ortes ausweisen. Mit `GROUP BY` geht das nicht, denn dabei verschwinden die Einzelzeilen.

```
SELECT id, kunde, ort, betrag,
  SUM(betrag) OVER (PARTITION BY ort)
AS ortsumme,
  betrag * 100.0 / SUM(betrag) OVER
(PARTITION BY ort) AS anteil,
  RANK() OVER (PARTITION BY ort ORDER
BY betrag DESC) AS rang
FROM posten;
```

**Das Ergebnis.**

| id | kunde | ort | betrag | ortsumme | anteil | rang |
|---|---|---|---|---|---|---|
| 13 | Weber | Köln | 900 | 2300 | 39,1 | 1 |
| 11 | Meier | Köln | 700 | 2300 | 30,4 | 2 |
| 10 | Meier | Köln | 500 | 2300 | 21,7 | 3 |
| 14 | Weber | Köln | 200 | 2300 | 8,7 | 4 |
| 15 | Braun | Bonn | 1500 | 1800 | 83,3 | 1 |
| 12 | Schulz | Bonn | 300 | 1800 | 16,7 | 2 |

*Die Einzelzeilen bleiben erhalten, und jede trägt zusätzlich eine über ihre Gruppe berechnete Größe. Genau das ist der Unterschied zu `GROUP BY`, das die Zeilen zusammenfasst.*

**Warum das mehr ist als Bequemlichkeit.**

| Lösung | Durchläufe | |
|---|---|---|
| Unterabfrage je Zeile | $n$ | bei $n$ Zeilen |
| Verbund mit Aggregat | 2 | |
| Fensterfunktion | 1 | mit Sortierung |

*Bei einer Million Zeilen ist der Unterschied zwischen einem und einer Million Durchläufen entscheidend. Fensterfunktionen sind seit SQL:2003 genormt und werden immer noch zu selten benutzt.*

**(c) Unterabfrage oder Verbund.**
**Drei Fassungen derselben Frage:** Welche Kunden haben mindestens einen Posten?

```
-- 1:  mit IN
SELECT * FROM kunde WHERE nr IN
(SELECT kunde FROM posten);
```

```
-- 2:  mit EXISTS
SELECT * FROM kunde k WHERE EXISTS
  (SELECT 1 FROM posten p WHERE
p.kunde = k.nr);
```

```
-- 3:  mit JOIN
SELECT DISTINCT k.* FROM kunde k JOIN
posten p ON p.kunde = k.nr;
```

| Fassung | Ergebnis | bei NULL | Plan |
|---|---|---|---|
| `IN` | richtig | unkritisch | Semi-Verbund |
| `EXISTS` | richtig | unkritisch | Semi-Verbund |
| `JOIN` | richtig | unkritisch | braucht `DISTINCT` |

*Alle drei liefern dasselbe, und moderne Optimierer erzeugen für die ersten beiden meist denselben Plan. Die dritte Fassung braucht `DISTINCT`, weil ein Kunde mit fünf Posten sonst fünfmal erschiene, und diese Duplikatentfernung kostet eine Sortierung.*

**Der Semi-Verbund.**

| | |
|---|---|
| **Gewöhnlicher Verbund** | Liefert alle passenden Paare |
| **Semi-Verbund** | Liefert jede linke Zeile höchstens einmal und hört bei der ersten Übereinstimmung auf zu suchen |

*Das Abbrechen bei der ersten Übereinstimmung ist der Gewinn. Bei einem Kunden mit tausend Posten wird nur einer angesehen. Deshalb sind `EXISTS` und `IN` bei dieser Frage der `JOIN`-Fassung meist überlegen, obwohl sie umständlicher aussehen.*

**Die Verneinung dreht das Bild**

| | `NOT IN` | `NOT EXISTS` |
|---|---|---|
| Bei `NULL` | **leeres Ergebnis** | richtig |
| Empfehlung | meiden | benutzen |

*Bei der bejahenden Form sind beide gleichwertig, bei der verneinenden nicht. Das ist eine der unangenehmsten Fallen in SQL, weil sie nur auftritt, wenn zufällig ein `NULL` im Spiel ist, und dann stillschweigend ein leeres Ergebnis liefert.*

Chamberlin und Boyce[6] entwarfen SEQUEL; Elmasri und Navathe[10] behandeln SQL vollständig; Ramakrishnan und Gehrke[23] verbinden Sprache und Ausführung.

# 4 Entwurf und Normalformen

### Was schiefgeht ohne Entwurf

Eine einzige große Tabelle scheint bequem und erzeugt drei Arten von Ärger.

| bnr | kunde | ort | artikel | preis |
|---|---|---|---|---|
| 1 | Meier | Köln | Bohrer | 29,90 |
| 2 | Meier | Köln | Säge | 45,00 |
| 3 | Schulz | Bonn | Bohrer | 29,90 |

| | |
|---|---|
| **Beim Ändern** | Meier zieht um: Der Ort steht an vielen Stellen; wird eine vergessen, widerspricht sich die Datenbank |
| **Beim Einfügen** | Ein neuer Kunde ohne Bestellung lässt sich nicht anlegen |
| **Beim Löschen** | Wird die letzte Bestellung gelöscht, verschwindet der Kunde mit |

*Alle drei haben dieselbe Ursache: Es stehen Angaben in einer Tabelle, die gar nichts miteinander zu tun haben. Der Ort gehört zum Kunden, der Preis zum Artikel, und nur die Zuordnung gehört zur Bestellung.*

### Funktionale Abhängigkeit

**Definition 4.1.** $X \to Y$ gilt in einer Relation, wenn je zwei Tupel, die in $X$ übereinstimmen, auch in $Y$ übereinstimmen. Man sagt: $X$ bestimmt $Y$ funktional.

| | |
|---|---|
| `kunde` → `ort` | Jeder Kunde hat genau einen Ort |
| `artikel` → `preis` | Jeder Artikel hat genau einen Preis |
| `bnr` → alles | Der Schlüssel bestimmt alles |

*Eine funktionale Abhängigkeit ist eine Aussage über die* Wirklichkeit*, nicht über die vorliegenden Daten. Dass in einer Tabelle mit drei Zeilen zufällig $X \to Y$ gilt, beweist nichts. Man muss wissen, ob es immer gilt.*

**Die Regeln von Armstrong.**

| | |
|---|---|
| Reflexivität | $Y \subseteq X \Rightarrow X \to Y$ |
| Verstärkung | $X \to Y \Rightarrow XZ \to YZ$ |
| Transitivität | $X \to Y$ und $Y \to Z \Rightarrow X \to Z$ |

*Diese drei Regeln sind vollständig: Alles, was aus einer Menge von Abhängigkeiten folgt, lässt sich mit ihnen ableiten. Armstrong bewies das 1974.*

### Die Normalformen

| | |
|---|---|
| **1NF** | Alle Werte sind atomar; keine Listen in einer Zelle |
| **2NF** | 1NF, und kein Nichtschlüsselattribut hängt von einem **Teil** des Schlüssels ab |
| **3NF** | 2NF, und kein Nichtschlüsselattribut hängt von einem anderen Nichtschlüsselattribut ab |
| **BCNF** | Bei jeder Abhängigkeit $X \to Y$ ist $X$ ein Superschlüssel |

*Es gibt eine bekannte Merkformel: Jedes Attribut hängt vom Schlüssel ab, vom ganzen Schlüssel, und von nichts als dem Schlüssel. Die drei Teile entsprechen 1NF, 2NF und 3NF.*

**Die Zerlegung des Beispiels.**

**kunde**

| name | ort |
|---|---|
| Meier | Köln |
| Schulz | Bonn |

**artikel**

| name | preis |
|---|---|
| Bohrer | 29,90 |
| Säge | 45,00 |

**bestellung**

| bnr | kunde | artikel |
|---|---|---|
| 1 | Meier | Bohrer |
| 2 | Meier | Säge |
| 3 | Schulz | Bohrer |

| Problem | vorher | nachher |
|---|---|---|
| Umzug eintragen | *n* Zeilen | eine Zeile |
| Kunde ohne Bestellung | unmöglich | möglich |
| Letzte Bestellung weg | Kunde weg | Kunde bleibt |
| Abfrage über beides | eine Tabelle | **Verbund nötig** |

*Die letzte Zeile ist der Preis. Normalisierung tauscht Redundanz gegen Verbunde. Bei kleinen Datenmengen ist das kostenlos, bei großen nicht.*

**Wann man absichtlich verletzt**

| | |
|---|---|
| **Historische Werte** | Der Preis auf einer alten Rechnung muss der **damalige** sein, nicht der heutige |
| **Auswertungen** | Sternschemata legen Redundanz absichtlich an, um Verbunde zu sparen |
| **Zwischenspeicher** | Eine vorberechnete Summe ist Redundanz mit Absicht |

*Die erste Zeile ist keine Verletzung, sondern ein Missverständnis über die Abhängigkeit. Der Rechnungspreis hängt nicht vom Artikel ab, sondern von Artikel* und Zeitpunkt*. Er gehört also sehr wohl in die Rechnungsposition.*

*Das ist der häufigste Entwurfsfehler in kaufmännischen Systemen: Man verweist auf den Artikelpreis, und drei Jahre später stimmen alle alten Rechnungen nicht mehr. Wer solche Systeme kennt, hat es erlebt.*

**Aufgabe 4.1.**

(a) Bestimmen Sie Schlüssel aus gegebenen Abhängigkeiten.
(b) Führen Sie eine Zerlegung bis zur dritten Normalform durch.
(c) Rechnen Sie aus, was Normalisierung kostet und spart.

**Lösung Schritt für Schritt.**

**(a) Schlüssel bestimmen.**
**Gegeben.** $R(A, B, C, D, E)$ mit

$$A \to BC, \qquad CD \to E, \qquad B \to D, \qquad E \to A.$$

**Die Hülle von** $A$**.**

| Schritt | Regel | Hülle |
|---|---|---|
| 0 | — | $\{A\}$ |
| 1 | $A \to BC$ | $\{A, B, C\}$ |
| 2 | $B \to D$ | $\{A, B, C, D\}$ |
| 3 | $CD \to E$ | $\{A, B, C, D, E\}$ |

$A^+ = \{A, B, C, D, E\} \quad \Rightarrow \quad A$ ist ein Schlüssel.

**Die weiteren Kandidaten.**

| Menge | Hülle | Schlüssel? |
|---|---|---|
| $A$ | alles | ja |
| $B$ | $\{B, D\}$ | nein |
| $C$ | $\{C\}$ | nein |
| $E$ | alles (über $E \to A$) | ja |
| $CD$ | alles (über $E \to A$) | ja |
| $BC$ | $\{B, C, D, E, A\}$ | ja |

*Vier Schlüsselkandidaten. Das ist nicht ungewöhnlich, sobald es zyklische Abhängigkeiten gibt: Hier bestimmt $A$ über Umwege $E$, und $E$ bestimmt $A$ unmittelbar.*

**Ist $R$ in BCNF?**

| Abhängigkeit | linke Seite | Superschlüssel? |
|---|---|---|
| $A \to BC$ | $A$ | ja |
| $CD \to E$ | $CD$ | ja |
| $B \to D$ | $B$ | **nein** |
| $E \to A$ | $E$ | ja |

*$B \to D$ verletzt BCNF. Da $D$ in keinem Schlüsselkandidaten vorkommt, ist es ein Nichtschlüsselattribut, und damit ist auch die dritte Normalform verletzt.*

**(b) Eine Zerlegung durchführen.**

**Die Ausgangsrelation.** `lieferung(lnr, land, artikel, menge, waehrung)` mit

$$\text{lnr} \to \text{land}, \qquad \text{land} \to \text{waehrung}, \qquad \{\text{lnr}, \text{artikel}\} \to \text{menge}.$$

**Schritt 1 – Schlüssel bestimmen.**

$$\{\text{lnr}, \text{artikel}\}^+ = \{\text{lnr}, \text{artikel}, \text{land}, \text{waehrung}, \text{menge}\}.$$

Der Schlüssel ist $\{\text{lnr}, \text{artikel}\}$.
**Schritt 2 – 2NF prüfen.**

$\text{lnr} \to \text{land}$ : linke Seite ist ein **Teil** des Schlüssels. Verletzung.

**Schritt 3 – Auf 2NF zerlegen.**

| | |
|---|---|
| `lieferant` | `(lnr, land, waehrung)` |
| `position` | `(lnr, artikel, menge)` |

**Schritt 4 – 3NF prüfen.**

In `lieferant`: $\text{lnr} \to \text{land} \to \text{waehrung}$. Transitiv.

**Schritt 5 – Auf 3NF zerlegen.**

| | |
|---|---|
| `lieferant` | `(lnr, land)` |
| `land` | `(land, waehrung)` |
| `position` | `(lnr, artikel, menge)` |

**Die Probe: Ist die Zerlegung verlustfrei?**
Eine Zerlegung von $R$ in $R_1$ und $R_2$ ist verlustfrei, wenn

$$R_1 \cap R_2 \to R_1 \quad \text{oder} \quad R_1 \cap R_2 \to R_2.$$

| **Zerlegung** | **Schnitt** | **bestimmt** |
|---|---|---|
| lieferant / position | `lnr` | `lieferant` ✓ |
| lieferant / land | `land` | `land` ✓ |

*Beide Zerlegungen sind verlustfrei: Der Verbund stellt die ursprüngliche Relation exakt wieder her. Ohne diese Prüfung kann eine Zerlegung Tupel erzeugen, die es nie gab, und das ist schlimmer als Redundanz.*

**(c) Was Normalisierung kostet und spart.**
**Die Ausgangslage.** Zehn Millionen Bestellzeilen, hunderttausend Kunden.
**Platzbedarf.**

| | **unnormalisiert** | **normalisiert** |
|---|---|---|
| Kundenname, Ort (60 B) | $10^7 \cdot 60$ | $10^5 \cdot 60$ |
| | $= 600$ MB | $= 6$ MB |
| Kundennummer (4 B) | — | $10^7 \cdot 4 = 40$ MB |
| **Zusammen** | 600 MB | **46 MB** |

Ersparnis: Faktor 13.

**Aufwand beim Ändern.**

| **Ein Umzug** | **unnormalisiert** | **normalisiert** |
|---|---|---|
| Geänderte Zeilen | $\approx 100$ | **1** |
| Risiko der Inkonsistenz | vorhanden | null |

**Aufwand beim Lesen.**

| **Abfrage** | **unnormalisiert** | **normalisiert** |
|---|---|---|
| Zeilen mit Kundenname | ein Durchlauf | **Verbund** |
| Gelesene Daten | 600 MB | 46 MB |
| Zusatzaufwand | — | Streutabelle über $10^5$ |
| Zeit auf SSD | 0,30 s | **0,04 s** |

*Hier ist die normalisierte Fassung sogar beim Lesen schneller, weil sie erheblich weniger Daten von der Platte holt. Der Verbund gegen hunderttausend Kunden passt vollständig in den Hauptspeicher und kostet fast nichts.*

**Wo es kippt.**

| Lage | besser | Grund |
|---|---|---|
| Kleine Verbundtabelle | normalisiert | passt in den Speicher |
| Viele Verbunde in Folge | unnormalisiert | jeder kostet |
| Auswertung über alles | **Sternschema** | Redundanz mit Absicht |
| Ständige Änderungen | normalisiert | Konsistenz |

*Die Faustregel lautet: Erst normalisieren, dann messen, und nur dort gezielt zurücknehmen, wo die Messung es verlangt. Der umgekehrte Weg führt fast immer zu Datenbeständen, die sich widersprechen.*

Codds zweiter Aufsatz[8] führte die Normalformen ein; Date[9] behandelt sie mit großer Sorgfalt; Elmasri und Navathe[10] zeigen den Entwurfsweg vom Modell zur Tabelle.

# Part II

## Wie es innen arbeitet

# 5 Speicherung

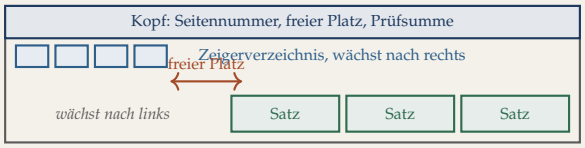
### Alles besteht aus Seiten

Ein Datenbanksystem liest und schreibt nicht Sätze, sondern Seiten fester Größe.

| System | Seitengröße | einstellbar |
|---|---|---|
| PostgreSQL | 8 KB | beim Übersetzen |
| MySQL InnoDB | 16 KB | ja |
| Oracle | 8 KB | je Bereich |
| SQL Server | 8 KB | nein |

*Der Grund ist derselbe wie beim Dateisystem in Band 5: Die Platte liefert Blöcke, und ein Zugriff auf ein Byte kostet dasselbe wie einer auf achttausend. Also nimmt man gleich alle mit.*

**Der Aufbau einer Seite.**




*Das Verzeichnis wächst von vorn, die Sätze wachsen von hinten, und dazwischen liegt der freie Platz. Wenn beide sich treffen, ist die Seite voll. Diese Anordnung ist seit Jahrzehnten praktisch unverändert.*

**Warum das Verzeichnis nötig ist.**

| | |
|---|---|
| **Ohne Verzeichnis** | Ein Satzverweis wäre eine Byteadresse in der Seite; beim Verschieben bräche jeder Index |
| **Mit Verzeichnis** | Der Verweis lautet „Seite 42, Eintrag 3“; das System darf innerhalb der Seite frei umräumen |

*Das ist dieselbe Art von Indirektion wie der Inode aus Band 5 oder die Seitentabelle aus Band 4: Eine Ebene dazwischen, damit sich das Darunterliegende ändern darf.*

### Zeilen oder Spalten

Eine Tabelle lässt sich auf zwei Arten ablegen, und der Unterschied ist folgenreich.




| | zeilenweise | spaltenweise |
|---|---|---|
| Ganzer Satz lesen | ein Zugriff | *n* Zugriffe |
| Eine Spalte über alles | alles lesen | nur diese Spalte |
| Einfügen | billig | teurer |
| Verdichtung | mäßig | **sehr gut** |
| Gut für | Tagesgeschäft | **Auswertungen** |

*Die vierte Zeile ist der unterschätzte Punkt. In einer Spalte stehen lauter gleichartige Werte nebeneinander, oft mit wenigen verschiedenen Ausprägungen. Das ist eine Quelle sehr geringer Entropie im Sinne von Band 7, und entsprechend gut lässt es sich verdichten.*

| Spalteninhalt | Verfahren | Faktor |
|---|---|---|
| Land (200 Werte) | Wörterbuch | $\approx 8$ |
| Datum, aufsteigend | Differenzen | $\approx 10$ |
| Status (5 Werte) | Lauflängen | $\approx 50$ |
| Betrag | Bitpackung | $\approx 2$ |

*Und es kommt ein zweiter Gewinn hinzu: Auf verdichteten Spalten lässt sich oft rechnen, ohne sie zu entpacken. Ein Vergleich auf Wörterbuchnummern statt auf Zeichenketten ist um Größenordnungen schneller.*

### Die Pufferverwaltung

Der Puffer hält Seiten im Hauptspeicher. Er ist der wichtigste Einzelparameter jedes Datenbanksystems.

| | |
|---|---|
| **Festhalten** | Eine benutzte Seite darf nicht verdrängt werden; ein Zähler hält sie fest |
| **Schmutzig** | Eine geänderte Seite muss vor der Verdrängung geschrieben werden |
| **Protokollregel** | Der Protokolleintrag muss vor der Datenseite auf der Platte sein |
| **Verdrängung** | Meist eine LRU-Näherung, oft mit zwei Listen |

*Die dritte Zeile ist der Grund, warum ein Datenbanksystem den Seitenpuffer des Betriebssystems umgeht. Der Kern kennt diese Reihenfolgeregel nicht und würde sie verletzen.*

**Die Trefferquote und was sie bewirkt.**

$$T_{\text{mittel}} = h \cdot T_{\text{Speicher}} + (1 - h) \cdot T_{\text{Platte}}.$$

| $h$ | **mittlere Zeit** | **Verhältnis** |
|---|---|---|
| 0,00 | 100 µs | 1000 |
| 0,90 | 10,1 µs | 101 |
| 0,99 | 1,1 µs | 11 |
| 0,999 | 0,20 µs | 2 |
| 1,00 | 0,10 µs | 1 |

*(mit* 0,1 *µs für den Speicher und* 100 *µs für die SSD)*

*Der Sprung von 99 auf 99,9 Prozent bringt Faktor fünf. Deshalb ist die Größe des Puffers so wirksam: Sie entscheidet nicht linear über die Leistung, sondern über die letzten Zehntelprozent der Trefferquote.*

### Der Puffer darf nicht zu groß sein

Man könnte meinen, mehr sei immer besser. Das gilt nur bis zu einer Grenze.

| | |
|---|---|
| **Doppelte Pufferung** | Ohne direkte Ein-/Ausgabe hält der Kern dieselben Seiten noch einmal |
| **Sortierung und Verbund** | Brauchen eigenen Arbeitsspeicher; nimmt der Puffer alles, weichen sie auf die Platte aus |
| **Wiederanlauf** | Ein großer Puffer bedeutet viele schmutzige Seiten und damit langes Wiederherstellen |

*Die verbreitete Faustregel, etwa ein Viertel des Hauptspeichers als Puffer zu nehmen, stammt aus der zweiten Zeile. Sie ist eine Faustregel und ersetzt keine Messung.*

### Aufgabe 5.1.

(a) Rechnen Sie aus, was Spaltenspeicherung bei einer Auswertung spart.
(b) Untersuchen Sie, was mit zu langen Sätzen geschieht.
(c) Bestimmen Sie die nötige Puffergröße.

### Lösung Schritt für Schritt.

**(a) Spaltenspeicherung bei einer Auswertung.**
**Die Tabelle.** Hundert Millionen Verkäufe, dreißig Spalten, im Mittel 8 Byte je Spalte.

| Größe | Wert |
|---|---|
| Satzlänge | 240 B |
| Tabellengröße | 24 GB |

**Die Abfrage.**

```
SELECT region, SUM(betrag) FROM verkauf GROUP BY region;
```

**Zeilenweise.**

$$\text{gelesen} = 24\ \text{GB}, \qquad \text{bei } 2\ \text{GB/s} \ \Rightarrow\ 12\ \text{s}.$$

**Spaltenweise, ohne Verdichtung.**

| Spalte | Größe | |
|---|---|---|
| region | 0,8 GB | $10^8 \cdot 8$ B |
| betrag | 0,8 GB | |
| **gelesen** | **1,6 GB** | 0,8 s |

Faktor 15.

**Spaltenweise, mit Verdichtung.**

| Spalte | roh | Faktor | verdichtet |
|---|---|---|---|
| region (8 Werte) | 0,8 GB | 40 | 20 MB |
| betrag | 0,8 GB | 2 | 400 MB |
| **gelesen** | | | **420 MB** |

0,21 s, Faktor **57** gegenüber der Zeilenform.

*Faktor 57 allein durch die Ablageform, ohne dass sich am Verfahren etwas ändert. Das erklärt, warum Auswertungssysteme seit etwa 2010 fast ausnahmslos spaltenweise arbeiten.*

**Die Gegenprobe.**

```
SELECT * FROM verkauf WHERE id = 4711;
```

| | zeilenweise | spaltenweise |
|---|---|---|
| Zugriffe | **1** | 30 |
| Zeit | 0,1 ms | 3 ms |

*Beim Einzelsatz ist es genau umgekehrt. Deshalb betreiben viele Häuser zwei Systeme nebeneinander: ein zeilenweises für das Tagesgeschäft und ein spaltenweises für die Auswertung, mit einem Abgleich dazwischen.*

**(b) Zu lange Sätze.**
**Das Problem.** Ein Satz mit einem Textfeld von 50 KB passt nicht in eine Seite von 8 KB.

| | |
|---|---|
| **Auslagern** | Große Werte kommen in eine Nebentabelle; im Satz steht ein Verweis |
| **Verdichten** | Vor dem Auslagern wird versucht, den Wert kleiner zu bekommen |
| **Zerteilen** | Sehr große Werte werden in Stücke zerlegt und einzeln abgelegt |

| **Wertgröße** | **Ablage** | **Zugriff** |
|---|---|---|
| $<$ 2 KB | in der Seite | sofort |
| 2–8 KB | verdichtet, meist drin | sofort |
| $>$ 8 KB | **ausgelagert** | zusätzlicher Zugriff |

*Der Gewinn ist erheblich und wird oft übersehen: Ein Durchlauf über die Tabelle liest die ausgelagerten Werte gar nicht mit, solange die Abfrage sie nicht anfordert. Die eigentliche Tabelle bleibt schlank.*

*Daraus folgt eine praktische Regel: `SELECT *` auf einer Tabelle mit großen Textfeldern kostet ein Vielfaches gegenüber der Auswahl der wirklich gebrauchten Spalten, und zwar nicht wegen der Übertragung, sondern wegen der zusätzlichen Zugriffe.*

**(c) Die nötige Puffergröße.**
**Die Lage.** Eine Datenbank von 500 GB, aber nicht alle Daten werden gleich oft gebraucht.

| **Anteil Daten** | **Anteil Zugriffe** | **Größe** | |
|---|---|---|---|
| 1 % | 50 % | 5 GB | sehr heiß |
| 10 % | 85 % | 50 GB | heiß |
| 30 % | 95 % | 150 GB | warm |
| 100 % | 100 % | 500 GB | alles |

**Die Trefferquote je Puffergröße.**

| Puffer | $h$ | mittlere Zeit | Verhältnis |
|---|---|---|---|
| 5 GB | 0,50 | 50 µs | 500 |
| 50 GB | 0,85 | 15 µs | 150 |
| 150 GB | 0,95 | 5 µs | 50 |
| 500 GB | 1,00 | 0,1 µs | 1 |

**Der Grenznutzen.**

| Schritt | Zusatz | Gewinn | je GB |
|---|---|---|---|
| 5 → 50 GB | 45 GB | 3,3-fach | 0,073 |
| 50 → 150 GB | 100 GB | 3,0-fach | 0,030 |
| 150 → 500 GB | 350 GB | 50-fach | 0,143 |

*Der Verlauf ist ungewöhnlich: Der Grenznutzen fällt zunächst und steigt am Ende sprunghaft. Der Grund ist, dass die letzten fünf Prozent der Zugriffe genau diejenigen sind, die sonst jedes Mal zur Platte gehen. Wer den gesamten Bestand in den Speicher bekommt, gewinnt überproportional.*

*Genau das ist Stonebrakers Argument von 2007: Wenn der Bestand vollständig in den Hauptspeicher passt, sind viele Annahmen der klassischen Architektur hinfällig. Kapitel 15 kommt darauf zurück.*

Abadi und Mitautoren[1] behandeln die Spaltenspeicherung umfassend; Hellerstein, Stonebraker und Hamilton[16] beschreiben die Pufferverwaltung; Garcia-Molina, Ullman und Widom[11] entwickeln die Seitenformate.

# 6 Indexstrukturen

### Der B-Baum

RUDOLF BAYER und EDWARD MCCREIGHT entwarfen 1972 eine Baumform, die auf Blockspeicher zugeschnitten ist.

| | |
|---|---|
| **Hoher Verzweigungsgrad** | Ein Knoten füllt eine Seite und enthält hunderte Schlüssel |
| **Immer ausgeglichen** | Alle Blätter liegen auf derselben Höhe |
| **Mindestens halb voll** | Jeder Knoten außer der Wurzel |
| **Wächst nach oben** | Beim Überlauf teilt sich ein Knoten; die Wurzel steigt |

*Der entscheidende Unterschied zu einem binären Suchbaum aus Band 1 ist der Verzweigungsgrad. Ein binärer Baum über einer Milliarde Schlüsseln ist dreißig Ebenen tief, ein B-Baum vier. Bei je einem Plattenzugriff je Ebene ist das der ganze Unterschied.*

| Schlüssel | binär | B-Baum | Faktor |
|---|---|---|---|
| $10^3$ | 10 | 2 | 5 |
| $10^6$ | 20 | 3 | 7 |
| $10^9$ | 30 | 4 | 8 |
| $10^{12}$ | 40 | 5 | 8 |

**Die Variante, die alle benutzen.**

| | B-Baum | $B^+$-Baum |
|---|---|---|
| Daten stehen | in allen Knoten | **nur in den Blättern** |
| Innere Knoten | Schlüssel und Daten | nur Wegweiser |
| Verzweigungsgrad | kleiner | größer |
| Blätter | einzeln | **verkettet** |
| Bereichsabfrage | umständlich | einfach |

*Die verketteten Blätter sind der praktische Hauptgrund. Eine Abfrage nach allen Werten zwischen zwei Grenzen sucht einmal den Anfang und läuft dann die Kette entlang, ohne den Baum erneut zu durchqueren.*

*Dieselbe Struktur trug in der mittleren Datentechnik den indexsequentiellen Zugriff aus Band 5, Kapitel 11: wahlfrei über den Baum, sequentiell über die Blattkette. Der Begriff ist alt, die Struktur ist es auch, und beide sind unverändert im Einsatz.*

### Was ein Index kostet

| Vorgang | ohne Index | je Index |
|---|---|---|
| `INSERT` | 1 Seite | +1–3 Seiten |
| `UPDATE` (indizierte Spalte) | 1 | +2–6 |
| `UPDATE` (andere Spalte) | 1 | +0 |
| `DELETE` | 1 | +1–3 |
| Platz | — | ≈ 15 % der Tabelle |

*Die dritte Zeile ist eine wichtige Einschränkung: Ein Index kostet nur, wenn sich die indizierte Spalte ändert. Wer eine selten geänderte Spalte indiziert, zahlt fast nichts.*

**Die Wirkung bei vielen Indizes.**

| Indizes | Einfügen | Durchsatz | |
|---|---|---|---|
| 0 | 50 µs | 20 000/s | |
| 2 | 110 µs | 9100/s | |
| 5 | 200 µs | 5000/s | |
| 12 | 410 µs | 2400/s | Faktor 8 |

*Zwölf Indizes senken den Schreibdurchsatz auf ein Achtel. Wenn davon acht nie benutzt werden, hat man den Preis bezahlt und nichts bekommen.*

### Zusammengesetzte Indizes und ihre Reihenfolge

Ein Index über $(A, B)$ ist nicht dasselbe wie einer über $(B, A)$.

| **Bedingung** | **Index** $(A, B)$ | **Index** $(B, A)$ |
|---|---|---|
| $A = 5$ | nutzbar | nein |
| $B = 7$ | nein | nutzbar |
| $A = 5$ und $B = 7$ | voll | voll |
| $A > 5$ und $B = 7$ | nur $A$ | voll |

*Der Index ist nach A sortiert, und innerhalb gleicher A-Werte nach B. Eine Bedingung auf B allein nützt deshalb nichts, so wenig wie ein Telefonbuch bei der Suche nach einem Vornamen hilft.*

**Die Regel für die Reihenfolge.**

1. Spalten mit Gleichheitsbedingung zuerst.
2. Danach die Spalte, nach der sortiert wird.
3. Zuletzt die Spalte mit Bereichsbedingung.

*Ein Bereich beendet die Nutzbarkeit des Index: Alles, was danach kommt, ist nicht mehr geordnet. Deshalb gehört die Bereichsbedingung ans Ende.*

**Der abdeckende Index.**
Enthält der Index alle Spalten, die die Abfrage braucht, so entfällt der Zugriff auf die Tabelle.

| | **Zugriffe** | **bei 1000 Treffern** |
|---|---|---|
| Index und Tabelle | $3 + 1000$ | 1003 |
| Abdeckender Index | $3 + 10$ | 13 |

*Faktor 77 dadurch, dass eine Spalte mehr im Index steht. Das ist eine der lohnendsten Maßnahmen überhaupt und der Grund, warum viele Systeme erlauben, Spalten in den Index aufzunehmen, ohne dass sie zur Sortierung beitragen.*

### Andere Indexformen

| Form | gut bei | schlecht bei |
|---|---|---|
| $B^+$-Baum | allem | — |
| Streuindex | Gleichheit | Bereiche, Sortierung |
| Bitmap | wenige Werte | häufige Änderungen |
| LSM-Baum | vielen Schreibvorgängen | Lesen, Verstärkung |
| Invertierter Index | Text | Kapitel 12 |

**Der Mischbaum.**
PATRICK O'NEIL und Mitautoren beschrieben 1996 eine Struktur für schreiblastige Lasten. Statt an Ort und Stelle zu ändern, schreibt man immer nur an und mischt später zusammen.

1. Änderungen sammeln sich in einer sortierten Tabelle im Hauptspeicher.
2. Ist sie voll, wird sie als sortierte Datei auf die Platte geschrieben.
3. Mehrere solche Dateien werden im Hintergrund zu größeren gemischt.
4. Eine Suche muss alle Ebenen befragen.

| | **$B^+$-Baum** | **LSM-Baum** |
|---|---|---|
| Schreiben | zufällig | **sequentiell** |
| Schreibverstärkung | ≳ 2 | 10–30 |
| Lesen | ≈ 3 Zugriffe | mehrere Ebenen |
| Platzverstärkung | ≈ 1,3 | ≈ 1,1 |
| Gut für | gemischt | **schreiblastig** |

*Die zweite Zeile überrascht: Der LSM-Baum schreibt insgesamt mehr, weil er ständig umschichtet. Er tut es aber sequentiell, und das war bei Festplatten hundertmal schneller als zufälliges Schreiben. Bei SSDs ist der Vorteil kleiner geworden und nicht verschwunden.*

*Es ist derselbe Gedanke wie beim protokollstrukturierten Dateisystem aus Band 7, Kapitel 12, und in beiden Fällen gilt: Der Ansatz war für Festplatten zu radikal und für Flashspeicher genau richtig.*

### Aufgabe 6.1.

(a) Fügen Sie Schlüssel in einen B$^+$-Baum ein und verfolgen Sie die Teilungen.
(b) Rechnen Sie die Höhe eines Baums über realistischen Datenmengen aus.
(c) Vergleichen Sie B$^+$-Baum und LSM-Baum quantitativ.

### Lösung Schritt für Schritt.

**(a) Einfügen mit Teilung.**
**Der Baum.** Ordnung 4: Ein Knoten fasst höchstens 3 Schlüssel.
**Ausgangslage.** Eingefügt werden 10, 20, 30.

```
[10 | 20 | 30]
```

**Schritt 1 –** 40 **einfügen.** Der Knoten läuft über und teilt sich.

| | |
|---|---|
| Wurzel | `[30]` |
| Blätter | `[10 \| 20]` → `[30 \| 40]` |

*Beim B$^+$-Baum bleibt der Trennschlüssel 30* auch *im rechten Blatt. Beim gewöhnlichen B-Baum wandert er nach oben und verschwindet unten.*

**Schritt 2 –** 50, 60 **einfügen.**

| | |
|---|---|
| Wurzel | `[30 \| 50]` |
| Blätter | `[10 \| 20]` → `[30 \| 40]` → `[50 \| 60]` |

**Schritt 3 –** 70, 80 **einfügen.**

| | |
|---|---|
| Wurzel | `[30 \| 50 \| 70]` |
| Blätter | `[10\|20]` → `[30\|40]` → `[50\|60]` → `[70\|80]` |

**Schritt 4 –** 90 **einfügen: die Wurzel teilt sich.**

| | |
|---|---|
| Wurzel | `[70]` |
| Ebene 1 | `[30 \| 50]  [90]` |
| Blätter | `[10\|20]` → `[30\|40]` → `[50\|60]` → `[70\|80]` → `[90]` |

*Der Baum ist um eine Ebene gewachsen, und alle Blätter liegen weiterhin auf derselben Höhe. Das ist die entscheidende Eigenschaft: Ein B-Baum wächst an der Wurzel, nicht an den Blättern, und bleibt deshalb ohne Zusatzaufwand ausgeglichen.*

**Das Anfügen in aufsteigender Reihenfolge.**

| **Reihenfolge** | **Füllgrad** | **Platzbedarf** |
|---|---|---|
| zufällig | ≈ 69 % | Bezugsgröße |
| aufsteigend, naiv | 50 % | +38 % |
| aufsteigend, optimiert | ≈ 95 % | −27 % |

*Bei fortlaufenden Schlüsseln teilt sich immer das rechteste Blatt, und die linke Hälfte bleibt für immer halb leer. Alle ernsthaften Systeme erkennen diesen Fall und teilen dann unsymmetrisch.*

**(b) Die Höhe über realistischen Mengen.**
**Die Größen.** Seite 8 KB, Schlüssel 8 Byte, Zeiger 6 Byte, Verwaltung 100 Byte.

$$f = \frac{8192 - 100}{8 + 6} = \frac{8092}{14} = 578.$$

**Bei realistischer Füllung von 69 Prozent:**

$$f_{\text{eff}} = 578 \cdot 0{,}69 = 399.$$

| Schlüssel | $\log_{399} n$ | Höhe | Zugriffe |
|---|---|---|---|
| $10^4$ | 1,54 | 2 | 2 |
| $10^6$ | 2,31 | 3 | 3 |
| $10^8$ | 3,08 | 4 | 4 |
| $10^{10}$ | 3,85 | 4 | 4 |
| $10^{12}$ | 4,62 | 5 | 5 |

**Was davon wirklich von der Platte kommt.**

| Ebene | Knoten | Größe | im Puffer? |
|---|---|---|---|
| Wurzel | 1 | 8 KB | immer |
| Ebene 1 | 399 | 3 MB | immer |
| Ebene 2 | $1{,}6 \cdot 10^5$ | 1,3 GB | meist |
| Blätter | $6{,}4 \cdot 10^7$ | 520 GB | selten |

*Von den vier Zugriffen kommen in der Praxis ein bis zwei von der Platte. Die oberen Ebenen sind winzig und bleiben dauerhaft im Puffer. Deshalb ist ein B-Baum über hundert Millionen Sätzen kaum langsamer als einer über zehntausend.*

**(c) B$^+$-Baum gegen LSM-Baum.**
**Die Last.** Eine Million Einfügungen je Sekunde in eine Tabelle mit einer Milliarde Sätzen, Sätze zu 100 Byte.
**Beim B$^+$-Baum.**

| Anteil | Wert | |
|---|---|---|
| Geschriebene Seite je Einfügung | 8 KB | für 100 B Nutzlast |
| Schreibverstärkung | **82** | |
| Nötige Schreibrate | 8 GB/s | unerreichbar |

*Ein B-Baum schreibt bei zufälligen Schlüsseln für jeden Satz eine ganze Seite. Sammelt der Puffer mehrere Änderungen je Seite, sinkt der Wert, aber bei einer Milliarde Sätzen und zufälligen Schlüsseln trifft es selten dieselbe Seite zweimal.*

**Beim LSM-Baum.**

| | | |
|---|---|---|
| Anfangs geschrieben | 100 B | sequentiell |
| Mischebenen | $\approx 6$ | |
| Schreibverstärkung | $\approx 12$ | jede Ebene einmal |
| Nötige Schreibrate | **1,2 GB/s** | sequentiell |

| | **B$^+$-Baum** | **LSM-Baum** |
|---|---|---|
| Schreibrate nötig | 8 GB/s zufällig | 1,2 GB/s sequentiell |
| SSD leistet | $\approx$ 0,5 GB/s zufällig | $\approx$ 3 GB/s sequentiell |
| Machbar? | nein | ja |

**Der Preis beim Lesen.**

| **Suche nach Schlüssel** | **B$^+$-Baum** | **LSM-Baum** |
|---|---|---|
| Zugriffe | 3–4 | 6–12 |
| Mit Bloom-Filtern | — | 3–5 |
| Bereichsabfrage | sehr gut | mehrere Läufe mischen |

*Die zweite Zeile rettet das Verfahren. Ein Bloom-Filter je Ebene beantwortet die Frage „ist der Schlüssel hier sicher nicht enthalten“ mit sehr wenig Speicher. Damit werden die meisten Ebenen übersprungen, ohne sie zu lesen.*

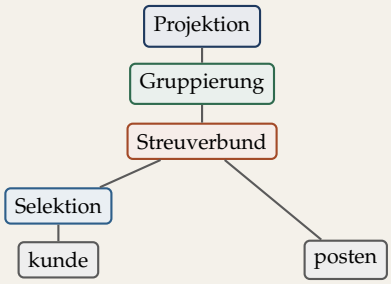
*Die Wahl ist damit klar umrissen: Bei überwiegendem Lesen und Bereichsabfragen der B$^+$-Baum, bei sehr hoher Schreiblast der LSM-Baum. Die meisten heutigen Systeme bieten beides an.*

Bayer und McCreight[2] führten den B-Baum ein; Graefe[13] beschreibt fünfzig Jahre Verfeinerungen; O’Neil und Mitautoren[22] entwarfen den Mischbaum.

# 7 Anfrageausführung

### Der Plan ist ein Baum aus Operatoren

Eine Abfrage wird zu einem Baum, dessen Knoten Operatoren sind und dessen Blätter die Tabellen.




| | |
|---|---|
| **Jeder Operator** | nimmt Tupel entgegen und liefert Tupel |
| **Abgeschlossenheit** | Deshalb lassen sie sich beliebig zusammensetzen |
| **Ausführung** | Von unten nach oben, meist als Fließband |

**Das Fließbandmodell.**
Jeder Operator bietet drei Funktionen: öffnen, nächstes Tupel liefern, schließen. Der oberste ruft, und der Ruf wandert nach unten.

| | |
|---|---|
| **Vorteil** | Kein Zwischenergebnis wird vollständig aufgebaut; das erste Tupel kommt sofort |
| **Nachteil** | Ein Funktionsaufruf je Tupel und Operator; bei Millionen Tupeln erheblich |

| Modell | Aufrufe | bei $10^7$ Tupeln |
|---|---|---|
| Tupelweise | $n \cdot k$ | $5 \cdot 10^7$ bei $k = 5$ |
| Blockweise | $n \cdot k / 1000$ | $5 \cdot 10^4$ |
| Übersetzt | keine | Maschinencode |

*Die dritte Zeile ist eine Entwicklung der letzten fünfzehn Jahre: Der Plan wird nicht abgearbeitet, sondern zu Maschinencode übersetzt und ausgeführt. Es ist genau die Übersetzung zur Laufzeit aus Band 6, Kapitel 13, angewandt auf Abfragen.*

| Ausführung | Tupel/s je Kern | Verhältnis |
|---|---|---|
| Tupelweise | $\approx 5 \cdot 10^6$ | 1 |
| Blockweise | $\approx 5 \cdot 10^7$ | 10 |
| Übersetzt | $\approx 2 \cdot 10^8$ | **40** |

### Die drei Verbundverfahren

| Verfahren | Aufwand | gut wenn |
|---|---|---|
| Schleifenverbund | $\mathcal{O}(\lvert R\rvert \cdot \lvert S\rvert)$ | $R$ **sehr klein** |
| Schleife mit Index | $\mathcal{O}(\lvert R\rvert \cdot \log \lvert S\rvert)$ | Index vorhanden |
| Sortierverbund | $\mathcal{O}(n \log n)$ | **schon sortiert** |
| Streuverbund | $\mathcal{O}(\lvert R\rvert + \lvert S\rvert)$ | **meist** |

**Der Streuverbund im Einzelnen.**

1. Die kleinere Relation wird gelesen und in eine Streutabelle gestellt.
2. Die größere wird gelesen; für jedes Tupel wird nachgeschlagen.
3. Treffer werden ausgegeben.

*Beide Relationen werden genau einmal gelesen. Das ist optimal, solange die Streutabelle in den Speicher passt. Tut sie das nicht, zerlegt man beide Seiten mit derselben Streufunktion in Teile und verbindet paarweise.*

| Lage | Durchläufe | Bemerkung |
|---|---|---|
| Passt in den Speicher | 1 | ideal |
| Zerlegung in $k$ Teile | 3 | schreiben, lesen, verbinden |
| Mehrfache Zerlegung | 5 | bei sehr großen Daten |

### Wann der Streuverbund versagt

| | |
|---|---|
| **Ungleichverteilung** | Ein Wert kommt millionenfach vor; sein Streufach läuft über |
| **Ungleichung** | Bei $R.a < S.b$ hilft Streuung nicht; nur Schleife oder Sortierung |
| **Sortierte Ausgabe** | Der Streuverbund liefert unsortiert; wird sortiert gebraucht, kann der Sortierverbund gewinnen |

*Die erste Zeile ist der häufigste Praxisfall. Wenn zwanzig Prozent aller Bestellungen auf einen Großkunden entfallen, landet ein Fünftel aller Tupel in einem Fach, und der Verbund wird langsam, obwohl die Statistik gut aussieht.*

### Sortieren mit begrenztem Speicher

Wenn die Daten nicht in den Speicher passen, sortiert man extern.

1. Lies so viel, wie in den Speicher passt, sortiere es, schreibe es als Lauf zurück.
2. Wiederhole, bis alles verarbeitet ist.
3. Mische die Läufe; bei $M$ Puffern lassen sich $M-1$ gleichzeitig mischen.
4. Wiederhole das Mischen, bis ein Lauf übrig ist.

$$\text{Durchläufe} = 1 + \left\lceil \log_{M-1} \frac{n}{M} \right\rceil .$$

| Daten | Speicher | Läufe | Durchläufe |
|---|---|---|---|
| 1 GB | 100 MB | 10 | 2 |
| 100 GB | 1 GB | 100 | 2 |
| 1 TB | 1 GB | 1000 | 2 |
| 100 TB | 1 GB | $10^5$ | 3 |

*Bemerkenswert ist, wie flach das wächst. Bei einem Gigabyte Speicher und Seiten von 8 KB stehen etwa 130 000 Puffer zur Verfügung, und damit lassen sich in einem Durchgang 130 000 Läufe mischen. Zwei Durchläufe genügen bis in den Terabytebereich.*

*Es ist dasselbe Verfahren, mit dem seit den 1950er Jahren Magnetbänder sortiert wurden. Der Aufbau der Speicherhierarchie hat sich geändert, das Verfahren nicht.*

**Aufgabe 7.1.**

(a) Vergleichen Sie die Verbundverfahren an konkreten Zahlen.
(b) Rechnen Sie einen externen Sortiervorgang vollständig durch.
(c) Untersuchen Sie, was Ungleichverteilung anrichtet.

**Lösung Schritt für Schritt.**

**(a) Die Verbundverfahren im Vergleich.**
**Die Lage.** `kunde` mit $10^5$ Zeilen (10 MB), `posten` mit $10^7$ Zeilen (1 GB), Speicher 500 MB.
**Verfahren 1 – Schleifenverbund ohne Index.**

$$\text{Vergleiche} = 10^5 \cdot 10^7 = 10^{12}.$$

bei $10^8$ Vergleichen/s $\Rightarrow$ $10^4$ s = **2,8 Stunden**.

**Verfahren 2 – Schleife mit Index auf `posten.kunde`.**

| | | |
|---|---|---|
| Kunden durchgehen | $10^5$ | |
| Je Kunde Indexsuche | 3 Zugriffe | |
| Je Kunde Treffer holen | $\approx 100$ | im Mittel |
| **Zugriffe** | $\approx 10^7$ | |
| **Zeit** | | **20 s** bei SSD |

**Verfahren 3 – Sortierverbund.**

| | | |
|---|---|---|
| `kunde` sortieren | 0,02 s | passt in den Speicher |
| `posten` sortieren | $2 \times 1$ GB | zwei Durchläufe |
| Mischen | 1 GB lesen | |
| **Zeit** | | $\approx$ **2,5 s** |

**Verfahren 4 – Streuverbund.**

| | | |
|---|---|---|
| Streutabelle aus `kunde` | 10 MB | passt bequem |
| `posten` einmal lesen | 1 GB | 0,5 s |
| Nachschlagen | $10^7$ | 0,1 s |
| **Zeit** | | $\approx$ **0,6 s** |

**Die Bilanz.**

| Verfahren | Zeit | Verhältnis |
|---|---|---|
| Schleife ohne Index | 10 000 s | 16 700 |
| Schleife mit Index | 20 s | 33 |
| Sortierverbund | 2,5 s | 4 |
| **Streuverbund** | **0,6 s** | **1** |

*Faktor 16 700 zwischen dem schlechtesten und dem besten Verfahren, bei identischem Ergebnis. Die Wahl trifft der Optimierer, und wenn er sie falsch trifft, ist das der Unterschied zwischen einer Sekunde und einem Vormittag.*

**(b) Externes Sortieren.**
**Die Aufgabe.** 50 GB sortieren, 2 GB Arbeitsspeicher, Seitengröße 8 KB.
**Schritt 1 – Läufe erzeugen.**

$$\text{Läufe} = \frac{50\ \text{GB}}{2\ \text{GB}} = 25.$$

**Schritt 2 – Mischgrad bestimmen.**

$$M = \frac{2\ \text{GB}}{8\ \text{KB}} = 262\,144\ \text{Puffer}.$$

*Es lassen sich also bis zu 262 143 Läufe gleichzeitig mischen. Fünfundzwanzig sind kein Problem; ein einziger Mischdurchgang genügt.*

**Schritt 3 – Die Bilanz.**

| Phase | Daten | Zeit bei 2 GB/s |
|---|---|---|
| Lesen (Läufe) | 50 GB | 25 s |
| Schreiben (Läufe) | 50 GB | 25 s |
| Lesen (Mischen) | 50 GB | 25 s |
| Schreiben (Ergebnis) | 50 GB | 25 s |
| **Zusammen** | 200 GB | **100 s** |

*Man bewegt das Vierfache der Datenmenge. Das ist der unvermeidliche Preis: zweimal für die Lauferzeugung, zweimal für das Mischen. Nur wenn die Daten in den Speicher passen, entfällt er ganz.*

**Die Wirkung des Arbeitsspeichers.**

| Speicher | Läufe | Durchgänge | Zeit |
|---|---|---|---|
| 100 MB | 500 | 1 | 100 s |
| 2 GB | 25 | 1 | 100 s |
| 10 MB | 5000 | 1 | 100 s |
| 1 MB | 50 000 | 2 | 150 s |

*Über einen sehr weiten Bereich ändert mehr Speicher gar nichts, weil ein Mischdurchgang so oder so genügt. Erst wenn der Mischgrad nicht mehr reicht, springt die Zeit. Wer einer Sortierung mehr Speicher gibt und keine Verbesserung sieht, hat diesen Bereich vor sich.*

**(c) Ungleichverteilung.**
**Die Lage.** Zehn Millionen Posten, hunderttausend Kunden. Ein Großkunde hat zwei Millionen Posten.
**Die Erwartung des Optimierers.**

$$\text{Posten je Kunde} = \frac{10^7}{10^5} = 100.$$

**Die Wirklichkeit.**

| Gruppe | Kunden | Posten je Kunde | Summe |
|---|---|---|---|
| Großkunde | 1 | $2 \cdot 10^6$ | $2 \cdot 10^6$ |
| Große | 100 | 10 000 | $10^6$ |
| Übrige | 99 899 | 70 | $7 \cdot 10^6$ |

**Die Wirkung auf den Streuverbund.**

| | erwartet | tatsächlich |
|---|---|---|
| Größtes Streufach | $\approx 1000$ | $2 \cdot 10^6$ |
| Passt in den Speicher | ja | nein |
| Durchläufe | 1 | 3 oder mehr |
| Zeit | 0,6 s | $\approx 4$ s |

**Die Wirkung bei Verteilung auf mehrere Rechner.**

| Knoten | Anteil | Zeit |
|---|---|---|
| Knoten mit Großkunde | 20 % | 4 s |
| Übrige neun | je ≈ 9 % | 1,8 s |
| **Gesamtdauer** | | **4 s** |

*Neun Knoten warten auf den zehnten. Das ist die Schwanzlatenz aus Band 9, Kapitel 14, an einer Datenbank: Der langsamste bestimmt die Gesamtdauer, und ein einziger schiefer Wert genügt.*

**Was dagegen hilft.**

| | |
|---|---|
| **Häufige Werte trennen** | Die Statistik führt eine Liste der häufigsten Werte; für sie wird ein eigener Plan gewählt |
| **Salzen** | Dem Schlüssel wird eine Zufallsziffer angehängt, um die Verteilung zu glätten |
| **Rundsenden** | Die kleine Seite wird an alle Knoten kopiert, statt beide zu verteilen |

*Die erste Zeile ist der Grund, warum die Statistiken eines Datenbanksystems nicht nur Mittelwerte enthalten, sondern ein Histogramm und eine Liste der häufigsten Werte. Kapitel 8 zeigt, was der Optimierer daraus macht.*

Graefes Übersicht[12] ist die maßgebliche Darstellung der Ausführungsverfahren; Garcia-Molina, Ullman und Widom[11] rechnen die Kosten vor; Ramakrishnan und Gehrke[23] führen die Operatoren ein.

# 8 Anfrageoptimierung

**Die Aufgabe**

Zu einer Abfrage gibt es sehr viele gleichwertige Pläne. Der Optimierer soll den billigsten finden, ohne alle zu prüfen.

| Tabellen | Reihenfolgen | mit 3 Verbundarten |
|---|---|---|
| 2 | 2 | 6 |
| 4 | 24 | $\approx 10^3$ |
| 6 | 720 | $\approx 10^5$ |
| 10 | $3{,}6 \cdot 10^6$ | $\approx 10^{10}$ |
| 15 | $1{,}3 \cdot 10^{12}$ | aussichtslos |

*Bei fünfzehn Tabellen kann man nicht mehr aufzählen. Selingers Arbeit von 1979 löste das mit dynamischer Programmierung: Man berechnet den besten Plan für jede Teilmenge und setzt daraus die größeren zusammen.*

| Verfahren | Aufwand | bei $n = 12$ |
|---|---|---|
| Alle aufzählen | $\mathcal{O}(n!)$ | $4{,}8 \cdot 10^8$ |
| Dynamisch | $\mathcal{O}(3^n)$ | $5{,}3 \cdot 10^5$ |
| Nur links tief | $\mathcal{O}(n \cdot 2^n)$ | $4{,}9 \cdot 10^4$ |
| Gierig | $\mathcal{O}(n^2)$ | 144 |

*Ab etwa zwölf Tabellen wechseln die meisten Systeme auf ein heuristisches Verfahren. Der beste Plan wird dann nicht mehr garantiert gefunden, und das ist eine bewusste Entscheidung: Ein guter Plan in zehn Millisekunden ist besser als der beste in zehn Sekunden.*

### Das Kostenmodell

Der Optimierer schätzt für jeden Plan die Kosten und wählt den kleinsten Wert.

| | |
|---|---|
| **Kosten** | Gewichtete Summe aus Seitenzugriffen, Prozessorzeit und gegebenenfalls Netzverkehr |
| **Grundlage** | Statistiken über die Daten |
| **Ergebnisgröße** | Die wichtigste Schätzung; Fehler pflanzen sich fort |

**Was in den Statistiken steht.**

| | |
|---|---|
| **Zeilenzahl** | Wie groß ist die Tabelle |
| **Verschiedene Werte** | Je Spalte; bestimmt die Selektivität |
| **Histogramm** | Wie sind die Werte verteilt |
| **Häufigste Werte** | Gegen Ungleichverteilung |
| **Ordnungsgrad** | Wie gut passt die physische Reihenfolge zum Index |

**Die Schätzung der Selektivität.**

| Bedingung | Schätzung | Annahme |
|---|---|---|
| $A = c$ | $1/\lvert A\rvert_{\text{versch}}$ | Gleichverteilung |
| $A > c$ | aus dem Histogramm | |
| $A$ zwischen | aus dem Histogramm | |
| $P_1$ und $P_2$ | $s_1 \cdot s_2$ | **Unabhängigkeit** |
| $P_1$ oder $P_2$ | $s_1 + s_2 - s_1 s_2$ | Unabhängigkeit |

*Die vierte Zeile ist die gefährlichste Annahme des ganzen Kapitels. Sie unterstellt, dass zwei Bedingungen nichts miteinander zu tun haben, und das ist bei realen Daten fast nie der Fall.*

### Der Fehler bei abhängigen Bedingungen

```
WHERE stadt = 'München' AND vorwahl = '089'
```

| Größe | Wert | |
|---|---|---|
| Städte | 5000 | $s_1 = 0{,}0002$ |
| Vorwahlen | 5000 | $s_2 = 0{,}0002$ |
| Geschätzt | $s_1 s_2 = 4 \cdot 10^{-8}$ | |
| Tatsächlich | **0,0002** | die Spalten sind gekoppelt |
| **Fehlfaktor** | **5000** | |

*Der Optimierer erwartet ein winziges Ergebnis und wählt einen Schleifenverbund. Tatsächlich kommen fünftausendmal mehr Zeilen, und der Plan wird um Größenordnungen zu langsam.*

| | |
|---|---|
| **Erweiterte Statistiken** | Manche Systeme erlauben, Abhängigkeiten zwischen Spalten ausdrücklich zu erfassen |
| **Rückkopplung** | Der tatsächliche Wert wird gemessen und beim nächsten Mal berücksichtigt |
| **Anpassung zur Laufzeit** | Der Plan wird gewechselt, wenn die Schätzung sich als falsch erweist |

*Die dritte Zeile ist der ehrlichste Ansatz und der aufwendigste. Ein Plan, der sich mitten in der Ausführung ändert, ist schwer zu bauen und schwer nachzuvollziehen.*

### Den Plan lesen

Jedes System zeigt seinen Plan an. Ihn zu lesen ist die wichtigste Einzelfähigkeit im Umgang mit Datenbanken.

```
EXPLAIN ANALYZE SELECT ...
```

```
HashAggregate (cost=4521..4530 rows=12
width=40)
  (actual time=88.2..88.3 rows=11
loops=1)
  -> Hash Join (cost=310..4200
rows=42000 width=36)
     (actual time=4.1..71.5 rows=41250
loops=1)
     -> Seq Scan on posten
(cost=0..3100 rows=100000 ...)
     -> Hash (cost=290..290 rows=1000
width=20)
        -> Index Scan on kunde
(cost=0..290 rows=1000 ...)
```

| | |
|---|---|
| `cost` | Geschätzt, in willkürlichen Einheiten |
| `rows` | **Geschätzte** Zeilenzahl |
| `actual` | Tatsächlich gemessen |
| `loops` | Wie oft der Knoten ausgeführt wurde |

*Worauf man zuerst sieht: die Abweichung zwischen geschätzten und tatsächlichen Zeilen. Weicht sie um mehr als etwa Faktor zehn ab, ist die Statistik das Problem und nicht der Plan.*

**Die häufigsten Befunde.**

| | |
|---|---|
| **Vollständiger Durchlauf, wo ein Index besser wäre** | Statistik veraltet, oder der Index ist wegen einer Funktion um die Spalte unbrauchbar |
| **Schleifenverbund mit vielen Durchläufen** | Die Schätzung war zu klein |
| **Sortierung auf der Platte** | Der Arbeitsspeicher reicht nicht |
| **`loops` sehr groß** | Ein Knoten wird je Zeile der äußeren Seite erneut ausgeführt |

*Der erste Fall in der zweiten Spalte ist ein Klassiker: `WHERE UPPER(name) = 'MEIER'` kann keinen Index auf `name` benutzen, weil dort die unveränderten Werte stehen. Abhilfe schafft ein Index über den Ausdruck.*

**Aufgabe 8.1.**

(a) Führen Sie eine Kostenschätzung für zwei Pläne durch.
(b) Bestimmen Sie mit dynamischer Programmierung die beste Verbundreihenfolge.
(c) Untersuchen Sie, was eine veraltete Statistik anrichtet.

**Lösung Schritt für Schritt.**

**(a) Zwei Pläne schätzen.**
**Die Abfrage.**

```
SELECT * FROM posten p JOIN kunde k ON p.kunde=k.nr WHERE
                    k.ort='Köln';
```

**Die Größen.**

| | | |
|---|---|---|
| `kunde` | $10^5$ Zeilen | 1250 Seiten |
| `posten` | $10^7$ Zeilen | 125 000 Seiten |
| Orte | 5000 | $s = 0{,}0002$ |
| Kunden in Köln | $\approx 1000$ | |
| Posten je Kunde | 100 | |

**Plan A – Streuverbund nach Selektion.**

| Schritt | Seiten | |
|---|---|---|
| Indexsuche auf `ort` | $3 + 25$ | 1000 Treffer |
| Streutabelle bauen | — | 1000 Einträge |
| `posten` durchlaufen | 125 000 | |
| **Summe** | $\approx 125\,030$ | |

**Plan B – Schleife mit Index auf `posten.kunde`.**

| | | |
|---|---|---|
| Indexsuche auf `ort` | 28 | 1000 Kunden |
| Je Kunde Indexsuche | $1000 \cdot 3$ | 3000 |
| Je Kunde Treffer holen | $1000 \cdot 100$ | 100 000 |
| **Summe** | $\approx 103\,030$ | |

**Der Vergleich mit Gewichtung.**
Zufällige Zugriffe sind teurer als sequentielle. Mit einem Gewicht von 4:

| Plan | sequentiell | zufällig | Kosten |
|---|---|---|---|
| A | 125 000 | 28 | **125 112** |
| B | 0 | 103 030 | 412 120 |

*Plan A gewinnt, obwohl er mehr Seiten anfasst. Der Grund ist die Zugriffsart: Ein sequentieller Durchlauf über 125 000 Seiten ist billiger als 103 000 zufällige Zugriffe. Genau diese Gewichtung macht das Kostenmodell aus.*

**Wann Plan B gewinnt.**

| Kunden in Köln | Plan A | Plan B | besser |
|---|---|---|---|
| 10 | 125 112 | 4120 | **B** |
| 100 | 125 112 | 41 200 | **B** |
| 300 | 125 112 | 123 600 | etwa gleich |
| 1000 | 125 112 | 412 120 | **A** |

*Der Umschlagpunkt liegt bei etwa dreihundert Kunden. Deshalb hängt die richtige Wahl an der Schätzung, und deshalb ist eine falsche Schätzung so folgenreich.*

**(b) Dynamische Programmierung.**
**Die Aufgabe.** Vier Tabellen $A, B, C, D$ verbinden.
**Die Größen.**

| Tabelle | Zeilen | Verbund | Ergebnis |
|---|---|---|---|
| $A$ | 1000 | $A \bowtie B$ | 5000 |
| $B$ | 50 000 | $B \bowtie C$ | 20 000 |
| $C$ | 2000 | $C \bowtie D$ | 8000 |
| $D$ | 500 | $A \bowtie C$ | kein Bezug |

**Schritt 1 – Paare.**

| Menge | Kosten | Größe | |
|---|---|---|---|
| $\{A, B\}$ | 51 000 | 5000 | |
| $\{B, C\}$ | 52 000 | 20 000 | |
| $\{C, D\}$ | 2500 | 8000 | billigstes Paar |

**Schritt 2 – Dreiermengen.**

| Menge | Zusammensetzung | Kosten | Größe |
|---|---|---|---|
| $\{A, B, C\}$ | $\{A, B\} \bowtie C$ | 58 000 | 2000 |
| | $\{B, C\} \bowtie A$ | 74 000 | 2000 |
| | **bestes** | **58 000** | |
| $\{B, C, D\}$ | $\{B, C\} \bowtie D$ | 72 500 | 32 000 |
| | $\{C, D\} \bowtie B$ | **60 500** | 32 000 |

**Schritt 3 – Alle vier.**

| Zusammensetzung | Kosten | |
|---|---|---|
| $\{A, B, C\} \bowtie D$ | 58 500 | **bester Plan** |
| $\{B, C, D\} \bowtie A$ | 61 500 | |
| $\{A, B\} \bowtie \{C, D\}$ | 61 500 | |

*Der beste Plan ist $((A \bowtie B) \bowtie C) \bowtie D$. Das Verfahren hat dafür sieben Teilmengen betrachtet statt vierundzwanzig Reihenfolgen, und es garantiert das Optimum unter der Voraussetzung, dass die Kosten sich zusammensetzen lassen.*

*Beachten Sie, dass der Plan mit dem billigsten Paar, $\{C, D\}$, am Ende nicht gewinnt. Ein gieriges Verfahren, das immer das billigste Paar zuerst nimmt, hätte hier den zweitbesten Plan gewählt.*

**(c) Eine veraltete Statistik.**
**Die Lage.** Eine Tabelle wächst binnen eines Monats von hunderttausend auf zehn Millionen Zeilen. Die Statistik stammt vom Anfang.

| | Statistik sagt | tatsächlich |
|---|---|---|
| Zeilen | $10^5$ | $10^7$ |
| Treffer bei $s = 0{,}01$ | 1000 | **100 000** |
| Gewählter Plan | Schleife mit Index | |
| Tatsächliche Kosten | erwartet 4000 | 400 000 |

**Die Wirkung über die Zeit.**

| Woche | Zeilen | Laufzeit | Plan |
|---|---|---|---|
| 1 | $10^5$ | 0,04 s | Index |
| 2 | $10^6$ | 0,4 s | Index |
| 3 | $5 \cdot 10^6$ | 2 s | Index |
| 4 | $10^7$ | 4 s | Index |
| nach Neuberechnung | $10^7$ | 0,6 s | **Streuverbund** |

*Die Laufzeit steigt allmählich, und niemand wird stutzig, weil die Datenmenge ja auch steigt. Erst die Neuberechnung der Statistik zeigt, dass ein anderer Plan siebenmal schneller wäre.*

**Die umgekehrte Falle.**

| | |
|---|---|
| **Plötzlicher Umschlag** | Nach einer automatischen Neuberechnung wechselt der Optimierer den Plan, und eine Abfrage wird über Nacht hundertmal langsamer |
| **Die Ursache** | Nicht die neue Statistik ist falsch, sondern die Kostenschätzung führt bei den neuen Zahlen zu einer anderen Wahl |

*Das ist der in der Praxis gefürchtetste Fall, und es ist genau der Preis der beschreibenden Sprache aus dem Vorwort. Wer sagt,* was *er will, überlässt das* wie *einem Verfahren, das seine Meinung ändern darf.*

| | |
|---|---|
| **Pläne festhalten** | Ein einmal guter Plan wird gespeichert und weiterverwendet |
| **Hinweise** | Die Abfrage schreibt dem Optimierer eine Wahl vor |
| **Regressionsprüfung** | Nach jeder Neuberechnung werden die wichtigen Abfragen gemessen |

*Die zweite Zeile ist der Rückfall in die satzorientierte Denkweise: Man nimmt dem System die Entscheidung wieder ab. Das hilft sofort und schadet auf Dauer, weil der festgeschriebene Plan bei geänderten Daten falsch wird und niemand mehr hinsieht.*

Selinger und Mitautoren[26] begründeten die kostenbasierte Optimierung; Graefe[12] behandelt die Ausführungsseite; Garcia-Molina, Ullman und Widom[11] entwickeln die Schätzverfahren.

# Part III

# Gleichzeitigkeit und Dauerhaftigkeit

# 9 Transaktionen und Isolation

### Was Serialisierbarkeit bedeutet

**Definition 9.1.** Ein Ablauf mehrerer Transaktionen heißt **serialisierbar**, wenn er dasselbe Ergebnis liefert wie irgendeine Reihenfolge, in der die Transaktionen nacheinander gelaufen wären.

> *Bemerkenswert ist das Wort* irgendeine. *Es wird nicht verlangt, dass das Ergebnis einer bestimmten Reihenfolge entspricht, nur dass es überhaupt eine gibt, die dasselbe geliefert hätte.*

**Warum das die richtige Forderung ist.**

| | |
|---|---|
| **Für den Anwender** | Er darf so tun, als sei er allein; alles Weitere kann er ignorieren |
| **Für das System** | Es darf verschränken, solange das Ergebnis stimmt; das bringt die Leistung |

**Der Abhängigkeitsgraph.**
Man zeichnet einen Knoten je Transaktion und eine Kante $T_i \to T_j$, wenn $T_j$ auf etwas zugreift, das $T_i$ vorher angefasst hat, und mindestens einer der beiden schreibt.

| | |
|---|---|
| **Der Satz** | Ein Ablauf ist genau dann serialisierbar, wenn dieser Graph **zyklenfrei** ist |

*Das ist dasselbe Kriterium wie bei den Verklemmungen aus Band 5, Kapitel 8, nur auf einen anderen Graphen angewandt. Ein Zyklus bedeutet dort, dass niemand vorankommt, und hier, dass sich keine Reihenfolge finden lässt.*

### Die vier Anomalien

| | |
|---|---|
| **Schmutziges Lesen** | Man liest, was eine andere Transaktion geschrieben und noch nicht festgeschrieben hat |
| **Nicht wiederholbares Lesen** | Zweimal dasselbe gelesen, zwei verschiedene Werte |
| **Phantom** | Dieselbe Bedingung liefert beim zweiten Mal zusätzliche Zeilen |
| **Schreibverlust** | Zwei Transaktionen lesen, rechnen, schreiben; eine Änderung geht unter |

| Stufe | schm. | n. wied. | Phant. | Verlust |
|---|---|---|---|---|
| `READ UNCOMMITTED` | ja | ja | ja | ja |
| `READ COMMITTED` | nein | ja | ja | ja |
| `REPEATABLE READ` | nein | nein | ja | ja |
| `SERIALIZABLE` | nein | nein | nein | nein |

*Die verbreitetste Voreinstellung ist die zweite Zeile. Auf ihr sind drei von vier Anomalien möglich, und die letzte, der Schreibverlust, ist die folgenreichste.*

### Die Norm beschreibt die Stufen falsch

BERENSON und Mitautoren zeigten 1995, dass die Definition der Norm unvollständig ist.

| | |
|---|---|
| **Die Norm** | Definiert die Stufen über eine Liste verbotener Anomalien |
| **Das Problem** | Die Liste ist nicht vollständig; es gibt Anomalien, die keine der genannten sind |
| **Die Folge** | Zwei Systeme können beide „`REPEATABLE READ`“ anbieten und sich verschieden verhalten |

**Die wichtigste ausgelassene Anomalie: die schiefe Schreibung.**

| Zeit | Transaktion 1 | Transaktion 2 |
|---|---|---|
| $t_1$ | liest: 2 Ärzte im Dienst | |
| $t_2$ | | liest: 2 Ärzte im Dienst |
| $t_3$ | Arzt A trägt sich aus | |
| $t_4$ | | Arzt B trägt sich aus |
| $t_5$ | `COMMIT` | `COMMIT` |
| | **Niemand ist im Dienst** | |

*Beide Transaktionen prüfen die Regel „mindestens ein Arzt muss im Dienst sein“ und beide sehen sie erfüllt. Sie schreiben auf* verschiedene *Zeilen, also gibt es keinen Schreibkonflikt und keinen Schreibverlust. Trotzdem ist die Regel am Ende verletzt.*

*Diese Anomalie tritt bei `REPEATABLE READ` und bei Mehrversionsverfahren auf, und die Norm kennt sie nicht. Sie ist der Hauptgrund, warum manche Anwendungen wirklich `SERIALIZABLE` brauchen.*

**Was Isolation kostet**

| Stufe | Durchsatz | Abbrüche |
|---|---|---|
| `READ COMMITTED` | 100 % | 0,1 % |
| `REPEATABLE READ` | 95 % | 1 % |
| `SERIALIZABLE` (MVCC) | 85 % | 5–15 % |
| `SERIALIZABLE` (Sperren) | 60 % | 0,5 % |

*Die dritte Zeile ist entscheidend für die Anwendung: Bei serialisierbarer Isolation über Mehrversionsverfahren muss jede Transaktion damit rechnen, abgebrochen zu werden, und die Anwendung muss sie wiederholen können. Wer das nicht vorgesehen hat, bekommt Fehlermeldungen im Betrieb.*

*Es ist dieselbe Struktur wie bei den Verklemmungen in Band 5: Der Abbruch ist kein Fehler, sondern der vorgesehene Ausgang, und die Wiederholung mit zufälligem Zurückweichen gehört zum Programm.*

**Aufgabe 9.1.**

(a) Prüfen Sie einen Ablauf auf Serialisierbarkeit.
(b) Führen Sie jede Anomalie an einem Beispiel vor.
(c) Untersuchen Sie den Schreibverlust und seine Behebung.

**Lösung Schritt für Schritt.**

**(a) Serialisierbarkeit prüfen.**
**Der Ablauf.**

| **Zeit** | $T_1$ | $T_2$ | $T_3$ |
|---|---|---|---|
| $t_1$ | lies $A$ | | |
| $t_2$ | | lies $B$ | |
| $t_3$ | schreib $A$ | | |
| $t_4$ | | | lies $A$ |
| $t_5$ | | schreib $B$ | |
| $t_6$ | | | schreib $C$ |
| $t_7$ | lies $B$ | | |

**Die Kanten.**

| **Kante** | **Grund** | **Zeitpunkte** |
|---|---|---|
| $T_1 \to T_3$ | $T_1$ schreibt $A$, $T_3$ liest | $t_3, t_4$ |
| $T_2 \to T_1$ | $T_2$ schreibt $B$, $T_1$ liest | $t_5, t_7$ |

**Der Graph.**

$$T_2 \to T_1 \to T_3.$$

*Zyklenfrei, also serialisierbar. Die äquivalente serielle Reihenfolge ist $T_2, T_1, T_3$.* ✓

**Eine Änderung, die es kippt.**
Fügen wir bei $t_8$ hinzu: $T_3$ schreibt $B$.

| | | |
|---|---|---|
| $T_1 \to T_3$ | wie oben | |
| $T_2 \to T_1$ | wie oben | |
| $T_1 \to T_3$ | $T_1$ liest $B$, $T_3$ schreibt | $t_7, t_8$ |
| $T_3 \to T_2$? | nein | $T_2$ war früher |

*Hier entsteht kein Zyklus. Ein Zyklus entstünde erst, wenn zusätzlich $T_3$ etwas schriebe, das $T_2$ vorher gelesen hat. Dann wäre keine serielle Reihenfolge mehr möglich, und das System müsste eine Transaktion abbrechen.*

**(b) Die Anomalien vorgeführt.**
**Schmutziges Lesen.**

| | | |
|---|---|---|
| $t_1$ | `UPDATE konto SET stand=0` | |
| $t_2$ | | liest Stand: 0 |
| $t_3$ | `ROLLBACK` | |

$T_2$ hat einen Wert gelesen, den es nie gab

**Nicht wiederholbares Lesen.**

| | | |
|---|---|---|
| $t_1$ | liest Stand: 1000 | |
| $t_2$ | | `UPDATE` auf 800, `COMMIT` |
| $t_3$ | liest Stand: 800 | |

**Phantom.**

| | | |
|---|---|---|
| $t_1$ | `COUNT(*) WHERE ort='Köln'`: 1000 | |
| $t_2$ | | `INSERT` eines Kölners, `COMMIT` |
| $t_3$ | dieselbe Abfrage: 1001 | |

*Der Unterschied zum vorigen Fall: Dort änderte sich ein vorhandener Satz, hier kam ein neuer hinzu. Eine Sperre auf gelesene Sätze verhindert den ersten Fall und nicht den zweiten, denn den neuen Satz gab es beim Sperren noch nicht.*

*Deshalb braucht man gegen Phantome Sperren auf* Bereiche *statt auf Sätze, etwa Lückensperren im Index. Es ist genau die Frage der Sperrgranularität aus Band 5, Kapitel 6, eine Ebene feiner.*

**(c) Der Schreibverlust.**
**Der Fall.** Zwei Kassen buchen gleichzeitig auf dasselbe Konto.

| Zeit | $T_1$ | $T_2$ | Stand |
|---|---|---|---|
| $t_1$ | liest 1000 | | 1000 |
| $t_2$ | | liest 1000 | 1000 |
| $t_3$ | rechnet 1000+100 | | 1000 |
| $t_4$ | | rechnet 1000+50 | 1000 |
| $t_5$ | schreibt 1100 | | 1100 |
| $t_6$ | | schreibt 1050 | **1050** |

*Erwartet wären* 1150. *Die Buchung über hundert Euro ist verschwunden. Es ist derselbe Fehler wie beim Zähler aus Band 5, Vorwort, und wie bei der verlorenen Aktualisierung aus Band 5, Kapitel 6.*

**Drei Behebungen.**
**Weg 1 – Sperrend lesen.**

```
SELECT stand FROM konto WHERE nr=4711
FOR UPDATE;
```

*$T_2$ wartet bei $t_2$, bis $T_1$ fertig ist. Das ist die pessimistische Lösung, die Satzsperre aus Band 5.*
**Weg 2 – Im System rechnen.**

```
UPDATE konto SET stand = stand + 100
WHERE nr=4711;
```

*Lesen und Schreiben werden zu einem Vorgang, und das System serialisiert ihn selbst. Das ist die einfachste und meist beste Lösung.*
**Weg 3 – Optimistisch mit Version.**

```
UPDATE konto SET stand=1100,
version=version+1
WHERE nr=4711 AND version=7;
- 0 betroffene Zeilen:  jemand war
schneller, wiederholen
```

| Weg | Sperrdauer | Konflikt | gut bei |
|---|---|---|---|
| `FOR UPDATE` | lang | Warten | wenigen Nutzern |
| `stand + 100` | sehr kurz | keiner | **fast immer** |
| Version prüfen | keine | Wiederholung | Weboberflächen |

*Der dritte Weg ist genau die optimistische Nebenläufigkeit aus Band 5, Kapitel 12, und er ist bei Weboberflächen alternativlos, weil dort niemand eine Sperre über eine Benutzereingabe hinweg halten kann.*

*Der zweite Weg wird am häufigsten übersehen. Wer den Wert erst holt, im Programm rechnet und zurückschreibt, hat sich das Problem selbst gebaut. Wer die Rechnung in die Anweisung legt, hat es nie.*

Berenson und Mitautoren[3] kritisierten die Norm; Gray und Reuter[14] behandeln Transaktionen umfassend; Härder und Reuter[15] prägten die Begriffe.

# 10 Nebenläufigkeitskontrolle

### Was serialisierbar heißt

Kapitel 9 hat gezeigt, welche Anomalien auftreten. Hier geht es darum, wie man sie verhindert.

**Definition 10.1.** Ein Ablauf mehrerer Transaktionen heißt **serialisierbar**, wenn er dasselbe Ergebnis liefert wie irgendeine Ausführung, bei der die Transaktionen nacheinander laufen.

> *Bemerkenswert ist das Wort „irgendeine“. Es wird nicht verlangt, dass die Reihenfolge der Ankunft eingehalten wird, sondern nur, dass es* eine *Reihenfolge gibt, die dasselbe leistet.*

**Wie man es prüft.**
Man zeichnet einen Graphen: einen Knoten je Transaktion, eine Kante von $T_i$ nach $T_j$, wenn $T_i$ auf ein Datum zugreift, bevor $T_j$ es auf unverträgliche Weise tut.

| | **dann Lesen** | **dann Schreiben** |
|---|---|---|
| **Erst Lesen** | verträglich | Kante |
| **Erst Schreiben** | Kante | Kante |



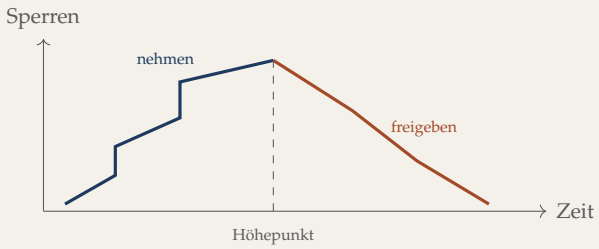

*Ein Ablauf ist genau dann serialisierbar, wenn dieser Graph zyklenfrei ist. Der Beweis ist die topologische Sortierung aus Band 1: Ein azyklischer gerichteter Graph lässt sich linear anordnen.*

> *Der Prüfweg ist theoretisch schön und praktisch unbrauchbar. Man müsste den Graphen laufend führen und bei jedem Zugriff auf Zyklen prüfen. Die Verfahren dieses Kapitels erreichen die Serialisierbarkeit deshalb auf Umwegen.*

### Zweiphasiges Sperren

Das älteste und einfachste Verfahren, das Serialisierbarkeit garantiert.

| | |
|---|---|
| **Wachstumsphase** | Es werden nur Sperren genommen, keine freigegeben |
| **Schrumpfphase** | Es werden nur Sperren freigegeben, keine genommen |




*Die Regel wirkt willkürlich und ist es nicht. Sobald eine Transaktion eine Sperre freigegeben hat, darf sie keine mehr nehmen; damit kann sie in der gedachten Reihenfolge nicht mehr nach hinten rutschen, und ein Zyklus wird unmöglich.*

**Die strenge Fassung.**
In der Praxis gibt man alle Sperren erst beim Festschreiben frei.

| | **einfaches 2PL** | **strenges 2PL** |
|---|---|---|
| Freigabe | nach dem Höhepunkt | **beim `COMMIT`** |
| Serialisierbar | ja | ja |
| Kaskadierendes Zurückrollen | möglich | ausgeschlossen |
| Parallelität | etwas höher | geringer |

*Die dritte Zeile entscheidet. Gibt eine Transaktion früh frei und rollt später zurück, so müssen alle zurückgerollt werden, die inzwischen ihre Werte gelesen haben. Diese Lawine will niemand, und deshalb benutzen alle Systeme die strenge Fassung.*

### Sperren erzeugen Verklemmungen

Zweiphasiges Sperren garantiert Serialisierbarkeit und verhindert keine Verklemmung. Das ist genau die Lage aus Band 5, Kapitel 8.

| **Zeit** | $T_1$ | $T_2$ |
|---|---|---|
| $t_1$ | sperrt $A$ | |
| $t_2$ | | sperrt $B$ |
| $t_3$ | will $B$: **wartet** | |
| $t_4$ | | will $A$: **wartet** |

| | |
|---|---|
| **Erkennen** | Wartegraph, regelmäßig auf Zyklen prüfen; ein Opfer wird zurückgerollt |
| **Verhindern** | Sperren stets in derselben Reihenfolge nehmen |
| **Zeitschranke** | Nach $n$ Sekunden abbrechen; einfach und ungenau |

*Die zweite Zeile ist in der Anwendung umsetzbar und wird zu selten getan. Wer in jeder Transaktion die Konten nach Nummer sortiert sperrt, bekommt nie eine Verklemmung zwischen Umbuchungen.*

*Und jede Anwendung, die Transaktionen benutzt, muss auf einen Verklemmungsfehler gefasst sein und die Transaktion wiederholen, mit zufällig wachsender Wartezeit. Wer das nicht tut, hat einen Fehler eingebaut, der nur unter Last auftritt.*

### Mehrversionenverfahren

Die meisten heutigen Systeme sperren beim Lesen gar nicht mehr. Stattdessen bewahren sie ältere Fassungen jedes Satzes auf.

| | |
|---|---|
| **Beim Schreiben** | Eine neue Fassung wird angelegt; die alte bleibt stehen |
| **Beim Lesen** | Es wird die Fassung gelesen, die zum Startzeitpunkt der eigenen Transaktion galt |
| **Folge** | **Leser blockieren keine Schreiber und umgekehrt** |

| **Satz** | **Wert** | **gültig ab** | **gültig bis** |
|---|---|---|---|
| 4711 | 1000 | $T_{10}$ | $T_{25}$ |
| 4711 | 900 | $T_{25}$ | $T_{31}$ |
| 4711 | 1200 | $T_{31}$ | — |

*Eine Transaktion mit der Nummer 28 sieht den Wert 900. Sie sieht ihn auch dann noch, wenn inzwischen die Transaktion 31 den Wert geändert hat. Damit ist ein wiederholtes Lesen ohne Sperre stabil.*

**Der Preis.**

| | |
|---|---|
| **Platz** | Alte Fassungen belegen Speicher, bis niemand sie mehr braucht |
| **Aufräumen** | Ein Hintergrundvorgang muss sie entfernen |
| **Lange Transaktionen** | Halten alle Fassungen seit ihrem Beginn am Leben |

*Die dritte Zeile ist eine der häufigsten Betriebsstörungen. Eine vergessene offene Transaktion verhindert das Aufräumen, die Tabellen wachsen, und die Leistung sinkt allmählich, ohne dass jemand versteht warum.*

*Es ist derselbe Mechanismus wie beim Kopieren bei Bedarf aus Band 5, Kapitel 2, und dieselbe Falle wie beim Löschen einer offenen Datei: Was noch jemand betrachtet, darf nicht verschwinden.*

### Aufgabe 10.1.

(a) Prüfen Sie einen Ablauf auf Serialisierbarkeit.
(b) Vergleichen Sie Sperren und Versionen quantitativ.
(c) Untersuchen Sie die Schreibschiefe und ihre Behebung.

### Lösung Schritt für Schritt.

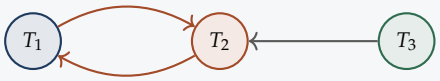
**(a) Einen Ablauf prüfen.**
**Der Ablauf.**

| **Zeit** | $T_1$ | $T_2$ | $T_3$ |
|---|---|---|---|
| $t_1$ | $R(A)$ | | |
| $t_2$ | | $R(B)$ | |
| $t_3$ | $W(B)$ | | |
| $t_4$ | | | $R(A)$ |
| $t_5$ | | $W(A)$ | |
| $t_6$ | | | $W(C)$ |

**Schritt 1 – Konflikte suchen.**

| **Datum** | **Zugriffe** | **Konflikt** | **Kante** |
|---|---|---|---|
| $A$ | $R_1, R_3, W_2$ | $R_1$ vor $W_2$ | $T_1 \to T_2$ |
| $A$ | | $R_3$ vor $W_2$ | $T_3 \to T_2$ |
| $B$ | $R_2, W_1$ | $R_2$ vor $W_1$ | $T_2 \to T_1$ |
| $C$ | $W_3$ | keiner | — |

**Schritt 2 – Den Graphen zeichnen.**




$T_1 \to T_2 \to T_1$ ist ein Zyklus.

*Der Ablauf ist **nicht** serialisierbar. Es gibt keine Reihenfolge von $T_1$ und $T_2$, die dasselbe Ergebnis liefert.*

**Schritt 3 – Was zweiphasiges Sperren getan hätte.**
$T_1$ hätte bei $t_1$ eine Lesesperre auf $A$ genommen und sie bis zum Ende gehalten. $T_2$ hätte bei $t_5$ auf sie warten müssen.

| Zeit | $T_1$ | $T_2$ | |
|---|---|---|---|
| $t_1$ | $S(A), R(A)$ | | |
| $t_2$ | | $S(B), R(B)$ | |
| $t_3$ | $X(B)$: **wartet** | | |
| $t_5$ | | $X(A)$: **wartet** | Verklemmung |

*Das System erkennt den Zyklus im Wartegraphen und rollt eine der beiden zurück. Der nicht serialisierbare Ablauf wird also nicht verhindert, indem er umgeordnet wird, sondern indem eine Transaktion scheitert und es erneut versucht.*

**(b) Sperren gegen Versionen.**
**Die Last.** Neunzig Prozent Leseanfragen, zehn Prozent Schreibvorgänge, auf denselben Sätzen.

| | **Sperren** | **Versionen** |
|---|---|---|
| Leser gegen Leser | frei | frei |
| Leser gegen Schreiber | **blockiert** | frei |
| Schreiber gegen Schreiber | blockiert | blockiert |
| Speicherbedarf | gering | höher |
| Aufräumen nötig | nein | ja |

**Die Rechnung.**
Sei $p$ die Wahrscheinlichkeit, dass zwei gleichzeitige Zugriffe denselben Satz betreffen, hier $p = 0{,}01$.

| Konfliktart | Anteil | blockiert bei Sperren |
|---|---|---|
| Lesen/Lesen | 81 % | nein |
| Lesen/Schreiben | 18 % | **ja** |
| Schreiben/Schreiben | 1 % | ja |
| **Blockierend** | 19 % | bei Versionen: 1 % |

*Von neunzehn Prozent auf ein Prozent. Bei leseintensiver Last ist das der Unterschied zwischen einem System, das unter Last einbricht, und einem, das durchhält. Deshalb hat sich das Mehrversionenverfahren durchgesetzt.*

**Die Kehrseite in Zahlen.**

| Längste offene Transaktion | Zusatzplatz | |
|---|---|---|
| 1 s | ≈ 0,1 % | unbedenklich |
| 1 min | ≈ 6 % | |
| 1 h | ≈ 100 % | Tabelle verdoppelt |
| 1 Tag | ≈ 2400 % | Betriebsstörung |

*Bei einer Änderungsrate, die der Tabellengröße je Stunde entspricht. Die Zahlen zeigen, warum das Überwachen der ältesten offenen Transaktion zu den wichtigsten Betriebskennzahlen gehört.*

**(c) Schreibschiefe.**
**Der Fall.** Zwei Ärzte haben Bereitschaft. Die Regel lautet: Mindestens einer muss eingetragen bleiben. Beide wollen sich gleichzeitig austragen.

| Zeit | Alice | Bob |
|---|---|---|
| $t_1$ | zählt: 2 im Dienst | |
| $t_2$ | | zählt: 2 im Dienst |
| $t_3$ | $2 > 1$, trägt sich aus | |
| $t_4$ | | $2 > 1$, trägt sich aus |
| $t_5$ | **niemand im Dienst** | |

*Beide haben gelesen, beide haben geprüft, beide haben geschrieben, und beide haben auf verschiedene Sätze geschrieben. Es gibt keinen Schreibkonflikt, und trotzdem ist die Regel verletzt.*

**Warum Mehrversionenverfahren das nicht abfangen.**

| | |
|---|---|
| **Sie prüfen** | Ob jemand denselben Satz geändert hat, den ich ändere |
| **Sie prüfen nicht** | Ob jemand einen Satz geändert hat, den ich *gelesen* habe |

| **Isolationsstufe** | **Schreibschiefe** | |
|---|---|---|
| `READ COMMITTED` | möglich | |
| `REPEATABLE READ` | möglich | |
| Momentaufnahme | möglich | **überrascht viele** |
| `SERIALIZABLE` | ausgeschlossen | |

**Vier Wege, es zu beheben.**

| | |
|---|---|
| **Serialisierbar** | Die höchste Stufe einstellen; das System erkennt es und rollt zurück |
| **Ausdrücklich sperren** | `SELECT ... FOR UPDATE` auf die gelesenen Sätze |
| **Bedingung in der Datenbank** | Eine Prüfregel, die den verbotenen Zustand ausschließt |
| **Ein gemeinsamer Satz** | Beide ändern denselben Zähler; dann gibt es einen echten Konflikt |

*Der zweite Weg ist der gebräuchlichste und der am leichtesten zu vergessende. Ein `SELECT`, dessen Ergebnis eine Entscheidung begründet, ist kein harmloser Lesevorgang, sondern Teil der Transaktionslogik.*

*Der vierte Weg ist der Umweg der mittleren Datentechnik aus Band 5: Man erzeugt absichtlich einen Konflikt auf einem einzigen Satz, damit die Satzsperre greift. Das ist unelegant und funktioniert überall.*

Bernstein, Hadzilacos und Goodman[4] sind die maßgebliche Darstellung der Nebenläufigkeitskontrolle; Gray und Reuter[14] behandeln die Umsetzung; Berenson und Mitautoren[3] führen die Anomalien einzeln vor.

# 11 Wiederanlauf

### Zwei Regeln, aus denen alles folgt

Ein Datenbanksystem muss nach einem Absturz einen definierten Zustand herstellen. Dafür genügen zwei Regeln über die Reihenfolge des Schreibens.

| | |
|---|---|
| **Protokoll zuerst** | Der Protokolleintrag muss auf der Platte sein, bevor die geänderte Datenseite geschrieben wird |
| **Festschreiben zwingt** | Beim `COMMIT` müssen alle Protokolleinträge der Transaktion sicher liegen |

*Die erste Regel sichert, dass man jede Änderung zurücknehmen kann. Die zweite sichert, dass man jede festgeschriebene Änderung nachholen kann. Zusammen ergeben sie Atomarität und Dauerhaftigkeit.*

**Was ein Protokolleintrag enthält.**

| Feld | Inhalt | wofür |
|---|---|---|
| LSN | laufende Nummer | Reihenfolge |
| Transaktion | Nummer | Zuordnung |
| Seite | welche | Zuordnung |
| **Vorher** | alter Wert | **Zurückrollen** |
| **Nachher** | neuer Wert | **Nachholen** |
| Vorgänger | LSN | Kette je Transaktion |

*Wer beide Werte protokolliert, kann in beide Richtungen. Es gibt Verfahren, die nur einen speichern; sie sind sparsamer und schränken ein, wann Seiten geschrieben werden dürfen.*

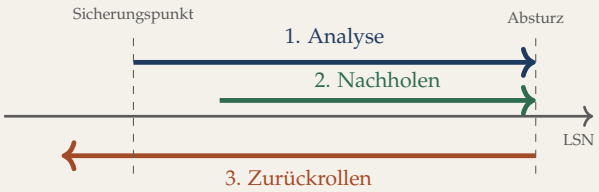
| Verfahren | Seiten schreiben | Bewertung |
|---|---|---|
| Nur Nachher | erst nach `COMMIT` | Puffer muss reichen |
| Nur Vorher | vor `COMMIT` erzwingen | teuer |
| **Beides** | **jederzeit** | flexibel, Standard |

### Der Wiederanlauf in drei Durchgängen

Das Verfahren ARIES von C. MOHAN und Mitautoren, 1992, ist der Standard geworden.

| Durchgang | Richtung | Zweck |
|---|---|---|
| Analyse | vorwärts | Wer war offen, welche Seiten sind schmutzig |
| **Nachholen** | vorwärts | **Alles wiederholen, auch von Verlierern** |
| Zurückrollen | rückwärts | Die Verlierer rückgängig machen |

*Die mittlere Zeile überrascht. Warum sollte man Änderungen von Transaktionen wiederholen, die man gleich danach zurücknimmt? Der Grund ist Einfachheit: Zuerst wird der Zustand exakt so hergestellt, wie er beim Absturz war, und danach wird sauber zurückgerollt. Man muss also nicht unterscheiden.*




**Der Sicherungspunkt.**
Ohne ihn müsste der Wiederanlauf das Protokoll von Anfang an lesen.

| | |
|---|---|
| **Was geschrieben wird** | Eine Liste der offenen Transaktionen und der schmutzigen Seiten |
| **Was nicht geschieht** | Das System hält **nicht** an; die Arbeit läuft weiter |
| **Der Gewinn** | Der Wiederanlauf beginnt beim Sicherungspunkt statt am Anfang |

| **Abstand** | **Protokoll zu lesen** | **Wiederanlaufzeit** |
|---|---|---|
| Kein Sicherungspunkt | alles | Stunden |
| 30 min | $\approx$ 2 GB | $\approx$ 60 s |
| 5 min | $\approx$ 350 MB | $\approx$ 10 s |
| 30 s | $\approx$ 35 MB | $\approx$ 1 s |

*Häufigere Sicherungspunkte verkürzen den Wiederanlauf und belasten den laufenden Betrieb, weil mehr Seiten zurückgeschrieben werden. Es ist derselbe Kompromiss wie beim Journal aus Band 5.*

### Wiederholbarkeit

Ein Wiederanlauf kann selbst abstürzen. Dann muss er von vorn beginnen, und zwar ohne Schaden anzurichten.

| | |
|---|---|
| **Seiten-LSN** | Jede Seite trägt die Nummer der letzten auf ihr ausgeführten Änderung |
| **Die Prüfung** | Ist die Seiten-LSN bereits $\geq$ der des Protokolleintrags, wurde die Änderung schon ausgeführt |
| **Folge** | Nachholen ist **idempotent** |

*Das ist genau die Idempotenz aus Band 9, hier auf Seitenebene. Ein Vorgang, der beliebig oft wiederholt werden darf, braucht keine Buchführung darüber, wie oft er schon gelaufen ist.*

**Beim Zurückrollen ist es schwerer.**
Eine Rücknahme ist kein Nachholen: Sie ändert den Zustand, und eine zweite Rücknahme wäre falsch.

| | |
|---|---|
| **Ausgleichssätze** | Jede Rücknahme wird selbst ins Protokoll geschrieben |
| **Rückwärtszeiger** | Sie verweist auf die Änderung *vor* der zurückgenommenen |
| **Folge** | Ein Absturz während des Zurückrollens setzt dort fort, wo er aufhörte |

*Ausgleichssätze werden nie zurückgenommen. Sie sind nur Nachholeinträge, und damit ist auch das Zurückrollen wiederholbar.*

### Aufgabe 11.1.

(a) Führen Sie einen Wiederanlauf vollständig durch.
(b) Rechnen Sie aus, was das Protokoll kostet.
(c) Untersuchen Sie Gruppenfestschreibung.

### Lösung Schritt für Schritt.

**(a) Ein vollständiger Wiederanlauf.**
**Das Protokoll beim Absturz.**

| LSN | TA | Seite | vorher | nachher |
|---|---|---|---|---|
| 10 | $T_1$ | $P_5$ | 100 | 150 |
| 11 | $T_2$ | $P_7$ | 200 | 250 |
| 12 | Sicherungspunkt: offen $\{T_1, T_2\}$, schmutzig $\{P_5, P_7\}$ | | | |
| 13 | $T_1$ | $P_9$ | 300 | 350 |
| 14 | $T_2$ | **COMMIT** | | |
| 15 | $T_3$ | $P_5$ | 150 | 180 |
| 16 | $T_1$ | $P_7$ | 250 | 270 |
| | Absturz | | | |

**Durchgang 1 – Analyse, ab LSN 12.**

| Transaktion | Ausgang | Ergebnis |
|---|---|---|
| $T_1$ | kein `COMMIT` | **Verlierer** |
| $T_2$ | `COMMIT` bei 14 | Gewinner |
| $T_3$ | kein `COMMIT` | **Verlierer** |

Schmutzige Seiten: $\{P_5, P_7, P_9\}$.

**Durchgang 2 – Nachholen, ab der kleinsten LSN der schmutzigen Seiten.**

| LSN | Seiten-LSN | Vergleich | Aktion |
|---|---|---|---|
| 10 | 10 | $10 \geq 10$ | schon drin |
| 11 | 9 | $9 < 11$ | **nachholen** |
| 13 | 0 | — | **nachholen** |
| 15 | 10 | $10 < 15$ | **nachholen** |
| 16 | 11 | $11 < 16$ | **nachholen** |

*Die Einträge 15 und 16 stammen von Verlierern und werden trotzdem nachgeholt. Danach ist der Zustand genau der vom Zeitpunkt des Absturzes.*

**Durchgang 3 – Zurückrollen der Verlierer, rückwärts.**

| LSN | TA | Aktion | Ausgleichssatz |
|---|---|---|---|
| 16 | $T_1$ | $P_7 \to 250$ | LSN 17, zeigt auf 13 |
| 15 | $T_3$ | $P_5 \to 150$ | LSN 18, zeigt auf — |
| 13 | $T_1$ | $P_9 \to 300$ | LSN 19, zeigt auf 10 |
| 10 | $T_1$ | $P_5 \to 100$ | LSN 20, zeigt auf — |

**Der Endzustand.**

| Seite | vor allem | nach Wiederanlauf | richtig? |
|---|---|---|---|
| $P_5$ | 100 | 100 | ✓ |
| $P_7$ | 200 | 250 | ✓ ($T_2$) |
| $P_9$ | 300 | 300 | ✓ |

*Genau die Wirkung von $T_2$ ist erhalten, alles von $T_1$ und $T_3$ ist verschwunden. Das ist Atomarität und Dauerhaftigkeit in einem Durchlauf.*

**(b) Was das Protokoll kostet.**
**Die Schreibmenge.**

| Größe | Wert | |
|---|---|---|
| Geänderter Satz | 200 B | |
| Protokolleintrag | ≈ 250 B | vorher + nachher + Kopf |
| Datenseite | 8192 B | |

| | | |
|---|---|---|
| **Ohne Protokoll** | 8192 B je Änderung | sofort schreiben |
| **Mit Protokoll** | 250 B sofort | Seite später, gebündelt |

*Das Protokoll schreibt weniger und schreibt es sequentiell. Die Datenseiten werden später geschrieben, und wenn eine Seite zehnmal geändert wurde, nur einmal. Das Protokoll ist also nicht der Preis für die Sicherheit, sondern zugleich eine Beschleunigung.*

**Die Rechnung für eine Last.**
Tausend Änderungen je Sekunde, verteilt auf hundert Seiten.

| Anteil | Menge/s | Art |
|---|---|---|
| Protokoll | 250 KB | sequentiell |
| Datenseiten (gebündelt) | 819 KB | zufällig |
| Ohne Bündelung wären es | 8 MB | zufällig |

*Faktor zehn weniger zufällige Schreibvorgänge. Auf einer Festplatte ist das der Unterschied zwischen brauchbar und unbrauchbar; auf einer SSD immer noch der zwischen langer und kurzer Lebensdauer.*

**(c) Gruppenfestschreibung.**
**Das Problem.** Jedes `COMMIT` verlangt ein erzwungenes Schreiben auf die Platte.

| Medium | Erzwungenes Schreiben | Höchstrate |
|---|---|---|
| Festplatte | 10 ms | 100/s |
| SSD | 200 µs | 5000/s |
| SSD mit Kondensator | 20 µs | 50 000/s |

*Hundert Transaktionen je Sekunde auf einer Festplatte, gleichgültig wie schnell der Prozessor ist. Diese Zahl war jahrzehntelang die harte Grenze jeder Buchungsanwendung.*

**Der Ausweg.**
Man sammelt mehrere Festschreibungen und erzwingt sie gemeinsam.

1. Eine Transaktion will festschreiben und stellt sich an.
2. Das System wartet kurz, ob weitere kommen.
3. Ein einziges erzwungenes Schreiben deckt alle ab.
4. Allen wird gemeinsam bestätigt.

| Gruppengröße | Durchsatz | Antwortzeit |
|---|---|---|
| 1 | 100/s | 10 ms |
| 10 | 1000/s | ≈ 11 ms |
| 50 | 5000/s | ≈ 15 ms |
| 200 | **20 000**/s | ≈ 30 ms |

*Der Durchsatz steigt um Faktor zweihundert, die Antwortzeit verdreifacht sich. Das ist der klassische Tausch zwischen Durchsatz und Verzögerung aus Band 9, hier in seiner reinsten Form.*

### Die verlockende Abkürzung

Viele Systeme bieten an, das erzwungene Schreiben abzuschalten oder zu verzögern.

| Einstellung | Durchsatz | Bei Stromausfall |
|---|---|---|
| Voll | 100/s | nichts verloren |
| Gruppen | 20 000/s | nichts verloren |
| Verzögert | 80 000/s | **letzte Sekunde weg** |
| Aus | 200 000/s | unbestimmt |

*Die dritte Zeile ist ein vertretbarer Kompromiss, wenn man weiß, was man tut: Bei einem Absturz gehen die letzten festgeschriebenen Transaktionen verloren, obwohl sie bestätigt wurden. Für Messwerte ist das gleichgültig, für Buchungen nicht.*

*Die vierte Zeile ist nie vertretbar. Sie gibt nicht nur die Dauerhaftigkeit auf, sondern auch die Atomarität: Nach dem Wiederanlauf kann die Datenbank in einem Zustand sein, den keine Transaktionsfolge je erzeugt hätte.*

Mohan und Mitautoren[21] beschrieben ARIES; Gray und Reuter[14] behandeln den Wiederanlauf umfassend; Bernstein, Hadzilacos und Goodman[4] entwickeln die Theorie dahinter.

# Part IV

# Finden statt Abfragen

# 12 Der invertierte Index

### Eine andere Art von Frage

Die bisherigen Kapitel beantworteten Fragen mit einer eindeutigen Antwort. Beim Suchen ist das anders.

| | Datenbank | Suche |
|---|---|---|
| Frage | genau festgelegt | **ungefähr** |
| Antwort | Menge | **geordnete Liste** |
| Richtigkeit | entscheidbar | **Urteilssache** |
| Fehlende Treffer | Fehler | normal |
| Zusätzliche Treffer | Fehler | normal |

*Die dritte Zeile ist der Kern. Ob ein Dokument zu einer Anfrage passt, lässt sich nicht ausrechnen, sondern nur beurteilen. Deshalb braucht dieses Gebiet Gütemaße, wo die Datenbank nur Richtigkeit kennt.*

**Warum ein Durchlauf nicht genügt.**

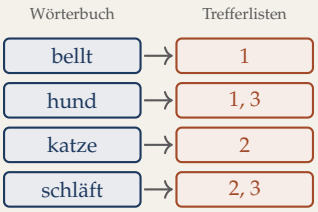
| Sammlung | Umfang | Durchlaufzeit |
|---|---|---|
| Ein Buch | 1 MB | 0,5 ms |
| Eine Bibliothek | 100 GB | 50 s |
| Ein Webindex | 100 TB | 14 Stunden |

*Bei zehntausend Anfragen je Sekunde ist selbst die zweite Zeile aussichtslos. Man braucht eine Struktur, die von der Sammlungsgröße weitgehend unabhängig ist.*

### Der invertierte Index

Die Umkehrung der natürlichen Anordnung: Statt zu jedem Dokument seine Wörter zu speichern, speichert man zu jedem Wort seine Dokumente.

| | |
|---|---|
| **Dokument 1** | *der hund bellt* |
| **Dokument 2** | *die katze schläft* |
| **Dokument 3** | *der hund schläft* |




| | |
|---|---|
| **Wörterbuch** | Alle vorkommenden Begriffe, sortiert; meist im Hauptspeicher |
| **Trefferlisten** | Je Begriff die Dokumentnummern, aufsteigend; auf der Platte |

*Die Sortierung der Trefferlisten ist wesentlich. Sie erlaubt es, zwei Listen im Gleichschritt zu durchlaufen und den Durchschnitt in linearer Zeit zu bilden, ohne eine davon vollständig zu lesen.*

**Der Weg vom Text zum Index.**

| | |
|---|---|
| **Zerlegen** | Text in Wörter trennen; scheinbar trivial |
| **Vereinheitlichen** | Groß- und Kleinschreibung, Umlaute, Akzente |
| **Stammformen** | *Hunde*, *Hunden*, *Hund* auf eine Form bringen |
| **Stoppwörter** | Sehr häufige Wörter weglassen oder gesondert behandeln |

*Die erste Zeile ist im Deutschen unangenehmer als im Englischen. Zusammensetzungen wie* Donaudampfschifffahrtsgesellschaft *müssen zerlegt werden, sonst findet man sie nur bei genauer Eingabe. Das verlangt ein Wörterbuch und liefert trotzdem Fehler.*

### Stammformen können schaden

Ein zu grober Stammformer wirft Bedeutungen zusammen.

| Wörter | gemeinsamer Stamm | Bewertung |
|---|---|---|
| Häuser, Haus | haus | richtig |
| laufen, läuft | lauf | richtig |
| Organ, Organismus | organ | falsch |
| Universität, Universum | univers | falsch |

*Ein Stammformer erhöht die Trefferzahl und senkt die Genauigkeit. Ob das ein Gewinn ist, hängt vom Anwendungsfall ab, und die Antwort lautet für eine Patentrecherche anders als für eine Websuche.*

*Die heute übliche Lösung besteht darin, beide Formen zu indexieren und die genaue Übereinstimmung höher zu bewerten. Man entscheidet nicht, sondern gewichtet.*

### Was der Index sonst noch trägt

| Zusatz | ermöglicht | Preis |
|---|---|---|
| Häufigkeit | Rangfolge (Kap. 13) | gering |
| **Positionen** | **Phrasensuche** | Index wird 2–4-mal so groß |
| Feld | Suche im Titel | gering |
| Vorkommensart | Überschrift höher werten | gering |

*Ohne Positionen kann man „roter Faden“ nur als „rot“ und „faden“ im selben Dokument suchen. Mit Positionen kann man verlangen, dass sie unmittelbar aufeinanderfolgen. Der Unterschied in der Ergebnisqualität ist erheblich, und der Preis ist es auch.*

**Warum Verdichtung hier so gut wirkt.**
Trefferlisten sind aufsteigende Zahlenfolgen. Speichert man Differenzen statt Werten, werden die Zahlen klein.

| | |
|---|---|
| **Original** | 12, 47, 48, 51, 199, 203, 204 |
| **Differenzen** | 12, 35, 1, 3, 148, 4, 1 |

| **Kodierung** | **bit je Eintrag** | |
|---|---|---|
| Feste 32 bit | 32 | |
| Differenzen, 32 bit | 32 | nichts gewonnen |
| Differenzen, variabel | $\approx 9$ | Byte-weise |
| Differenzen, Elias-Gamma | $\approx 7$ | |
| **Differenzen, PForDelta** | $\approx 6$ | schnell und klein |

*Faktor fünf, und das ist genau die Verdichtung aus Band 7: Die Differenzen sind ungleich verteilt, kleine Werte kommen viel häufiger vor, und ein Code mit variabler Länge nutzt das aus.*

*Bemerkenswert ist die letzte Zeile. Sie ist nicht die kürzeste Kodierung, sondern die, bei der das Entpacken am schnellsten geht, weil sie mit den Vektorbefehlen aus Band 4 arbeitet. Bei einem Index, der milliardenfach gelesen wird, zählt das mehr als das letzte bit.*

**Aufgabe 12.1.**

(a) Bauen Sie einen invertierten Index und beantworten Sie Anfragen damit.
(b) Rechnen Sie die Größe eines Index aus.
(c) Untersuchen Sie den Aufbau eines Index über große Sammlungen.

**Lösung Schritt für Schritt.**

**(a) Index bauen und benutzen.**
**Die Sammlung.**

| | |
|---|---|
| $D_1$ | *der schnelle braune fuchs springt* |
| $D_2$ | *der faule hund schläft* |
| $D_3$ | *der schnelle hund springt* |
| $D_4$ | *ein brauner hund* |

**Nach Vereinheitlichung und Stoppwörtern (*der*, *ein*).**

| **Begriff** | **Vorkommen** | **Liste** | **df** |
|---|---|---|---|
| braun | $D_1, D_4$ | $1, 4$ | 2 |
| faul | $D_2$ | 2 | 1 |
| fuchs | $D_1$ | 1 | 1 |
| hund | $D_2, D_3, D_4$ | $2, 3, 4$ | 3 |
| schlafen | $D_2$ | 2 | 1 |
| schnell | $D_1, D_3$ | $1, 3$ | 2 |
| springen | $D_1, D_3$ | $1, 3$ | 2 |

**Anfrage 1: `schnell UND hund`.**
Durchschnitt der Listen $\{1, 3\}$ und $\{2, 3, 4\}$ im Gleichschritt:

| **Schritt** | **Zeiger A** | **Zeiger B** | **Aktion** |
|---|---|---|---|
| 1 | 1 | 2 | $1 < 2$: A weiter |
| 2 | 3 | 2 | $3 > 2$: B weiter |
| 3 | 3 | 3 | **Treffer, beide weiter** |
| 4 | Ende | 4 | fertig |

Ergebnis: $\{D_3\}$.

*Der Aufwand ist $\mathcal{O}(|A| + |B|)$, nicht $\mathcal{O}(|A| \cdot |B|)$. Das ist derselbe Gleichschritt wie beim Mischsortieren aus Band 1 und beim Mischverbund aus Kapitel 7.*

**Anfrage 2: `hund UND NICHT faul`.**

$$\{2, 3, 4\} \setminus \{2\} = \{D_3, D_4\}.$$

**Anfrage 3: `braun ODER schnell`.**

$$\{1,4\} \cup \{1,3\} = \{D_1, D_3, D_4\}.$$

**Die Reihenfolge der Verknüpfung.**
Bei mehreren `UND`-Bedingungen beginnt man mit den kürzesten Listen.

| Reihenfolge | Schritte | |
|---|---|---|
| Kürzeste zuerst | $\lvert A\rvert + \lvert B\rvert$, dann klein | gut |
| Längste zuerst | großes Zwischenergebnis | schlecht |

*Das ist dieselbe Regel wie bei der Verbundreihenfolge im Optimierer aus Kapitel 8: Zwischenergebnisse klein halten. Die Dokumenthäufigkeit steht im Wörterbuch und dient hier als Statistik.*

**(b) Die Größe eines Index.**
**Die Sammlung.** Zehn Millionen Dokumente, im Mittel 1000 Wörter, davon 400 verschiedene.

| Größe | Wert | |
|---|---|---|
| Wörter insgesamt | $10^{10}$ | |
| Rohtext | $\approx$ 60 GB | bei 6 B je Wort |
| Einträge im Index | $4 \cdot 10^9$ | Dokument-Begriff-Paare |
| Verschiedene Begriffe | $\approx 10^6$ | |

**Ohne Verdichtung.**

$$4 \cdot 10^9 \cdot 4\ \text{B} = 16\ \text{GB} \qquad (27\,\% \text{ des Rohtexts}).$$

**Mit Differenzen und variabler Länge.**

$$4 \cdot 10^9 \cdot 0{,}75\ \text{B} = 3\ \text{GB} \qquad (5\,\% \text{ des Rohtexts}).$$

| Bestandteil | Größe | Anteil |
|---|---|---|
| Wörterbuch | ≈ 30 MB | im Hauptspeicher |
| Trefferlisten | 3 GB | auf SSD |
| Mit Häufigkeiten | 4,5 GB | |
| **Mit Positionen** | **12 GB** | Faktor 4 |

*Die Faustregel lautet: Ein verdichteter Index ohne Positionen kostet etwa fünf Prozent des Rohtexts, mit Positionen etwa zwanzig. Das ist eine der erstaunlichsten Zahlen des Gebiets, denn man erkauft damit eine Suchzeit, die von der Sammlungsgröße fast unabhängig ist.*

**(c) Den Index aufbauen.**
**Das Problem.** Die Einträge entstehen in Dokumentreihenfolge und müssen in Begriffsreihenfolge liegen. Vier Milliarden Einträge passen nicht in den Hauptspeicher.

1. Dokumente lesen und Einträge sammeln, bis der Speicher voll ist.
2. Diesen Block sortieren und als Teilindex auf die Platte schreiben.
3. Wiederholen, bis alle Dokumente gelesen sind.
4. Alle Teilindizes im Gleichschritt zusammenmischen.

*Das ist das externe Mischsortieren aus Kapitel 7, angewandt auf einen anderen Gegenstand. Der Aufwand beträgt zwei Durchläufe über die Daten, unabhängig davon, wie viele Teilindizes entstehen.*

| Schritt | Daten | Zeit bei 1 GB/s |
|---|---|---|
| Lesen und zerlegen | 60 GB | 60 s + Rechenzeit |
| Teilindizes schreiben | 16 GB | 16 s |
| Mischen: lesen | 16 GB | 16 s |
| Endindex schreiben | 3 GB | 3 s |
| **Gesamt** | | ≈ 2–4 min |

**Die schwierigere Frage: Änderungen.**

| | |
|---|---|
| **Neu aufbauen** | Einfach, teuer; für sich langsam ändernde Sammlungen brauchbar |
| **Zusatzindex** | Ein kleiner Index im Speicher für Neues; Anfragen fragen beide |
| **Mischen in Stufen** | Kleine Indizes werden zu größeren verschmolzen |
| **Löschen** | Über eine Streichliste; die Einträge bleiben zunächst stehen |

*Die dritte Zeile ist genau der protokollstrukturierte Mischbaum aus Kapitel 6. Dieselbe Struktur löst hier dasselbe Problem: viele kleine Änderungen in einer Struktur, die sequentielles Schreiben bevorzugt.*

*Die vierte Zeile hat eine Folge, die Benutzer bemerken: Ein gelöschtes Dokument verschwindet aus den Ergebnissen, zählt aber noch in den Häufigkeitsstatistiken mit, bis das nächste Mischen stattgefunden hat. Die Rangfolge ist dann leicht verzerrt, und niemand merkt es.*

Manning, Raghavan und Schütze[20] sind die maßgebliche Einführung; Zobel und Moffat[28] behandeln invertierte Indizes und ihre Verdichtung erschöpfend; Salton[25] legte die Grundlagen.

# 13 Rangfolge

### Von der Menge zur Reihenfolge

Eine Boolesche Anfrage liefert eine Menge. Bei einer Websuche hat diese Menge Millionen Elemente, und niemand sieht mehr als zehn davon an.

| Position im Ergebnis | Anteil der Klicks | |
|---|---|---|
| 1 | ≈ 28 % | |
| 2 | ≈ 15 % | |
| 3 | ≈ 11 % | |
| 4–10 | ≈ 30 % | |
| ab 11 | ≈ 5 % | **zweite Seite** |

*Die letzte Zeile ist der Grund, warum die Rangfolge fast das ganze Problem ist. Ein Dokument auf Platz zwölf ist praktisch nicht vorhanden, auch wenn der Index es zurückgibt.*

**Drei Gedanken, aus denen sich alles ergibt.**

| | |
|---|---|
| **Häufigkeit** | Ein Wort, das oft im Dokument vorkommt, ist wahrscheinlich wichtig |
| **Seltenheit** | Ein Wort, das in wenigen Dokumenten vorkommt, unterscheidet gut |
| **Länge** | Ein langes Dokument enthält jedes Wort öfter, ohne relevanter zu sein |

*Alle gebräuchlichen Bewertungsfunktionen sind Kombinationen dieser drei Gedanken. Sie unterscheiden sich darin, wie stark jeder eingeht und wie die Sättigung geregelt ist.*

### tf-idf und das Vektorraummodell

GERARD SALTON schlug in den 1970er Jahren vor, Dokumente und Anfragen als Vektoren aufzufassen.

**Definition 13.1.** Das Gewicht eines Begriffs $t$ in Dokument $d$:

$$w_{t,d} = \underbrace{\mathrm{tf}_{t,d}}_{\text{Häufigkeit}} \cdot \underbrace{\log \frac{N}{\mathrm{df}_t}}_{\text{inverse Dokumenthäufigkeit}}.$$

| **Begriff** | $\mathrm{df}_t$ | $N/\mathrm{df}_t$ | idf |
|---|---|---|---|
| *und* | $10^7$ | 1 | **0,00** |
| *haus* | $10^5$ | 100 | 4,61 |
| *quasar* | $10^3$ | $10^4$ | 9,21 |
| *Fachbegriff* | 10 | $10^6$ | 13,82 |

*Die erste Zeile erklärt, warum man Stoppwörter gar nicht gesondert behandeln muss: Ein Wort, das in jedem Dokument vorkommt, bekommt automatisch das Gewicht null. Die Formel erledigt das von selbst.*

**Die Ähnlichkeit.**
Anfrage und Dokument werden als Vektoren aufgefasst; die Ähnlichkeit ist der Kosinus des Winkels zwischen ihnen:

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \, \|\vec{d}\|}.$$

*Die Normierung im Nenner löst das Längenproblem: Ein doppelt so langes Dokument mit denselben Anteilen hat denselben Winkel zur Anfrage. Nur die Richtung zählt, nicht der Betrag.*

### BM25

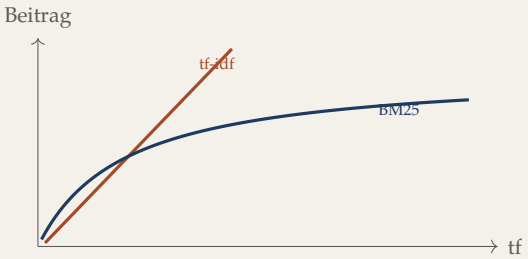
Die heute verbreitetste Funktion stammt aus einer Reihe von Arbeiten um STEPHEN ROBERTSON, entstanden ab den 1970er Jahren.

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{idf}_t \cdot \frac{\mathrm{tf}_{t,d} \, (k_1 + 1)}{\mathrm{tf}_{t,d} + k_1 \left(1 - b + b \, \frac{|d|}{\overline{|d|}}\right)}.$$

| | |
|---|---|
| $k_1 \approx 1{,}2$ | Steuert die Sättigung der Häufigkeit |
| $b \approx 0{,}75$ | Steuert, wie stark die Länge eingeht |
| $\overline{\lvert d \rvert}$ | Mittlere Dokumentlänge in der Sammlung |

**Der wichtigste Unterschied zu tf-idf: die Sättigung.**

| tf | **tf-idf** | **BM25-Anteil** | |
|---|---|---|---|
| 1 | 1,0 | 1,00 | |
| 2 | 2,0 | 1,47 | |
| 5 | 5,0 | 2,03 | |
| 20 | 20,0 | 2,39 | |
| 100 | 100,0 | **2,45** | gesättigt |




*Der Gedanke dahinter ist einleuchtend: Der Unterschied zwischen einem und zwei Vorkommen eines Wortes ist erheblich. Der Unterschied zwischen neunzig und hundert ist bedeutungslos. Bei tf-idf zählt er trotzdem zehnmal so viel.*

*Ohne Sättigung lässt sich die Rangfolge außerdem trivial täuschen: Man wiederholt ein Wort tausendmal. Genau das war in den 1990er Jahren ein verbreitetes Verfahren, und die Sättigung nimmt ihm den Ertrag.*

**Warum BM25 so schwer zu schlagen ist**

Die Funktion ist über vierzig Jahre alt, hat zwei Parameter und schlägt in vielen Vergleichen deutlich aufwendigere Verfahren.

| Verfahren | Güte | Aufwand |
|---|---|---|
| Boolesch | niedrig | sehr gering |
| tf-idf | mittel | gering |
| **BM25** | **gut** | gering |
| Gelernte Rangfolge | besser | hoch |
| Neuronale Verfahren | am besten | sehr hoch |

*Die letzten beiden Zeilen sind besser und um Größenordnungen teurer. Deshalb ist die heute übliche Bauweise zweistufig: BM25 wählt aus Milliarden Dokumenten einige hundert aus, und ein aufwendiges Verfahren ordnet nur diese neu.*

*Damit gilt für BM25 vor allem, dass es nichts Wichtiges übersehen darf. Ob die Reihenfolge innerhalb der ersten hundert stimmt, klärt die zweite Stufe.*

**Aufgabe 13.1.**

(a) Berechnen Sie tf-idf und BM25 für eine kleine Sammlung.
(b) Bestimmen Sie Precision, Recall und deren Kombinationen.
(c) Untersuchen Sie, wie man die ersten $k$ Treffer findet, ohne alles zu bewerten.

**Lösung Schritt für Schritt.**

**(a) Bewertung an einem Beispiel.**
**Die Sammlung.** $N = 1000$ Dokumente, mittlere Länge $\overline{|d|} = 200$.

| **Begriff** | df | idf |
|---|---|---|
| *hund* | 100 | $\log \frac{1000}{100} = 3{,}32$ |
| *tollwut* | 10 | $\log \frac{1000}{10} = 6{,}64$ |

**Zwei Dokumente.**

| | **Länge** | $\mathrm{tf}_{\mathrm{hund}}$ | $\mathrm{tf}_{\mathrm{tollwut}}$ |
|---|---|---|---|
| $D_A$ | 100 | 5 | 2 |
| $D_B$ | 800 | 20 | 4 |

**tf-idf, ohne Normierung.**

$$D_A = 5 \cdot 3{,}32 + 2 \cdot 6{,}64 = 16{,}60 + 13{,}28 = 29{,}88,$$
$$D_B = 20 \cdot 3{,}32 + 4 \cdot 6{,}64 = 66{,}40 + 26{,}56 = \mathbf{92{,}96}.$$

*$D_B$ gewinnt deutlich, weil es viermal so lang ist. Das ist genau der Fehler, den die Längennormierung beheben soll.*

**BM25, mit $k_1 = 1{,}2$ und $b = 0{,}75$.**
**Der Längenfaktor.**

$$D_A: \quad K = 1{,}2\left(1 - 0{,}75 + 0{,}75 \cdot \tfrac{100}{200}\right) = 1{,}2 \cdot 0{,}625 = 0{,}750,$$
$$D_B: \quad K = 1{,}2\left(1 - 0{,}75 + 0{,}75 \cdot \tfrac{800}{200}\right) = 1{,}2 \cdot 3{,}250 = 3{,}900.$$

**Die Beiträge für $D_A$.**

$$\text{hund}: \quad 3{,}32 \cdot \frac{5 \cdot 2{,}2}{5 + 0{,}750} = 3{,}32 \cdot 1{,}913 = 6{,}35,$$
$$\text{tollwut}: \quad 6{,}64 \cdot \frac{2 \cdot 2{,}2}{2 + 0{,}750} = 6{,}64 \cdot 1{,}600 = 10{,}62.$$

$$\mathrm{BM25}(D_A) = \mathbf{16{,}97}.$$

**Die Beiträge für $D_B$.**

$$\text{hund}: \quad 3{,}32 \cdot \frac{20 \cdot 2{,}2}{20 + 3{,}900} = 3{,}32 \cdot 1{,}841 = 6{,}11,$$
$$\text{tollwut}: \quad 6{,}64 \cdot \frac{4 \cdot 2{,}2}{4 + 3{,}900} = 6{,}64 \cdot 1{,}114 = 7{,}40.$$

$$\mathrm{BM25}(D_B) = \mathbf{13{,}51}.$$

| | **tf-idf** | **BM25** | |
|---|---|---|---|
| $D_A$ | 29,88 | **16,97** | |
| $D_B$ | **92,96** | 13,51 | |
| Sieger | $D_B$ | $D_A$ | umgekehrt |

*Die Rangfolge dreht sich um. $D_A$ ist kurz und handelt offensichtlich vom Thema; $D_B$ ist lang und erwähnt es nebenbei. Beide Wirkungen zusammen, Sättigung und Längennormierung, erzeugen dieses Ergebnis.*

**(b) Precision und Recall.**
**Die Lage.** Von 1000 Dokumenten sind 50 relevant. Das System liefert 40 Treffer, davon 30 relevant.

| | **relevant** | **nicht rel.** | **Summe** |
|---|---|---|---|
| **gefunden** | 30 | 10 | 40 |
| **nicht gefunden** | 20 | 940 | 960 |
| **Summe** | 50 | 950 | 1000 |

$$\text{Precision} = \frac{30}{40} = 0{,}75, \qquad \text{Recall} = \frac{30}{50} = 0{,}60.$$

$$F_1 = \frac{2 \cdot 0{,}75 \cdot 0{,}60}{0{,}75 + 0{,}60} = \frac{0{,}90}{1{,}35} = \mathbf{0{,}667}.$$

**Warum das harmonische und nicht das arithmetische Mittel.**

| $P$ | $R$ | **arithm.** | **harmonisch** |
|---|---|---|---|
| 0,75 | 0,60 | 0,675 | 0,667 |
| 1,00 | 0,10 | 0,550 | **0,182** |
| 0,55 | 0,55 | 0,550 | 0,550 |

*Die mittlere Zeile zeigt es: Ein System, das ein einziges Dokument liefert und dabei richtig liegt, hat volle Precision und ist trotzdem nutzlos. Das arithmetische Mittel belohnt es, das harmonische bestraft es.*

**Warum die Genauigkeit als Maß unbrauchbar ist.**

$$\text{Genauigkeit} = \frac{30 + 940}{1000} = 0{,}97.$$

*Siebenundneunzig Prozent, und ein System, das* gar nichts *liefert, käme auf 95 Prozent. Bei seltenen Zielklassen ist dieses Maß wertlos, und zwar aus demselben Grund wie beim diagnostischen Test in Band 7.*

**Maße für Rangfolgen.**

| | |
|---|---|
| **P@k** | Precision unter den ersten $k$; einfach, ignoriert die Reihenfolge darin |
| **MAP** | Mittel über die Precision an jeder Trefferstelle |
| **MRR** | Kehrwert der Position des ersten Treffers |
| **nDCG** | Gewichtet Relevanz mit $1/\log(\text{Rang})$; erlaubt Abstufungen |

*Das letzte Maß ist heute Standard, weil es zweierlei zugleich behandelt: dass frühe Positionen mehr zählen, und dass Relevanz nicht nur ja oder nein ist.*

**(c) Die ersten $k$ ohne vollständige Bewertung.**
**Das Problem.** Eine Anfrage trifft zehn Millionen Dokumente. Gesucht sind die besten zehn.

| **Vorgehen** | **Bewertungen** | **Zeit** |
|---|---|---|
| Alle bewerten, dann sortieren | $10^7$ | $\approx 2$ s |
| Alle bewerten, Halde der Größe 10 | $10^7$ | $\approx 1$ s |
| **Frühzeitig abbrechen** | $\approx 10^5$ | $\approx 10$ ms |

**Das Verfahren.**

1. Die Trefferlisten werden nach Beitrag absteigend sortiert gespeichert.
2. Man führt eine Halde mit den bisher besten $k$ Ergebnissen.
3. Für jede Liste kennt man den größtmöglichen verbleibenden Beitrag.
4. Ist die Summe dieser Obergrenzen kleiner als der $k$-beste bisherige Wert, bricht man ab.

*Schritt 4 ist der Einfall. Man muss nicht wissen, wie gut die restlichen Dokumente sind, sondern nur, dass keines von ihnen gut genug sein* kann. *Das ist dieselbe Art von Schranke wie beim Verzweigen und Beschränken aus Band 3.*

**Ein Zahlenbeispiel.**

| Nach $n$ Dok. | 10.-bester | Obergrenze Rest | weiter? |
|---|---|---|---|
| 100 | 4,2 | 18,3 | ja |
| 1000 | 9,8 | 14,1 | ja |
| 10 000 | 12,6 | 12,9 | ja |
| 50 000 | 13,4 | **13,1** | **abbrechen** |

*Von zehn Millionen wurden fünfzigtausend angesehen, also ein halbes Prozent. Das Ergebnis ist trotzdem exakt: Die Verfahren dieser Art liefern nachweislich dieselben ersten k wie die vollständige Bewertung.*

*Es gibt auch Fassungen, die die Genauigkeit gegen weitere Geschwindigkeit tauschen, indem sie die Schranke lockern. Bei einer Websuche ist das vertretbar; bei einer Patentrecherche wäre es das nicht.*

Robertson und Zaragoza[24] stellen BM25 und seine Herleitung dar; Salton und Mitautoren[25] begründeten das Vektorraummodell; Manning, Raghavan und Schütze[20] behandeln die Bewertung und die Gütemaße gründlich.

# 14 Verbindungen und Vektoren

### Was der Text nicht verrät

BM25 bewertet ein Dokument allein nach seinem Inhalt. Im Web genügt das nicht, denn der Inhalt lässt sich beliebig fälschen.

| | |
|---|---|
| **Das Problem** | Wer eine Seite besitzt, bestimmt ihren Text vollständig |
| **Der Ausweg** | Ein Merkmal benutzen, das *andere* über die Seite aussagen |
| **Die Verkörperung** | Verweise von einer Seite auf eine andere |

*Ein Verweis ist eine Empfehlung, die nicht vom Empfänger stammt. Das ist der ganze Gedanke, und er wurde 1998 unabhängig von zwei Gruppen ausgearbeitet.*

**PageRank.**
SERGEY BRIN und LAWRENCE PAGE definierten die Bedeutung einer Seite rekursiv: Eine Seite ist wichtig, wenn wichtige Seiten auf sie verweisen.

$$PR(p) = \frac{1-d}{N} + d \sum_{q \to p} \frac{PR(q)}{|q|},$$

mit dem Dämpfungsfaktor $d \approx 0{,}85$ und der Zahl der ausgehenden Verweise $|q|$.

| | |
|---|---|
| **Der erste Term** | Jede Seite bekommt einen Grundwert; sonst könnten Teile des Netzes leer ausgehen |
| **Die Summe** | Jede verweisende Seite gibt ihren Wert weiter, geteilt durch die Zahl ihrer Verweise |
| **Die Teilung** | Wer auf tausend Seiten verweist, empfiehlt jede einzelne weniger |

*Die Formel ist zirkulär, und das ist kein Fehler. Sie beschreibt einen Fixpunkt, und der lässt sich durch wiederholtes Einsetzen finden. Es ist dieselbe Struktur wie bei der Datenflussanalyse aus Band 6.*

**Die andere Lesart.**
PageRank ist die stationäre Verteilung eines zufälligen Spaziergangs: Ein Wanderer folgt mit Wahrscheinlichkeit $d$ einem zufälligen Verweis und springt sonst irgendwohin.

*Der Wert einer Seite ist dann der Anteil der Zeit, den der Wanderer auf ihr verbringt. Der Dämpfungsfaktor sorgt dafür, dass er aus Sackgassen und Kreisen wieder herauskommt.*

### Suche nach Bedeutung statt nach Wörtern

Ein invertierter Index findet Wörter. Wer „Auto“ sucht, findet keine Seite über „Kraftfahrzeuge“.

| Problem | Beispiel | Wirkung |
|---|---|---|
| Synonyme | Auto, Wagen, Pkw | Treffer fehlen |
| Mehrdeutigkeit | Bank, Golf | falsche Treffer |
| Umschreibung | „Wie heiße ich?“ | keine Wortübereinstimmung |

**Einbettungen.**
Man bildet Wörter, Sätze oder ganze Dokumente auf Vektoren in einem Raum von einigen hundert Dimensionen ab, so dass Ähnliches nah beieinander liegt.

| Vergleich | Kosinus | Deutung |
|---|---|---|
| Auto – Wagen | 0,89 | sehr ähnlich |
| Auto – Fahrrad | 0,61 | verwandt |
| Auto – Bank | 0,12 | unverwandt |

*Der Vergleich benutzt denselben Kosinus wie das Vektorraummodell aus Kapitel 13. Der Unterschied liegt darin, was die Achsen bedeuten: dort ein Begriff je Dimension, hier gelernte Merkmale ohne benennbare Bedeutung.*

| | Invertierter Index | Vektorsuche |
|---|---|---|
| Findet | Wörter | **Bedeutung** |
| Dimensionen | $10^6$, sehr dünn | $10^3$, dicht |
| Genaue Begriffe | sehr gut | schwächer |
| Umschreibungen | schlecht | gut |
| Erklärbarkeit | gut | gering |

*Die dritte Zeile wird oft übersehen. Wer eine Artikelnummer oder einen Eigennamen sucht, ist mit dem invertierten Index besser bedient. Deshalb benutzen ernsthafte Systeme beides und führen die Ergebnisse zusammen.*

### Nachbarsuche in hohen Dimensionen

Die Aufgabe lautet: Finde zu einem Vektor die ähnlichsten aus einer Sammlung von Millionen.

| **Verfahren** | **Aufwand** | **bei $10^7$, $d = 768$** |
|---|---|---|
| Alle vergleichen | $\mathcal{O}(nd)$ | $\approx 3$ s |
| $k$-d-Baum | $\mathcal{O}(n)$ bei großem $d$ | nutzlos |
| **HNSW** | $\mathcal{O}(\log n)$ | $\approx 1$ ms |

*Die mittlere Zeile ist bemerkenswert. Baumstrukturen, die in zwei oder drei Dimensionen hervorragend arbeiten, versagen ab etwa zwanzig Dimensionen vollständig und werden langsamer als der vollständige Durchlauf.*

**Der Fluch der Dimensionen.**

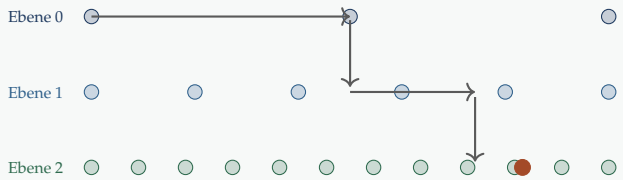
| *d* | **nächster Abstand** | **fernster Abstand** | **Verhältnis** |
|---|---|---|---|
| 2 | 0,05 | 1,31 | 26,2 |
| 10 | 0,62 | 1,88 | 3,0 |
| 100 | 3,42 | 4,61 | 1,35 |
| 1000 | 12,1 | 13,4 | **1,11** |

*In tausend Dimensionen ist der nächste Nachbar kaum näher als der fernste. Damit verliert jede Struktur, die auf Abstandsschranken beruht, ihre Wirkung: Man kann nichts mehr ausschließen.*

*Der Ausweg besteht darin, die Genauigkeit aufzugeben. Man sucht nicht mehr den nächsten Nachbarn, sondern einen sehr nahen, und akzeptiert eine Trefferquote von etwa 95 bis 99 Prozent.*

### Der navigierbare Kleine-Welt-Graph

HNSW baut einen Graphen, in dem jeder Vektor mit einigen nahen Nachbarn verbunden ist, und legt darüber mehrere Ebenen mit immer weniger Knoten.




1. Man beginnt in der obersten, dünn besetzten Ebene.
2. Man geht gierig zum Nachbarn, der dem Ziel am nächsten liegt.
3. Geht es dort nicht mehr näher, steigt man eine Ebene tiefer.
4. Unten angekommen, sammelt man die besten $k$ ein.

*Die oberen Ebenen liefern große Sprünge, die unteren die Feinarbeit. Es ist derselbe Aufbau wie bei einer Sprungliste aus Band 1, nur im Raum statt auf einer Ordnung.*

| Verfahren | Zeit | Trefferquote | Speicher |
|---|---|---|---|
| Vollständig | 3 s | 100 % | 30 GB |
| HNSW | 1 ms | 98 % | 45 GB |
| HNSW + Quantisierung | 1,5 ms | 94 % | 4 GB |

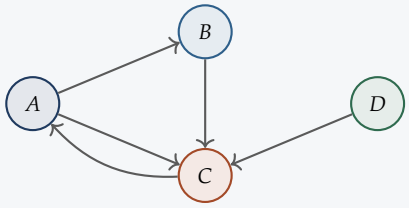
*Die letzte Zeile benutzt die Produktquantisierung: Der Vektor wird in Abschnitte zerlegt, und jeder Abschnitt wird durch den nächstgelegenen aus einem kleinen Vorrat ersetzt. Das ist verlustbehaftete Verdichtung im Sinne von Band 7, angewandt auf Vektoren.*

### Aufgabe 14.1.

(a) Berechnen Sie PageRank für ein kleines Netz.
(b) Untersuchen Sie, wie sich Vektorsuche und Textsuche verbinden lassen.
(c) Rechnen Sie die Produktquantisierung durch.

### Lösung Schritt für Schritt.

**(a) PageRank für vier Seiten.**
**Das Netz.** $A \to B$, $A \to C$, $B \to C$, $C \to A$, $D \to C$.




| Seite | verweist auf | $\lvert q \rvert$ | bekommt von |
|---|---|---|---|
| $A$ | $B, C$ | 2 | $C$ |
| $B$ | $C$ | 1 | $A$ |
| $C$ | $A$ | 1 | $A, B, D$ |
| $D$ | $C$ | 1 | — |

**Die Gleichungen** mit $d = 0{,}85$ und $N = 4$, also Grundwert $0{,}0375$:

$$
\begin{aligned}
PR(A) &= 0{,}0375 + 0{,}85 \cdot PR(C), \\
PR(B) &= 0{,}0375 + 0{,}85 \cdot \tfrac{PR(A)}{2}, \\
PR(C) &= 0{,}0375 + 0{,}85 \left( \tfrac{PR(A)}{2} + PR(B) + PR(D) \right), \\
PR(D) &= 0{,}0375.
\end{aligned}
$$

**Die Iteration, Start bei** $0{,}25$**.**

| **Runde** | $PR(A)$ | $PR(B)$ | $PR(C)$ | $PR(D)$ |
|---|---|---|---|---|
| 0 | 0,2500 | 0,2500 | 0,2500 | 0,2500 |
| 1 | 0,2500 | 0,1438 | 0,5771 | 0,0375 |
| 2 | 0,5280 | 0,1438 | 0,2917 | 0,0375 |
| 3 | 0,2854 | 0,2619 | 0,4835 | 0,0375 |
| 5 | 0,4212 | 0,1918 | 0,4131 | 0,0375 |
| 10 | 0,3855 | 0,2013 | 0,4232 | 0,0375 |
| 20 | **0,3877** | **0,2023** | **0,4232** | **0,0375** |

| **Rang** | **Seite** | **Begründung** |
|---|---|---|
| 1 | *C* | drei eingehende Verweise |
| 2 | *A* | einer, aber von *C* |
| 3 | *B* | einer, von *A* geteilt |
| 4 | *D* | **keiner** |

*Bemerkenswert ist Platz zwei. A hat nur einen eingehenden Verweis, aber er kommt von der wichtigsten Seite und wird nicht geteilt. Genau das unterscheidet PageRank vom bloßen Zählen der Verweise.*

**Die Konvergenz.**

| **Runde** | **Änderung** | |
|---|---|---|
| $1 \to 2$ | 0,278 | |
| $5 \to 6$ | 0,036 | |
| $10 \to 11$ | 0,003 | |
| $20 \to 21$ | $2 \cdot 10^{-5}$ | ausreichend |

*Die Änderung schrumpft je Runde etwa um den Faktor $d = 0{,}85$. Für die gewünschte Genauigkeit braucht man also rund fünfzig Runden, unabhängig von der Netzgröße. Bei Milliarden Seiten ist das die entscheidende Eigenschaft.*

**(b) Textsuche und Vektorsuche verbinden.**
**Das Problem.** Beide liefern Rangfolgen mit unvergleichbaren Bewertungen.

| Dokument | BM25 | Kosinus | |
|---|---|---|---|
| $D_1$ | 18,4 | 0,71 | |
| $D_2$ | 12,1 | 0,88 | |
| $D_3$ | — | 0,83 | nur im einen |
| $D_4$ | 15,7 | — | nur im anderen |

*Die Werte lassen sich nicht addieren: Der eine ist unbeschränkt, der andere liegt zwischen minus eins und eins. Und für manche Dokumente fehlt einer von beiden.*

**Die gebräuchliche Lösung: nach Rängen zusammenführen.**

$$\mathrm{RRF}(d) = \sum_i \frac{1}{k + \mathrm{Rang}_i(d)}, \qquad k \approx 60.$$

| Dok. | Rang T | Rang V | RRF | Platz |
|---|---|---|---|---|
| $D_1$ | 1 | 3 | 0,01639 + 0,01587 | **1** |
| $D_2$ | 3 | 1 | 0,01587 + 0,01639 | **1** |
| $D_3$ | — | 2 | 0,01613 | 3 |
| $D_4$ | 2 | — | 0,01613 | 3 |

*Der Kunstgriff besteht darin, die Bewertungen wegzuwerfen und nur die Ränge zu benutzen. Damit entfällt jede Frage nach Vergleichbarkeit. Die Konstante k dämpft den Einfluss der vordersten Plätze und verhindert, dass ein einzelnes System das Ergebnis bestimmt.*

*Das Verfahren ist erstaunlich robust und lässt sich auf beliebig viele Quellen erweitern. Es setzt nichts über die Bewertungen voraus, sondern nur, dass jede Quelle eine sinnvolle Ordnung liefert.*

**(c) Produktquantisierung.**
**Der Ausgangspunkt.** Ein Vektor mit 768 Zahlen zu je 4 Byte belegt 3072 Byte.

| Größe | Wert | |
|---|---|---|
| Vektoren | $10^7$ | |
| Je Vektor | 3072 B | |
| **Gesamt** | **30,7 GB** | passt nicht in den Speicher |

**Das Verfahren.**

1. Den Vektor in $m = 96$ Abschnitte zu je 8 Zahlen zerlegen.
2. Für jeden Abschnitt vorab 256 Vertreter bestimmen (Clusterzentren).
3. Jeden Abschnitt durch die Nummer seines nächsten Vertreters ersetzen.
4. Der Vektor besteht nun aus 96 Byte.

$$\text{Faktor} = \frac{3072}{96} = \mathbf{32}.$$

| Abschnitte $m$ | Byte | Faktor | Trefferquote |
|---|---|---|---|
| 192 | 192 | 16 | 97 % |
| 96 | 96 | 32 | 94 % |
| 48 | 48 | 64 | 88 % |
| 24 | 24 | 128 | 76 % |

**Warum das Vergleichen dabei sogar schneller wird.**
Man berechnet vorab für jeden Abschnitt die Abstände des Anfragevektors zu allen 256 Vertretern.

| Schritt | Aufwand | |
|---|---|---|
| Tabelle aufbauen | $96 \cdot 256$ Abstände | einmal je Anfrage |
| Ein Vektor vergleichen | 96 **Nachschläge** | statt 768 Multiplikationen |
| **Verhältnis** | $\approx$ 8-fach | zusätzlich zum Speichergewinn |

*Aus einer Rechnung wird ein Tabellenzugriff. Das ist derselbe Gedanke wie bei jeder Vorberechnung, und er wirkt hier doppelt: Der Vergleich wird schneller, und die Daten passen in den Hauptspeicher, was den Zugriff nochmals um Größenordnungen beschleunigt.*

**Die Trefferquote ist keine Nebensache**

Eine Trefferquote von 94 Prozent bedeutet, dass jeder sechzehnte richtige Nachbar fehlt.

| | |
|---|---|
| **Produkt-empfehlung** | Unbedenklich; es gibt genug ähnliche Artikel |
| **Duplikatsuche** | Bedenklich; man übersieht Dubletten |
| **Rechtsrecherche** | Nicht vertretbar; ein übersehenes Urteil kann entscheidend sein |

*Und die Quote wird meist gegen einen Maßstab gemessen, der selbst nur eine Näherung ist: die exakte Nachbarschaft im Einbettungsraum. Ob diese der* inhaltlichen *Ähnlichkeit entspricht, ist eine ganz andere Frage, die niemand mitmisst.*

Brin und Page[5] beschrieben PageRank; Kleinberg[18] entwickelte gleichzeitig einen verwandten Ansatz; Malkov und Yashunin[19] entwarfen HNSW; Jégou, Douze und Schmid[17] führten die Produktquantisierung ein.

# 15 Der Stand im Jahr 2026

**Was gelöst ist und was nicht**

| Frage | Stand | Bemerkung |
|---|---|---|
| Relationales Modell | gelöst | seit 1970 unverändert tragfähig |
| Indexstrukturen | gelöst | B-Baum seit 1972 |
| Transaktionen | gelöst | auf einem Rechner |
| Invertierter Index | gelöst | |
| Optimierung | in Arbeit | **das ewige Sorgenkind** |
| Verteilte Transaktionen | in Arbeit | teuer, aber machbar (Band 9) |
| Text und Vektoren vereinen | in Arbeit | seit etwa 2022 |
| Selbstverwaltung | in Arbeit | Indizes automatisch wählen |
| Kostenschätzung | offen | **Fehler wachsen exponentiell** |
| Relevanz messen | offen | bleibt Urteilssache |
| Schema und Wirklichkeit | offen | Daten altern anders als Modelle |

*Die erste Zeile des unteren Abschnitts ist die folgenreichste. Der Optimierer schätzt die Größe von Zwischenergebnissen, und diese Schätzungen multiplizieren sich über die Verbundstufen. Ein Fehler von Faktor zwei je Stufe wird bei fünf Verbunden zu Faktor zweiunddreißig.*

| Verbunde | Schätzfehler | Folge |
|---|---|---|
| 1 | ≈ 1,5 | unbedenklich |
| 3 | ≈ 4 | meist noch tragbar |
| 5 | ≈ 15 | falscher Plan möglich |
| 8 | ≈ 100 | falscher Plan wahrscheinlich |

*Deshalb sind Abfragen mit vielen Verbunden so unberechenbar, und deshalb bieten alle Systeme Möglichkeiten an, dem Optimierer den Plan vorzuschreiben. Das ist ein Eingeständnis und gelegentlich unvermeidlich.*

### Fünf Entwicklungen

**Erstens: Speicher und Rechenwerk werden getrennt.**
Die klassische Bauweise hält Daten auf den Platten desselben Rechners, der auch rechnet. Das ändert sich.

| | klassisch | getrennt |
|---|---|---|
| Daten liegen | lokal | im Objektspeicher |
| Rechenknoten | **fest** | beliebig zuschaltbar |
| Bei Ausfall | Wiederanlauf | anderer Knoten übernimmt |
| Kosten | für Höchstlast | nach Verbrauch |
| Zugriffszeit | 0,1 ms | 5–20 ms |

*Die letzte Zeile ist der Preis, und er wird durch aggressive Zwischenspeicherung gemildert. Der Gewinn steht in der zweiten Zeile: Eine Auswertung, die sonst Stunden bräuchte, bekommt für zehn Minuten hundert Knoten und wird danach abgeschaltet.*

**Zweitens: Zeilen und Spalten in einem System.**
Die Trennung zwischen Systemen für laufende Geschäfte und solchen für Auswertungen weicht auf.

| | Zeilenweise | Spaltenweise |
|---|---|---|
| Gut für | einzelne Sätze | **Aggregation** |
| Verdichtung | mäßig | sehr gut |
| Einfügen | schnell | langsamer |
| Typische Last | Buchungen | Berichte |

*Heutige Systeme halten beides: die aktuellen Daten zeilenweise, die älteren spaltenweise, und schieben im Hintergrund von einer Form in die andere. Damit entfällt die nächtliche Überführung in ein getrenntes Auswertungssystem, an der viele Betriebe hingen.*

*Die zweite Zeile hängt unmittelbar an Band 7: Eine Spalte enthält gleichartige Werte, und die Entropie einer solchen Folge ist erheblich niedriger als die einer gemischten Zeile. Verdichtungsfaktoren von zehn sind üblich.*

**Drittens: Vektoren als Datentyp.**
Was 2023 noch eigene Produkte verlangte, ist heute eine Spaltenart wie jede andere.

```
SELECT titel, inhalt
FROM dokument
WHERE bereich = 'Kardiologie'
ORDER BY einbettung <=> :anfragevektor
LIMIT 10;
```

*Bemerkenswert ist die dritte Zeile. Eine gewöhnliche Bedingung schränkt ein, und die Vektorsuche arbeitet nur auf dem Rest. Genau das leistet ein getrenntes Vektorsystem schlecht, weil ihm die übrigen Spalten fehlen.*

| | eigenes System | in der Datenbank |
|---|---|---|
| Filter davor | umständlich | selbstverständlich |
| Transaktionen | selten | vorhanden |
| Abgleich nötig | ja | nein |
| Spitzenleistung | höher | ausreichend |

*Die dritte Zeile ist der stärkste Punkt. Zwei Systeme, die dieselben Daten führen, laufen auseinander, und der Abgleich ist eine dauerhafte Fehlerquelle.*

**Viertens: Suche in zwei Stufen als Standard.**

| Stufe | Verfahren | Umfang |
|---|---|---|
| Auswahl | BM25 + Vektoren | $10^9 \rightarrow 10^3$ |
| Neuordnung | aufwendiges Modell | $10^3 \rightarrow 10^1$ |

*Der Aufwand des zweiten Schritts liegt je Dokument etwa tausendmal über dem des ersten. Er lohnt sich nur, weil er auf so wenige Dokumente angewandt wird. Die erste Stufe muss deshalb nicht gut ordnen, sondern nur nichts Wichtiges verlieren.*

**Fünftens: Abfragen aus natürlicher Sprache.**
Sprachmodelle erzeugen SQL aus einer Frage in Umgangssprache. Das ändert weniger, als es verspricht.

| | |
|---|---|
| **Funktioniert gut** | Einfache Abfragen über ein bekanntes, gut benanntes Schema |
| **Funktioniert schlecht** | Mehrere Verbunde, feine Unterscheidungen, unklare Fachbegriffe |
| **Bleibt** | **Jemand muss das Ergebnis prüfen** |

*Die dritte Zeile ist entscheidend. Eine falsche Abfrage liefert keine Fehlermeldung, sondern ein plausibles falsches Ergebnis. Wer die Zahl nicht selbst nachvollziehen kann, kann nicht erkennen, ob sie stimmt.*

*Es gibt eine Rückwirkung, die man begrüßen darf: Ein sauber benanntes und dokumentiertes Schema wird plötzlich wieder wertvoll, weil es die Erfolgsquote spürbar hebt.*

*Wer die Entwicklungen dieses Abschnitts einordnen will, lese Stonebrakers Streitschrift von 2007[27] daneben: Fast jede der fünf Entwicklungen ist eine Antwort auf eine dort aufgeworfene Frage, und keine fällt so aus, wie er es vorhergesagt hatte.*

### Drei Einsichten

**Erstens: Das Modell überlebt die Umsetzung.**

| Idee | Jahr | heute |
|---|---|---|
| Relationen | 1970 | unverändert |
| B-Baum | 1972 | unverändert |
| Normalformen | 1972 | unverändert |
| ACID | 1983 | unverändert |
| Vektorraum | 1975 | im Kern unverändert |

*Die Systeme wurden mehrfach ausgetauscht, die Begriffe nicht. Wer das relationale Modell versteht, versteht jedes Produkt in wenigen Tagen. Wer nur ein Produkt kennt, fängt bei jedem Wechsel von vorn an.*

**Zweitens: Die Ebenen hängen zusammen.**

| | |
|---|---|
| **Eine Abfrage** | ist langsam, weil ein Index fehlt |
| **Ein Index** | fehlt, weil das Schema falsch geschnitten ist |
| **Das Schema** | ist falsch, weil die Anforderungen unklar waren |

*Deshalb behandelt dieser Band beide Richtungen. Wer nur die Oberfläche kennt, sucht den Fehler an der falschen Stelle; wer nur die Innereien kennt, optimiert etwas, das gar nicht hätte sein müssen.*

**Drittens: Es gibt zwei Arten von Fragen.**

| | Datenbank | Suche |
|---|---|---|
| Frage | festgelegt | ungefähr |
| Antwort | richtig | **passend** |
| Prüfung | Rechnung | **Urteil** |
| Fortschritt | schnellere Ausführung | **bessere Bewertung** |

*Die letzte Zeile erklärt, warum die beiden Hälften dieses Bandes sich so verschieden entwickelt haben. Bei Datenbanken ist seit dreißig Jahren klar, was richtig ist, und die Arbeit steckt in der Geschwindigkeit. Beim Suchen ist die Geschwindigkeit gelöst, und die Frage nach dem Richtigen bleibt offen.*

*Band 11 wendet sich den Graphen und der Netzwerkanalyse zu. PageRank aus Kapitel 14 war ein erster Vorgeschmack; dort geht es um die Verfahren selbst, um Zentralitätsmaße, Gemeinschaften und um die Frage, was sich aus einer Verbindungsstruktur ablesen lässt.*

*Zeige mir deine Tabellen, und ich brauche deine Programme nicht;*
*zeige mir deine Programme, und ich weiß meist*
*immer noch nicht, was sie tun.*

— eine Zuspitzung nach einer bekannten Bemerkung von
FRED BROOKS *über Datenstrukturen und Ablaufpläne.*
*Sie gilt hier besonders, weil das Schema die einzige*
*Stelle ist, an der ein Betrieb aufschreiben muss,*
*wovon er eigentlich handelt.*

# Part V

# Anhang

# A Zahlen und Formeln

**Größenordnungen**

| Vorgang | Zeit | Bemerkung |
|---|---|---|
| Zugriff im Puffer | ≈ 100 ns | |
| Zugriff SSD | ≈ 100 µs | |
| Zugriff Festplatte | ≈ 8 ms | |
| Erzwungenes Schreiben, SSD | ≈ 200 µs | je `COMMIT` |
| Erzwungenes Schreiben, Platte | ≈ 10 ms | 100 TA/s |
| B-Baum-Suche, $10^9$ Sätze | 4 Ebenen | meist 1 Zugriff |
| Sequentiell lesen, SSD | 2 GB/s | |

**Formeln**

| | |
|---|---|
| B-Baum-Höhe | $h = \lceil \log_f n \rceil$ |
| Verbundgröße | $\lvert R \bowtie S \rvert \approx \frac{\lvert R \rvert \, \lvert S \rvert}{\max(V_R, V_S)}$ |
| Blockverbund | $B_R + \lceil B_R/(M-2) \rceil \cdot B_S$ |
| Externes Sortieren | $2B(1 + \lceil \log_{M-1}(B/M) \rceil)$ |
| tf-idf | $\mathrm{tf} \cdot \log(N/\mathrm{df})$ |
| BM25 | $\sum \mathrm{idf} \cdot \frac{\mathrm{tf}(k_1+1)}{\mathrm{tf} + k_1(1 - b + b\frac{\lvert d \rvert}{\overline{\lvert d \rvert}})}$ |
| $F_1$ | $\frac{2PR}{P+R}$ |
| PageRank | $\frac{1-d}{N} + d \sum_{q \to p} \frac{PR(q)}{\lvert q \rvert}$ |

## Isolationsstufen und Anomalien

| Stufe | schmutzig | nicht wieder-holb. | Phantom | Schiefe |
|---|---|---|---|---|
| `READ UNCOMMITTED` | ja | ja | ja | ja |
| `READ COMMITTED` | nein | ja | ja | ja |
| `REPEATABLE READ` | nein | nein | ja | ja |
| Momentaufnahme | nein | nein | nein | **ja** |
| `SERIALIZABLE` | nein | nein | nein | nein |

*Die vorletzte Zeile ist die praktisch wichtigste und die am wenigsten bekannte: Eine Momentaufnahme verhindert Phantome und lässt die Schreibschiefe zu.*

# B Landkarte des Bandes

### Wo was steht

| Kap. | Thema | Verweise |
|---|---|---|
| 1 | Wozu ein System | Band 5 (Dateien) |
| 2 | Relationales Modell | Mathematikreihe (Mengen) |
| 3 | SQL | Band 6 (Sprachentwurf) |
| 4 | Entwurf, Normalformen | Band 2 (Abhängigkeiten) |
| 5 | Speicherung | Band 5 (Puffer), Band 7 |
| 6 | Indexstrukturen | **Band 1 (Bäume)**, Band 5 |
| 7 | Ausführung | Band 1 (Sortieren) |
| 8 | Optimierung | **Band 6 (Übersetzer)** |
| 9 | Transaktionen | Band 5 (Wettläufe) |
| 10 | Nebenläufigkeit | Band 5 (Sperren, Verklemmung) |
| 11 | Wiederanlauf | Band 5 (Journal), Band 9 |
| 12 | Invertierter Index | Band 7 (Verdichtung) |
| 13 | Rangfolge | Band 7 (Maße), Band 3 |
| 14 | Verbindungen, Vektoren | **Band 11 (Graphen)** |
| 15 | Stand 2026 | Band 13 |

### Sechs Verbindungen zur Mathematikreihe

| | |
|---|---|
| Relationale Algebra | Mengenlehre, Operationen auf Relationen |
| Normalformen | Funktionale Abhängigkeiten, Hüllenbildung |
| Kostenschätzung | Wahrscheinlichkeitsrechnung, Unabhängigkeit |
| Vektorraummodell | Skalarprodukt, Winkel, Normen |
| PageRank | Markow-Ketten, Eigenvektoren, Fixpunkte |
| Nachbarsuche | Metrische Räume, Konzentration in hohen Dimensionen |

*Die fünfte Zeile ist die elegante Verbindung: PageRank ist der Eigenvektor der Übergangsmatrix zum Eigenwert eins, und die Iteration aus Kapitel 14 ist nichts anderes als die Potenzmethode.*

# C Glossar

**Begriffe**

| | |
|---|---|
| **ACID** | Atomarität, Konsistenz, Isolation, Dauerhaftigkeit |
| **ARIES** | Wiederanlauf in drei Durchgängen |
| **BM25** | Verbreitetste Bewertungsfunktion beim Suchen |
| **B-Baum** | Auf Blockspeicher zugeschnittener Suchbaum |
| **Einbettung** | Abbildung von Bedeutung auf einen Vektor |
| **Fremdschlüssel** | Verweis auf einen Satz einer anderen Tabelle |
| **Idf** | Inverse Dokumenthäufigkeit; Seltenheit eines Begriffs |
| **Invertierter Index** | Zu jedem Wort seine Dokumente |
| **LSM-Baum** | Struktur für schreiblastige Lasten |
| **Mehrversionen** | Alte Fassungen bleiben für Leser stehen |
| **Normalform** | Zerlegung, die Redundanz vermeidet |
| **Phantom** | Neuer Satz erscheint in wiederholter Abfrage |
| **Protokoll** | Aufzeichnung aller Änderungen vor ihrer Ausführung |
| **Recall** | Anteil der gefundenen relevanten Dokumente |
| **Schreibschiefe** | Anomalie ohne Schreibkonflikt |
| **Selektivität** | Anteil der Sätze, die eine Bedingung erfüllen |
| **Serialisierbar** | Wirkt wie eine Ausführung nacheinander |
| **Sichtbarkeit** | Welche Fassung eine Transaktion sieht |
| **Trefferliste** | Dokumentnummern zu einem Begriff |
| **Zweiphasig** | Erst nur nehmen, dann nur freigeben |

# D Zeittafel

**Von 1961 bis 2026**

| | |
|---|---|
| 1961 | Bachman: IDS |
| 1966 | IMS, hierarchisch |
| 1968 | Salton: SMART |
| **1970** | **Codd: Das relationale Modell** |
| **1972** | **Bayer, McCreight: B-Bäume** |
| 1972 | Codd: Normalformen |
| 1973 | System R beginnt |
| 1974 | Chamberlin, Boyce: SEQUEL |
| 1975 | Salton u. a.: Vektorraummodell |
| 1976 | Chen: Entity-Relationship |
| 1976 | Ingres in Berkeley |
| **1979** | **Selinger u. a.: kostenbasierte Optimierung** |
| 1979 | Oracle liefert aus |
| 1981 | Codd: Turing-Award |
| **1983** | **Härder, Reuter: ACID** |
| 1986 | SQL-86 |
| 1987 | Bernstein u. a.: Nebenläufigkeit |
| 1992 | Mohan u. a.: ARIES |
| 1993 | Graefe: Ausführungsverfahren |
| 1994 | Robertson u. a.: BM25 |
| 1995 | Berenson u. a.: Kritik der Isolationsstufen |
| 1996 | O'Neil u. a.: LSM-Baum |

| | |
|---|---|
| **1998** | **Brin, Page: PageRank** |
| 1999 | Kleinberg: HITS |
| 2003 | Google: GFS und MapReduce |
| 2005 | Spaltenspeicher: C-Store |
| 2006 | Bigtable |
| 2007 | Dynamo (Band 9) |
| 2007 | Stonebraker: Ende einer Architekturepoche |
| 2008 | Manning u. a.: Lehrbuch zum Retrieval |
| 2009 | Robertson, Zaragoza: BM25 im Rückblick |
| 2009 | „NoSQL“ als Sammelbegriff |
| 2011 | Jégou u. a.: Produktquantisierung |
| **2012** | **Spanner: verteilt und konsistent** |
| 2013 | Abadi u. a.: Spaltenspeicherung |
| 2016 | Einbettungen im Retrieval |
| **2018** | **Malkov, Yashunin: HNSW** |
| 2020 | Trennung von Speicher und Rechenwerk |
| 2023 | Vektordatenbanken als Produktklasse |
| 2024 | Vektoren als Spaltenart in SQL-Systemen |
| 2026 | Zweistufige Suche als Standard |

# E Kommentierte Quellen


**[1]** Abadi, D., Boncz, P., Harizopoulos, S., u. a.: *The Design and Implementation of Modern Column-Oriented Database Systems*. Foundations and Trends in Databases 5(3), 2013.

*Die umfassende Darstellung der Spaltenspeicherung: Verdichtung, verzögerte Materialisierung, Vektorverarbeitung. Erklärt sauber, warum der Vorteil bei Auswertungen nicht ein Faktor zwei, sondern ein Faktor zwanzig ist.*

**[2]** Bayer, R., McCreight, E.: *Organization and Maintenance of Large Ordered Indices*. Acta Informatica 1(3), 1972.

***Der B-Baum.*** *Eine Struktur, die auf Blockspeicher zugeschnitten ist und seither praktisch unverändert in jedem Datenbanksystem steckt. Bemerkenswert kurz und klar geschrieben.*

**[3]** Berenson, H., Bernstein, P., Gray, J., u. a.: *A Critique of ANSI SQL Isolation Levels*. SIGMOD 1995.

*Zeigte, dass die Norm die Isolationsstufen über Anomalien definiert und dabei Fälle übersieht. Führte die Momentaufnahme-Isolation und die Schreibschiefe in die Diskussion ein. Pflichtlektüre für jeden, der Transaktionen benutzt.*

**[4]** Bernstein, P., Hadzilacos, V., Goodman, N.: *Concurrency Control and Recovery in Database Systems*. Addison-Wesley 1987.

*Die maßgebliche theoretische Darstellung. Serialisierbarkeit, Sperrverfahren, Wiederanlauf, verteilte Transaktionen, alles mit vollständigen Beweisen. Frei verfügbar und trotz des Alters unersetzt.*

**[5]** Brin, S., Page, L.: *The Anatomy of a Large-Scale Hypertextual Web Search Engine*. WWW 1998.

*Beschreibt PageRank und den Aufbau einer vollständigen Suchmaschine. Lesenswert auch wegen der Bescheidenheit: Die Autoren beschreiben ein studentisches Projekt und rechnen vor, was die Speicherung des Webs kosten würde.*

**[6]** Chamberlin, D., Boyce, R.: *SEQUEL: A Structured English Query Language*. SIGFIDET 1974.

*Der Ursprung von SQL. Das Ziel war ausdrücklich, eine Sprache für Nichtprogrammierer zu schaffen, was nur teilweise gelang. Man sieht dem Aufsatz an, wie viele spätere Eigenheiten aus diesem Anspruch folgen.*

**[7]** Codd, E. F.: *A Relational Model of Data for Large Shared Data Banks*. CACM 13(6), 1970.

***Der Gründungsaufsatz.*** *Elf Seiten, die eine ganze Industrie begründeten. Der Kernpunkt ist nicht die Tabellenform, sondern die Datenunabhängigkeit. Frei verfügbar und gut lesbar.*

**[8]** Codd, E. F.: *Further Normalization of the Data Base Relational Model*. 1972.

*Die ersten drei Normalformen. Codds Kriterium ist durchgehend dasselbe: Redundanz erzeugt Änderungsanomalien, und die Zerlegung beseitigt sie.*

**[9]** Date, C. J.: *An Introduction to Database Systems*. 8. Aufl., Pearson 2003.

*Der gründlichste und streitbarste Erklärer des relationalen Modells. Date verteidigt Codds Ideen gegen ihre Verwässerung in SQL, teils polemisch und meist mit guten Gründen. Besonders lesenswert zu fehlenden Werten.*

**[10]** Elmasri, R., Navathe, S.: *Fundamentals of Database Systems*. 7. Aufl., Pearson 2016.

*Ein breites Lehrbuch mit starkem Gewicht auf Entwurf und Modellierung. Gut für den Weg vom Entity-Relationship-Modell zur fertigen Tabellenstruktur.*

**[11]** Garcia-Molina, H., Ullman, J., Widom, J.: *Database Systems: The Complete Book*. 2. Aufl., Pearson 2008.

***Das gründlichste Lehrbuch zu den Innereien.*** *Speicherung, Indizes, Ausführung, Optimierung, Transaktionen, alles mit durchgerechneten Kostenmodellen. Wenn Sie ein Buch zur Vertiefung wählen, dann dieses.*

**[12]** Graefe, G.: *Query Evaluation Techniques for Large Databases*. ACM Computing Surveys 25(2), 1993.

*Über hundert Seiten Übersicht über die Ausführungsverfahren: Sortieren, Streuen, Verbundvarianten, Parallelität. Die maßgebliche Darstellung, auf die sich seither alle beziehen.*

**[13]** Graefe, G.: *Modern B-Tree Techniques*. Foundations and Trends in Databases 3(4), 2011.

*Fünfzig Jahre Verfeinerungen des B-Baums an einer Stelle gesammelt: Sperren, Wiederanlauf, Verdichtung, Massenladen. Zeigt, wie viel in einer scheinbar abgeschlossenen Struktur noch stecken kann.*

**[14]** Gray, J., Reuter, A.: *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann 1993.

***Das Standardwerk zu Transaktionen.*** *Über tausend Seiten aus jahrzehntelanger Praxis, mit vielen Anekdoten und Betriebszahlen. Gray hat das Gebiet geprägt wie kaum ein anderer.*

**[15]** Härder, T., Reuter, A.: *Principles of Transaction-Oriented Database Recovery*. ACM Computing Surveys 15(4), 1983.

*Der Aufsatz, in dem das Kürzel ACID geprägt wurde. Ordnet die Wiederanlaufverfahren systematisch und benennt die Zusagen so klar, dass die Begriffe seither unverändert benutzt werden.*

**[16]** Hellerstein, J., Stonebraker, M., Hamilton, J.: *Architecture of a Database System*. Foundations and Trends in Databases 1(2), 2007.

*Ein Überblick über den Gesamtaufbau, von der Verbindungsverwaltung bis zum Speicher. Die beste Antwort auf die Frage, wie die Teile zusammenhängen. Etwa hundert Seiten, frei verfügbar.*

**[17]** Jégou, H., Douze, M., Schmid, C.: *Product Quantization for Nearest Neighbor Search*. IEEE PAMI 33(1), 2011.

*Verdichtet Vektoren um Faktor dreißig und macht das Vergleichen dabei schneller, weil aus Rechnungen Tabellenzugriffe werden. Grundlage jeder Vektorsuche im großen Maßstab.*

**[18]** Kleinberg, J.: *Authoritative Sources in a Hyperlinked Environment*. Journal of the ACM 46(5), 1999.

*Das Verfahren HITS, gleichzeitig und unabhängig von PageRank entstanden. Unterscheidet zwischen Autoritäten und Verteilern und ist mathematisch eleganter, praktisch aber schwerer anzuwenden, weil es je Anfrage neu gerechnet werden muss.*

**[19]** Malkov, Y., Yashunin, D.: *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. IEEE PAMI 42(4), 2018.

*Das Verfahren, das die Vektorsuche praktikabel gemacht hat. Verbindet den Gedanken der Sprunglisten mit dem der Kleine-Welt-Graphen und erreicht logarithmische Suchzeit bei hoher Trefferquote.*

**[20]** Manning, C., Raghavan, P., Schütze, H.: *Introduction to Information Retrieval*. Cambridge University Press 2008.

***Die maßgebliche Einführung ins Retrieval.*** *Frei im Netz verfügbar. Behandelt Indexbau, Verdichtung, Rangfolge, Auswertung und Klassifikation, durchgehend klar und mit vielen Beispielen.*

**[21]** Mohan, C., Haderle, D., Lindsay, B., u. a.: *ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks*. ACM TODS 17(1), 1992.

*Das Wiederanlaufverfahren, das Standard geworden ist. Der Aufsatz ist lang und detailliert; wer die drei Durchgänge und die Ausgleichssätze verstanden hat, hat den Kern.*

**[22]** O'Neil, P., Cheng, E., Gawlick, D., O'Neil, E.: *The Log-Structured Merge-Tree*. Acta Informatica 33(4), 1996.

*Eine Struktur für schreiblastige Lasten: Änderungen sammeln, sortieren, in Stufen zusammenmischen. Damals wenig beachtet, seit etwa 2008 in fast jedem verteilten Speichersystem.*

**[23]** Ramakrishnan, R., Gehrke, J.: *Database Management Systems*. 3. Aufl., McGraw-Hill 2003.

*Ein ausgewogenes Lehrbuch, das Sprache und Umsetzung gleichermaßen behandelt. Gut geeignet, wenn man beide Seiten gleichzeitig lernen will.*

**[24]** Robertson, S., Zaragoza, H.: *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval 3(4), 2009.

*Die Herleitung von BM25 aus dem wahrscheinlichkeitstheoretischen Ansatz, geschrieben von einem der Urheber. Erklärt, warum die Formel so aussieht, wie sie aussieht, und was die Parameter bedeuten.*

**[25]** Salton, G., Wong, A., Yang, C. S.: *A Vector Space Model for Automatic Indexing*. CACM 18(11), 1975.

*Das Vektorraummodell. Der Gedanke, Dokumente als Punkte in einem Raum aufzufassen und Ähnlichkeit als Winkel zu messen, trägt fünfzig Jahre später die Vektorsuche mit Einbettungen.*

**[26]** Selinger, P., Astrahan, M., Chamberlin, D., u. a.: *Access Path Selection in a Relational Database Management System*. SIGMOD 1979.

***Die Arbeit, die das relationale Modell praktikabel machte.*** *Kostenmodell, Statistiken, dynamische Programmierung über Verbundreihenfolgen. Jeder heutige Optimierer folgt diesem Aufbau.*

**[27]** Stonebraker, M., Madden, S., Abadi, D., u. a.: *The End of an Architectural Era*. VLDB 2007.

*Die These, die klassische Architektur stamme aus den 1970er Jahren und passe nicht mehr. Überzogen und einflussreich: Ein erheblicher Teil der Entwicklung der folgenden fünfzehn Jahre lässt sich als Antwort darauf lesen.*

**[28]** Zobel, J., Moffat, A.: *Inverted Files for Text Search Engines*. ACM Computing Surveys 38(2), 2006.

*Alles über invertierte Indizes: Aufbau, Verdichtung, Anfragebearbeitung, Aktualisierung. Sehr gründlich und mit vielen Messwerten. Die Ergänzung zu Manning und Mitautoren, wenn es um die Umsetzung geht.*

### Wenn Sie nur drei Bücher kaufen

| | | |
|---|---|---|
| **[11]** | **Garcia-Molina u. a.** | Zum Lernen der Innereien. Vollständig, mit durchgerechneten Kostenmodellen |
| **[20]** | **Manning u. a.** | Zum Retrieval. Frei verfügbar und durchgehend klar |
| **[14]** | **Gray/Reuter** | Zum Nachschlagen bei Transaktionen. Aus jahrzehntelanger Praxis |

*Wer danach noch etwas möchte: [7] Codd und [26] Selinger sind zusammen dreißig Seiten und erklären, warum dieses Gebiet so aussieht, wie es aussieht. [16] Hellerstein und Mitautoren geben in hundert Seiten den besten Überblick über den Gesamtaufbau.*